

**DigiDine: Developing an Ordering Management System for 28 Treasures HK
Roast + Dimsum-Makati**

**A Capstone Project Proposal Presented to the Faculty of the Information and
Communications Technology Program STI College Cubao**

**In Partial Fulfilment of the
Requirements for the Degree Bachelor
of Science in Information Technology**

**Carille B. Pabiona
Ricky Z. Roldan
Jecka Vien R. Laguerta
Jan Gabrielle M. Intod**

October 19, 2025

ENDORSEMENT FORM FOR FINAL DEFENSE

TITLE OF RESEARCH: **DigiDine: Developing an Ordering Management System
for 28 Treasures HK Roast + Dimsum-Makati**

NAME OF PROPONENTS: Carille B. Pabiona
 Ricky Z. Roldan
 Jecka Vien R. Laguerta
 Jan Gabrielle M. Intod

In Partial Fulfilment of the Requirements
for the degree Bachelor of Science in Information Technology
has been examined and is recommended for Oral Defense.

ENDORSED BY:

Ms. Mheldred M. Halichic, LPT
Capstone Project Adviser

APPROVED FOR FINAL DEFENSE:

Engr. John Ivan Bachiller
Capstone Project Coordinator

NOTED BY:

Mr. Brian H. Calata, LPT
Program Head

October 19, 2025

APPROVAL SHEET

This capstone project proposal, titled: “**DigiDine: Developing an an Ordering Management System for 28 Treasures HK Roast + Dimsum-Makati**” was prepared and submitted by **Carille B. Pabiona, Ricky Z. Roldan, Jecka Vien R. Laguerta,** and **Jan Gabrielle M. Intod**, in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology, has been examined and is recommended for acceptance and approval.

Ms. Mheldred M. Halichic, LPT
Capstone Project Adviser

Accepted and approved by the Capstone Project Review Panel in
partial fulfillment of the requirements for the degree of
Bachelor of Science in Information Technology

Mr. Jonathan Mañago
Panel Member

Ms. Mary Joy Banal
Panel Member

Mr. Erwin L. Mendoza
Lead Panelist

Noted:

Engr. John Ivan Bachiller
Capstone Project Coordinator

Mr. Brian H. Calata, LPT
Program Head

October 19, 2025

ACKNOWLEDGEMENT

We, the researchers, would like to express our sincere gratitude to everyone who has supported and guided us throughout the completion of our Capstone Project entitled “DigiDine: A Mobile Ordering System for 28 Treasures HK Roast + Dimsum – Makati.”

First and foremost, we would like to extend our deepest appreciation to our Capstone Project Adviser, Ms. Mheldred M. Halichic, LPT, for her continuous guidance, valuable insights, and unwavering support. Her encouragement and expertise have been a great help in completing this study successfully.

We also express our heartfelt thanks to our Capstone Project Review Panel Members—Mr. Jonathan Mañago, Ms. Mary Joy Banal, and Mr. Erwin L. Mendoza, Lead Panelist—for their time, constructive feedback, and meaningful suggestions that helped us improve our work.

Our gratitude also goes to Engr. John Ivan Bachiller, Capstone Project Coordinator, and Mr. Brian H. Calata, LPT, Program Head, for their guidance, understanding, and support during the entire process of this research.

We would like to extend our deepest appreciation to our parents for their endless love, sacrifices, and motivation that inspired us to strive harder; to our friends for their encouragement, patience, and companionship throughout our journey; and to our loved ones for their understanding and support, which gave us the strength to persevere. Above all, we offer our sincerest gratitude to Almighty God for granting us wisdom, guidance, and blessings that made this achievement possible.

ABSTRACT

Title of research: **DigiDine: Developing an Ordering Management System for 28 Treasures HK Roast + Dimsum-Makat**

Researchers: **Carille B. Pabiona**
Jeck Vien R. Laguerta
Ricky Z. Roldan
Jan Gabrielle M. Intod

Degree: **Bachelor of Science in Information Technology**

Date of Completion: **2025**

Keywords: **Mobile Ordering System, Point of Sale (POS), Kitchen Display System (KDS), .NET MAUI Blazor, Restaurant Automation, Customer Satisfaction, Prototype Method, Local Access Point (AP), Digital Menu, Order Management System**

This study presents the development of DigiDine, a mobile ordering system designed for 28 Treasures HK Roast + Dimsum – Makati to improve its manual and paper-based ordering process. The traditional method led to long wait times, miscommunication, and order inaccuracies that affected customer satisfaction and operational efficiency. DigiDine aims to address these issues by integrating a Menu Application, Point of Sale (POS), and Kitchen Display System (KDS) into one unified digital platform.

The system allows customers to browse a digital menu, customize orders, communicate with staff through in-app messaging, and track order status in real time. It operates through a local access point (AP) network, enabling reliable offline functionality. Developed using the Prototype Method and built with .NET MAUI Blazor, DigiDine underwent iterative testing and refinement based on feedback from restaurant staff and management.

Results showed significant improvements in service speed, order accuracy, and communication between customers and staff. The system successfully digitized the restaurant's workflow, enhancing overall efficiency and customer experience. Future enhancements may include online payments, AI-driven recommendations, and multi-branch scalable

TABLE OF CONTENTS

	Page
Title Page	i
Endorsement form for Final Defense	ii
Approval Sheet	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
Introduction	1
Project Context	1
Purpose and Description	2
Objectives	4
Scope and Limitation	6
Review of Related Literature/Studies/Systems	9
Related Literature	9
Related Studies and/or Systems	10
Synthesis	12
Technical Background	14
Overview of Current Technologies to be Used in the System	14
Calendar of Activities	15
Resources	18
Methodology	21
Requirements Analysis	23
Requirements Documentation	25
Design of Software, System, Product, and/or Processes	42
Development	59
Results and Discussion	62
Testing	62
Description of Prototype	62
Implementation Plan	63
Implementation Results	64

Conclusion	65
References	66
Appendices	69
Appendix A: Resource Persons	69
Appendix B: Relevant Source Code	70
Appendix C: Evaluation Tools/Test Documents	86
Appendix D: Sample Input/Output/Reports	88
Appendix E: User's Guide	103
Appendix F: Personal Technical Vitae	109

INTRODUCTION

Project Context

Customer satisfaction and efficiency are the keys to the hospitality and restaurant business. Many restaurants face challenges in handling customer orders efficiently, particularly in restaurants with various locations like indoor dining, outdoor seating, or private rooms. Traditionally, the ordering process of a restaurant is that customers must go to the cashier, call a staff member, or wait for the service just to place an order (Statista, 2023). This process can be frustrating and time-consuming, leading to service delays, miscommunication, and incorrect orders. These inefficiencies negatively impact the overall dining experience and customer satisfaction.

According to the developer's interview, 28 Treasures HK Roast + DimsumMakati is a Chinese restaurant that offers dine-in and outdoor seating. 28 Treasures HK Roast + Dimsum-Makati's current ordering process in the restaurant is manual, where the customers will call the staff to tell them their orders, and the staff will write their orders using paper and pen. After collecting orders, the staff gives the list to the kitchen staff so that the restaurant's chef will know what to prepare after ordering. Lastly, the receipt will be provided after their meal, requiring them to pay for the food they consumed. This manual process may lead to inefficiencies in the service industry. Many customers tend to experience challenges ordering food and beverages because of long waiting times and the unavailability of staff. Moreover, miscommunication between restaurant staff and customers occasionally leads to misplaced or untimely orders, impacting customer satisfaction and operational efficiency (HospitalityNet, 2022)

To solve the issue, 28 Treasures HK Roast + Dimsum-Makati is suggested to create a mobile app ordering service. Customers can order directly from their allocated seats through the mobile app without leaving or calling a staff member. The app will also integrate flawlessly with the Point of Sale (POS) and Kitchen Display (KD) systems to process and deliver orders efficiently.

The app will seek to update the restaurant's service strategy through technology to provide guests with increased convenience and satisfaction. It will include real-time order tracking, digital menus, a best-selling suggestion feature, and an in-app messaging feature for easy interaction with the staff. The system will also include an Access Point (AP) to ensure orders are accepted within the vicinity, regardless of network availability (online or offline). 28 Treasures HK Roast + Dimsum-Makati can use this system to enhance operational efficiency, minimize human error, and enhance the overall customer experience.

Purpose and Description

The project aims to create a mobile ordering app for 28 Treasures HK Roast + Dimsum-Makati. The app will make an easy ordering process for customers seated in dine-in and outdoor areas, giving them a convenient and efficient experience.

The mobile ordering app is an innovative and exclusive solution that eliminates the need for guests to place orders with employees, thus minimizing wait times, long queues, and other service inefficiencies. Customers can view a digital menu, see item descriptions and prices, add special orders and instructions, and order directly from their mobile devices.

In addition, guests can enjoy the ordering experience throughout their stay at the restaurant. They can order multiple times at their convenience without settling each transaction immediately. Instead, all orders are recorded, and the final bill will be provided when they decide to bill out, ensuring a hassle-free and convenient payment system.

The system also ensures the timely delivery of orders to the correct destination by assigning a unique table ID corresponding to the seating area. When the "Start Ordering" button is clicked, the system automatically identifies the source table of the order. Using the app, it can accurately track their location so that food and drinks are delivered directly to the right table of the customers. The

system incorporates an in-app chat feature so the customers can communicate directly with the pos and kitchen staff. To resolve connectivity concerns, the system will use an access point device to ensure orders are received even in areas of poor network connectivity.

To further improve the customer experience, tablets are recommended for each table in the restaurant. To avoid loss or theft, the tablets will be firmly fixed to the tables through anti-theft devices such as locking stands or cable locks. This setup ensures the availability of tablets to enhance customer interaction while maintaining security and efficiency in the order process. In addition, the tablets will have a constant power supply via a wired charging system integrated with the tablet's design, ensuring uninterrupted use without running out of battery.

For take-out orders, customers who did not place their orders through the application shall proceed directly to the cashier's POS located inside the restaurant. This system ensures an efficient transaction flow for walk-in customers while maintaining proper organization between app-based and instore orders.

This study establishes the app as a valuable enhancement to customer satisfaction and business efficiency for 28 Treasures HK Roast + DimsumMakati.

Objectives

This study aims to develop a mobile ordering platform for 28 Treasures HK Roast + Dimsum-Makati to take its ordering process into the digital era. The process will be an effortless, automated system that reduces manual workload, minimizes errors, and maximizes operational efficiency. Digital menus, realtime monitoring, and a custom-designed POS and kitchen display system will maximize employee performance, order accuracy, and service operations, thereby maximize customer satisfaction, and create a more efficient and streamlined order management process.

Statement of the Problem

The current ordering process at 28 Treasures HK Roast + Dimsum-Makati is manual, and the customers are obliged to call out to the staff, who write down the orders on paper and then relay the orders to the kitchen. This traditional method has inefficiencies that negatively impact operational productivity and customer satisfaction. Customers are typically exposed to waiting for extended hours due to excess demand and a lack of staff members. Furthermore, miscommunication of orders and delays in operations result from human error in taking orders and processing. Additionally, a lack of real-time tracking of orders makes it hard for customers to determine the status of their orders. The absence of a digital order system compromises the efficiency and ability of the restaurant to handle peak dining hours.

Specific Problem

1. How will the system minimize the time customers wait and efficiently improve order processing?
2. How can the system minimize human errors in ordering and ensure order accuracy?

3. How can the system facilitate real-time order tracking for customers and restaurant staff?
4. How would the system ensure communication and order processing through a local network access point without relying on an internet connection?
5. How can a custom POS and kitchen display system improve the operational efficiency of 28 Treasures HK Roast + Dimsum-Makati Point-of-sales (POS) and Kitchen Display System (KDS)?

General Objectives

This research aims to create a mobile ordering system that maximizes ordering efficiency, accuracy, and customer service at 28 Treasures HK Roast + Dimsum-Makati. The system must also resolve typical issues, such as long waiting times, miscommunication, and operational inefficiencies, through advanced technological features, including real-time order tracking, instant communication, and POS integration. The project will modernize the restaurant's operations for the digital era and enhance the overall dining experience.

Specific Objectives

1. To develop and deploy a mobile ordering system that reduces customer wait times by enabling customers to order from the convenience of their table.
2. To create a user-friendly interface that guides customers and staff to improve its use and operational efficiency.
3. To add real-time order-tracking capability that allows customers and staff to monitor order status and movement.
4. To implement a stable and secure local access point system that ensures connectivity for order transactions with or without an internet connection.

5. To design and implement a custom-fit POS and kitchen display system that modernizes the ordering process, improves staff communication, and increases restaurant operations.

This approach will provide 28 Treasures HK Roast + Dimsum-Makati with a practical solution to these issues, ultimately improving restaurant efficiency and customer service.

Scope and Limitations

Scope:

- The DigiDine app is built exclusively for 28 Treasures HK Roast + Dimsum in Makati City.
- The app will make it easier to manage food and drink orders for dine-in and outdoor customers with real-time order tracking and notifications for better service.
- The mobile application will only be available on Android tablets provided by the restaurant.
- The mobile application will connect to its Access Point, ensuring the order is accepted within the vicinity regardless of network availability (online or offline).
- The system will support a "Bill Out" feature, allowing customers to notify staff when the payment process can be initiated.
- Some of the features include:
 - Providing a Digital menu in the app with item descriptions, prices, and customization.
 - Location detection for correct order delivery: Each table has its unique table ID. Once the table is booked, the system automatically links the tablet to the corresponding table number, allowing the staff and kitchen to accurately determine the customers' location for correct order delivery.
 - Best-selling feature
 - In-app messaging for communication with the POS.
 - Integration with the restaurant's POS and kitchen display system.
- Secure tablets attached to tables using anti-theft mounts or locking stands to prevent loss or theft.

- Continuous power supply for tablets through wired charging stations in the tables to ensure uninterrupted service.
- Allowing customers to select spicy levels (mild, medium, hot).
- Providing options for protein type (chicken, beef, pork, seafood, vegetarian).
- Displaying status time indicators (e.g., “10 min prep,”) track order progress.
- Sending predefined messages (e.g., “Need Assistance,” “Need Utensils,” “Extra Tissue”, “More Ice”) for faster communication)
- **Technical Requirements:**

Requirement	Details
• Device	Intel VistaTab30
• OS	Android 13
• RAM	4GB
• Storage	128GB (expandable with microSDXC)
• Connectivity	Local Access Point (AP)
• Display	11" IPS LCD, 1200x1920 resolution
• Processor	Octa-core Unisoc T606
• Battery	7000mAh with a wired tablet charging setup

Limitations:

- The application will be accessible only to 28 Treasures HK Roast + Dimsum dinein customers and will not accommodate external users.
- The application will not be available on the iOS operating system.
- The application will not assess the speed of food preparation
- The application will not evaluate the quality of the food.
- It will be limited to food and beverage ordering and will not cover other restaurant services like table reservations or event bookings.
- Maintenance and software updates will be regularly required to guarantee maximum performance and user satisfaction.
- Spicy levels and protein type are limited to predefined categories; custom requests outside these are not fully supported.

- Predefined messages are restricted to standard responses and may not address all customer concerns.
- Status time relies on staff maximum preparation that chefs prepared per food, which may not always match real-time conditions

REVIEW OF RELATED LITERATURE/SYSTEMS

Related Literature

Foreign Literature

Shahril et al. (2021) investigated the impact of self-service kiosks (SSKs) on customer satisfaction in Quick Service Restaurants (QSRs) in Klang Valley. According to their study, the speed of ordering impacts customer satisfaction significantly, with slower waiting times causing frustration to the customers. Digital ordering solutions, such as DigiDine, provide real-time order tracking and immediate confirmation, enhancing customer experience and reducing uncertainty. The study also noted the importance of convenience in customer adoption of self-service technologies. By facilitating customers to order straight from their mobile phones, DigiDine eliminates the need for manual taking of orders, supporting evidence that technology should reduce service delays and offer an uninterrupted ordering experience.

According to Dolan and Widayanti (2022), authentication systems for hotspot network users enhance network security. Their research analyzed different authentication methods to prevent unauthorized access and improve data integrity. These findings highlight the importance of secure digital infrastructures, which are critical in digital ordering systems like DigiDine and ensure customer data privacy and secure transactions.

Nguyen et al. (2024) proposed a personalized product recommendation model for ecommerce, utilizing advanced retrieval strategies to enhance recommendation accuracy. The model analyzes user behavior and preferences to deliver relevant product suggestions. Such innovations in recommendation algorithms can be integrated into digital ordering platforms like DigiDine to provide personalized menu suggestions, improving customer engagement and satisfaction

Local Literature

Tamondong et al. (2022-2023) assessed self-service kiosks as a food service tool in fast-food chains in Manila. Their study explored consumer satisfaction

and operational efficiency, revealing that while SSKs reduce service time, their implementation faces challenges such as cost and consumer acceptance. These insights can inform the adoption of digital ordering systems like DigiDine, ensuring a balance between technological convenience and consumer readiness. Borbon (2025) discussed the transformation of the fast-food industry in the Philippines due to self-service kiosks. The study traced the evolution of these kiosks, highlighting their impact on customer experience and operational efficiency. The discussion also covered challenges related to technology adoption in developing economies, which are relevant when implementing digital ordering solutions in similar markets.

Castillo et al. (2020) introduced an automated drive-through ordering system aimed at improving fast-food restaurant operations. Their findings suggest that automation significantly enhances service efficiency, minimizing order errors and reducing wait times. These insights reinforce the benefits of integrating digital ordering platforms like DigiDine to optimize order management and customer satisfaction.

Related Studies

Foreign Studies

Yong (2022) examined determinants of consumers' self-ordering kiosk intention at restaurants in terms of ease of use, information clarity, perceived service quality, and freedom of choice. According to the researchers, a good ordering system must highlight easy-to-use interfaces and real-time order tracking. The findings are directly applicable to DigiDine as they highlight areas to improve user experience in digital food ordering systems.

Taylor (2020) examined mobile food-ordering apps among college students and concluded that convenience, ease, and technology trust were the determinants. Using the Technology Acceptance Model (TAM), the study highlights user trust in online ordering systems as the central theme. The findings are crucial when creating DigiDine with simple transactions and improved security features.

Sam et al. (2023) studied the implementation of a mobile ordering system for the F&B industry. Their study depicted how mobile ordering systems make services more efficient, reduce waiting times, and improve customer satisfaction. By automating, food service operations could be streamlined and customer experience improved using mobile ordering systems like DigiDine.

Shah and Rai (2022) discussed the effect of customer feedback on business performance. Customer feedback mechanisms enhance customer retention and service quality, their research indicated. Integration of feedback mechanisms in DigiDine would assist in gaining insight into customer choice and enhancing services continuously.

Local Studies

Yeung et al. (2023) developed a POS-integrated object detection system for bakeries to enhance business efficiency. Their study established the effectiveness of AI-based object detection in reducing errors and ensuring maximum order accuracy. The technology can be integrated into DigiDine to enhance its ordering and checkout functions.

Buenaventura et al. (2021) created a mobile ordering app for fast-food chains to simplify the ordering process and enhance service efficiency. Their research revealed that mobile ordering apps greatly minimize wait times and maximize order management. The success of their mobile app demonstrates that innovations such as DigiDine can efficiently solve service inefficiencies in the restaurant sector.

De Gabriel et al. (2024) investigated improving customer experience in Philippine food courts with an integrated self-service ordering system. From their findings, the ability of digital ordering systems to enhance the speed of service, lower the cost of operation, and improve customer satisfaction is established. This establishes the viability of implementing DigiDine to improve the dining experience in fast-food restaurant settings.

Synthesis

The literature and supporting studies highlight the increasing application of digital solutions in the food service industry, i.e., in mobile apps and self-service ordering. Foreign and local studies are equally concerned with technology adoption for improved customer satisfaction, wait time, and operational effectiveness. Shahril et al.'s (2021) and Tamondong et al.'s (2022, 2023) studies demonstrate that the process of self-service kiosks significantly enhances ordering efficiency and customer experience by cutting down waiting times. Similarly, Borbon (2025) discusses the effects of kiosk technology on the fast-food industry in the Philippines, where it is used extensively and functions seamlessly.

The foreign indeed validates the importance of digital solutions in managing food service. For example, Yong (2022) and Taylor (2020) proved that user-friendliness, perceived quality of the services, and consumer trust are the main points in the adoption of kiosks for self-ordering and mobile food-ordering applications. In addition, Nguyen et al. (2024) explored how personalized product recommendations boost the user experience in e-commerce, and the online store food can adopt this process. The role of secure authentication in digital platforms will be particularly important in securing the history of transactions in the online food ordering system, as evidenced by Dolan and Widayanti (2022).

Studies like Castillo et al. (2020) and Buenaventura et al. (2021) were conducted locally to support the view that mobile order applications improve customer experience and work efficiency in fast-food restaurants. De Gabriel et al. (2024) said that the benefits of having self-service ordering systems in food courts in the Philippines are customer flow and service speed improvements. In addition to this, Yeung et al. (2023) demonstrated how AI-powered object detection at point-of-sale systems can optimize transaction processes, minimize human errors, and increase accuracy.

The overall outcome from the literature studies indicates that digital ordering systems installed in self-service kiosks, mobile applications, or AI-enhanced solutions play an important role in customer satisfaction and business efficiency. Such results support developing innovative ordering solutions identified for modern consumer demands, highlighting the need for userfriendly interfaces and secure authentication with relevant personalization.

TECHNICAL BACKGROUND

Overview of Current Technologies to be Used in the System

The mobile ordering system for 28 Treasures HK Roast + Dimsum-Makati will employ the most advanced and reliable technologies to convert its current manual process into a streamlined digital solution. The core of the system is a custom-built Android-based mobile application designed for tablets that will enable customers to view a digital menu, order, and interact directly with restaurant personnel. The application will be run on Itel VistaTab30 tablets with Android 14 OS, performance and reliability enhanced with a quadcore processor, 4GB RAM, and 128GB expandable storage.

To ensure orders are processed correctly, the solution will be linked to a Point of Sale (POS) and Kitchen Display System (KDS). This integration sends customer orders instantly to the kitchen and cashier, preventing the use of paper notes and the possibility of errors. Orders are associated with a unique table ID, helping to deliver them to the proper destination, in or out. The electronic interface further accommodates special requests via the note to staff feature in the Add to Cart page and best-seller suggestions to engage customers and maximize opportunities for sales.

The system allows customers and employees to see the status of each order in real-time. There is also a messaging feature in the app that allows customers and employees to communicate more effectively, which reduces the necessity for face-to-face interaction and allows for instant response to inquiries or problems. To make the service always seamless, the app uses a local Access Point (AP) that offers a strong and secure network connection even without the internet. This setup allows for the quick processing of orders within the restaurant despite connectivity problems. The tablets will be safely locked to anti-theft stands or cable locks, and there will be wired power for all the tablets for continuous charging. The arrangement ensures that every customer has a functioning device to utilize while the customer is dining. The tablets are locked to their respective tables by unique table IDs, which enable the system to identify where every order originates for efficient and timely service.

The proposed solution uses mobile technology, local network communication, and smooth connection with POS and kitchen systems to improve how the business runs and make customers happier. This system shows what is popular in the food service industry today, like digital self-service, offline network support, and contactless ordering, making 28 Treasures HK Roast + Dimsum-Makati a modern and customer-focused place.

Calendar of Activities

- **Brainstorming:** The researchers collected and compared potential capstone titles and which client to select.
- **Searching for a Client:** The team identifies potential local restaurants that could benefit from a digital solution. This includes evaluating their current processes and service gaps, and eventually selecting 28 Treasures HK Roast + Dimsum-Makati as the client.
- **Interviewing:** After searching for a client, the researchers penned down questions regarding the issues that 28 Treasures HK Roast + Dimsum is currently facing and reviewed interview answers to determine the client's needs and feature requests.
- **Data Collection:** Collect interview findings answered by the client to develop concepts on users' preferences, features, and design of DigiDine.
- **Requirement Gathering:** Review and summarize the outcomes of brainstorming sessions and interviews, and determine which application and programming languages the researcher will employ.
- **Writing Documentation:** We will begin writing things down after we have brainstormed and done interviews. We will begin by thinking about potential final titles. Then we will write Chapters 1 and 2, editing between chapters. This also involves searching for potential links for Chapter 1 relevant studies and literature.
- **Prototyping:** Developing designs and sketches of how things will function in DigiDine to show the app layout, user interface, and overall flow before building it. These prototypes were reviewed and revised to reflect client feedback and usability expectations.

- **Developing:** The researchers began partial development of DigiDine, implementing essential modules such as the digital menu, order cart, and order tracking using C#, .NET MAUI Blazor, and APIs required for connectivity and POS integration.
- **Testing and Debugging:** The developing system are tested for functional errors, UI issues, and data integrity, especially in offline/online switching and real-time updates.
- **System Completion:** Final optimization of code, security integration (tablet anti-theft setup), and full feature inclusion are completed.
- **Deployment:** The DigiDine application is loaded on restaurantprovided Android tablets and interconnected with the Access Point and POS/KDS infrastructure.
- **Documentation Completion:** The Final chapters are done. They discuss system evaluation, testing results, conclusions, and recommendations.

The Gantt Chart presents a summary of activities. Listed are the activities, and opposite them are their duration or periods of execution.

.

Gant Chart of Activities

ACTIVITY	FEBURUARY				MARCH				APRIL				MAY				AUGUST				SEPTEMBER				OCTOBER				NOVEMBER			
MONTH																																
WEEKS	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Brainstorming																																
Searching For client																																
Interviewing																																
Data Collection																																
Requirement Gathering																																
<u>Writing Documentation</u>																																
Prototyping																																
Developing																																
Testing and Debugging																																
System Completion																																

Ongoing	
Completed	

Table Chart of Activities

Timeline (Months)	Timeline (Weeks)	Task / Activity
February	Week 1 – 2	Brainstorming
February – March	Week 3 – 4	Searching Client and Interview
March	Week 5	Data Collection
March	Week 6	Requirements Gathering
March – May	Week 6 – 14	Writing Documentation
April – May	Week 9– 14	Prototyping
May – November	Week 13 – 31	Developing
October – November	Week 25– 31	Testing and Debugging
October – November	Week 27 – 31	System Completion
November	Week 29 – 31	Deployment
November	Week 29 – 31	Documentation and Completion

Resources

Hardware:

Laptop 1:

- Windows 11 Home Single Language
- **Processor:** AMD Ryzen 5 5600H with Radeon Graphics 3.30 GHz
- **RAM:** 16GB
- **GPU:** AMD Radeon™ Graphics

Laptop 2:

- Windows 11 Home Single Language 64-bit
- **Processor:** Intel(R) Core (TM) i3-10110U CPU @ 2.10GHz (4 CPUs), ~2.6GHz
- **RAM:** 8GB
- **GPU:** Intel(R) UHD Graphics Family

Laptop 3:

- Windows 11 Home Single Language 64-bit
- **Processor:** Intel® Core™ i5-13500HX processor Tetradeca-core 2.50 GHz
- **RAM:** 8GB
- **GPU:** NVIDIA® GeForce RTX™ 4050 with 6 GB dedicated memory Android

Tablet (MENU ORDERING):

- **Brand Name:** Itel o Product Name: VistaTab 30
- **Model Name:** itel P10003L
- **Android Version:** 14
- **Chipset:** Unisoc T606 (12 nm)
- **CPU:** Octa-core
- **Storage:** 128GB, 4GB RAM

Android Tablet (POS)

- **Brand Name:** Huawei
- **Product Name:** Matepad PaperMatte Edition
- **Storage:** 256GB, 4GB RAM
- **CPU:** Qualcomm Snapdragon 7 Gen 1

Software:

- **NET MAUI (Multi-platform App UI):** This technology helps in creating apps that can run on other platforms, like the Itel VistaTab30 tablet that uses Android 13. MAUI is selected since it offers good performance and nearly the same user experience on various devices, like the tablet's 11.0-inch IPS LCD screen at a resolution of 1200 x 1920 pixels.

This guarantees that the app will run smoothly with the tablet hardware, giving users a seamless and responsive interface.

- **Canva:** It is a free web-based collaborative design and creation software for a visual communications platform for individuals to create professional images with ease. It will be utilized by researchers for posters, presentations, logos, and others.
- **Microsoft Office 365** is an online productivity suite with tools like Word, Teams, Outlook, OneDrive, and PowerPoint. For the Capstone project, it will be used for writing, file sharing, presentations, and team collaboration. Teams and Outlook will support effective communication and planning, enabling remote collaboration among team members.
- **GitHub** is an online software development platform. It allows developers to create, store, track changes, merge, and manage code and files. It also provides developers with the ability to collaborate and work in teams.
- **Google Forms:** It is a web-based, free tool to create quizzes or surveys. It shows results as they come in. This will be used to ask potential users about the application.
- **Draw.io** is a web application utilized for drawing diagrams that outline processes, systems, or workflows. It will be utilized for drawing, editing, and posting flowcharts that accurately show the steps or choices in the researcher's research process.
- **SQLite** is a lightweight, serverless database engine for local data storage. It will be utilized in the Capstone project to store and handle app data effectively in the app. Its ease of use and offline capability make it ideal for seamless integration and reliable performance.

METHODOLOGY

Prototype Method

The Prototype Approach facilitated the development of DigiDine, 28 Treasures HK Roast + Dimsum - Makati's mobile ordering system. The approach enables rapid modification, early user feedback, and continuous improvement of the system's features and functionality. It is aimed at developing functional versions of the app that iterate through user feedback.

1. Planning and gathering are necessary.

The first phase was to determine what stakeholders and restaurant staff needed. This was to be able to discover core things like seeing the digital menu, changing orders, seeing orders in real-time, compatibility with POS and KDS systems, and to be able to use it offline with the assistance of a local access point (AP). The planning phase helped ensure that the prototype addressed the real needs of the restaurant.

2. Building Simple Prototypes

To get a sense of the main features of DigiDine, initial wireframes were created with software such as Figma and by hand. The layouts covered the general organization and how individuals would navigate around, for instance:

- Room and table reservation
- Menu navigation
- Order cart interaction
- In-app messaging
- Checking order status

This allowed stakeholders to understand the design and functionality of the app easily.

3. In-Depth Interactive Prototyping

One part of the system was developed using .NET MAUI Blazor to create detailed, interactive models.

They were:

- Operational menu and category filtering
- Interactive shopping cart
- Order and status simulation submission
- Offline test access through local Wi-Fi

This prototype enabled stakeholders to see for themselves how the system operates on actual mobile devices.

4. User Testing and Feedback Collection

The management and team of the 28 Treasures saw the prototype for usability testing.

They offered comments on:

- Ease of navigation
- Accuracy of order flow
- Clarity of the visual interface
- Responsiveness to real-time messaging and tracking

All feedback and inputs were noted and categorized as priority and feasibility.

5. Step-by-Step Improvement

- We made a few minor changes according to the feedback we got. They were these:
- Improved visual hierarchy of menu items
- Core cart management
- Enhanced user triggers for online/offline status
- Improved chat support message notifications.

Every step led the prototype closer to the actual restaurant operation standards.

6. Proceeding to Final System Development

Once the stakeholders had signed off on the upgraded prototype, it was utilized to construct the whole system. The prototype was enhanced across all aspects to be deployable-ready modules with full backend integration (database, KDS, and POS).

7. Final Testing and Release

The working system was extensively tested, comprising: •

Unit ordering workflow and integration testing

- Restaurant staff acceptance testing.
- Offline/online mode network stress tests

After successful pilots, the DigiDine system was put into use in the day-to-day operations of the restaurant.

Requirements Analysis

To ensure the development of a relevant and functional system, the team conducted interviews and observations with the staff and management of 28 Treasures HK Roast + Dimsum-Makati. The goal was to identify key requirements for the mobile ordering system, which were categorized as follows:

Who

- Customers (Dine-in and Outdoor): Individuals who will place food orders using tablets at every table. They will utilize the menu on the digital screen, order food and beverages, converse via in-app messaging, and request the bill.
- Wait Staff/Service Staff: They assist customers with their inquiries via chat, serve orders correctly, and assist with billing or other service tasks.
- Kitchen Staff: Take orders directly through the Kitchen Display System (KDS) and cook meals to order.
- Cashiers/POS Operators: Employ the POS system to process payment, handle requests for bills, and finalize payment.

What

- Order Processing and Taking: The system does away with the traditional method of taking orders as it allows customers to select and transmit their orders directly from an electronic menu.
- Order Delivery Coordination: The orders get delivered automatically to the respective staff or kitchen as per the table ID.
- Real-Time Order Tracking: Employees and customers can see the status of the order during preparation and delivery.
- In-App Communication: Facilitates customers to communicate with employees, reducing the requirement for calls or visits.
- Billing and Payment Notice: Clients may request to bill out through the application, making the checkout more convenient.
- Menu Management: The restaurant can update the electronic menu, e.g., item description, prices, and best-seller status.

- System Administrator/Restaurant Manager: Manage system operations, change menu items, maintain tablets and technical settings in line, and ensure proper functioning.

Where

- Physical Environment: There are inside and outside seats in the 28 Treasures HK Roast + Dimsum-Makati restaurant.
- Device Environment: The system will be installed on IteL VistaTab30 Android tablets securely mounted on tables with anti-theft devices. The tablets will have fixed wired charging points.
- Network Environment: Functions through a Local Access Point (AP) in the restaurant to allow orders to be sent without the requirement for outside internet connectivity.

When

- Order Timing: At any moment, customers present on site without any assistance from personnel can place orders. They can place multiple orders simultaneously.
- Service Response Time: Real-time processing ensures minimum wait times for confirmation and kitchen preparation.
- Billing: Customers can pick when to get billed; all orders are combined and shown in one final receipt.
- System Availability: The application is readily available for use on the tablets. It does not need internet connectivity from the outside, so you can use it continuously during working hours.

How

- Current (Manual) System: Customers instruct waiters to deliver orders, which are put on paper. These are then carried to the kitchen to be prepared. The bill is handed over manually at the end of the meal. This process has some issues:
 - **Service Delays** are owing to a lack of staff or during peak hours.
 - **Order Inaccuracies** from miscommunication or illegible handwriting
 - **Limited transparency**, as customers are unable to track their orders in real-time.
 - **Operational inefficiency** happens because there is no automated system to coordinate between the service, kitchen, and billing.

- **Proposed System:** Converts manual tasks to a computerized one that minimizes waiting time, makes them precise, and ensures maximum customer satisfaction through a self-service model based on innovative technology (POS, KDS, APbased local network, Android tablets).

Requirements Documentation

According to the developer's interview and observation, the 28 Treasures HK Roast + Dimsum restaurant traditionally relied on a manual ordering process. Staff members wrote customer orders on paper and handed them to the kitchen for preparation. This system often resulted in long wait times, unavailability of staff, miscommunication, and incorrect orders, which negatively affected customer satisfaction.

To address these problems, the developers proposed an easy-to-use Mobile Ordering Management System called DigiDine. This digital solution simplifies the ordering process, reduces human error, and improves overall service efficiency. The system is composed of three core components: the Menu Application (Customer-Side), the POS Application (Cashier-Side), and the Kitchen Display System (KDS-Side). Together, these subsystems create an integrated platform that enhances communication, accuracy, and customer experience.

Customer-Side: Menu Application

The Menu Application is installed on designated tablets fixed to each dining table. The process begins with the Start Ordering Screen, which automatically links the tablet to its assigned table number. Customers then access the Menu Screen, where food items are categorized for easy browsing. Features such as a search bar, chat function, and simple navigation ensure a smooth and hassle-free user experience.

When selecting an item, customers are redirected to the Item Detail Screen, which displays product information, customization options, and price details. Customers can adjust quantities, select add-ons, and add the item to their Order Cart. The Order Cart

Screen consolidates all chosen items, allowing customers to review, modify, or add notes for special requests before confirming their order.

Once an order is placed, it is automatically transmitted to the POS and KDS systems. Customers may then track their order progress using the Order Status Tracking feature, which displays real-time updates such as Pending, Preparing, Ready, and Delivered. To further enhance transparency, the system also provides status time indicators (e.g., “10 minutes prep”), giving customers an estimate of waiting times.

Communication is facilitated through the Chat with POS Staff feature, which now includes predefined messages such as “Need Assistance,” “Please wait 5 minutes,” or “Order Ready for Pickup.” These options allow quick and standardized communication between staff and customers. Finally, the Bill Out Request feature enables customers to request their bill at any time, showing a breakdown of all accumulated orders for a seamless checkout process.

Cashier-Side: POS Application

The POS Application acts as the central hub for validating orders and managing payments. Orders placed from the table tablets are instantly displayed on the cashier’s terminal, complete with details such as table number, ordered items, and any special instructions.

For walk-in or takeout customers, the cashier encodes the orders directly into the POS system. The terminal categorizes items, computes the total bill, and presents the final amount due. Since the restaurant follows a cash-only policy, the cashier collects payment in cash and records the transaction in the POS. After payment, the POS generates an official receipt and simultaneously updates the system database, including sales reports and inventory levels.

The POS also processes Bill Out requests from customer tablets, ensuring quick and transparent billing. Additionally, the POS interface allows cashiers to view order status

time indicators and receive predefined messages sent by customers, improving efficiency in communication and response.

Kitchen-Side: Kitchen Display System (KDS)

The Kitchen Display System (KDS) is directly integrated with the POS to provide kitchen staff with real-time access to confirmed orders. Each order is displayed by table number along with the food items, quantities, and any special notes.

The KDS supports status time indicators, allowing kitchen staff to assign and adjust preparation times as needed. This ensures that customers receive accurate expectations regarding waiting times. As orders progress, staff can update their status to Preparing, Ready, or Delivered, which is reflected instantly on the customer tablets and the POS terminal.

To improve coordination, the KDS also enables staff to send predefined messages such as “Order Ready for Pickup”, which appear immediately on the customer’s device. By digitizing the kitchen workflow, the KDS reduces errors, eliminates reliance on paper tickets, and ensures that food preparation is timely and accurate.

Summary

The DigiDine Mobile Ordering Management System integrates the Menu Application, POS Application, and Kitchen Display System into a unified process. The inclusion of status time indicators and predefined messaging improves communication, transparency, and efficiency. This system reduces manual workload, minimizes errors, and enhances customer satisfaction by delivering a streamlined, cash-only, and fully digital order management process.

Wireframe

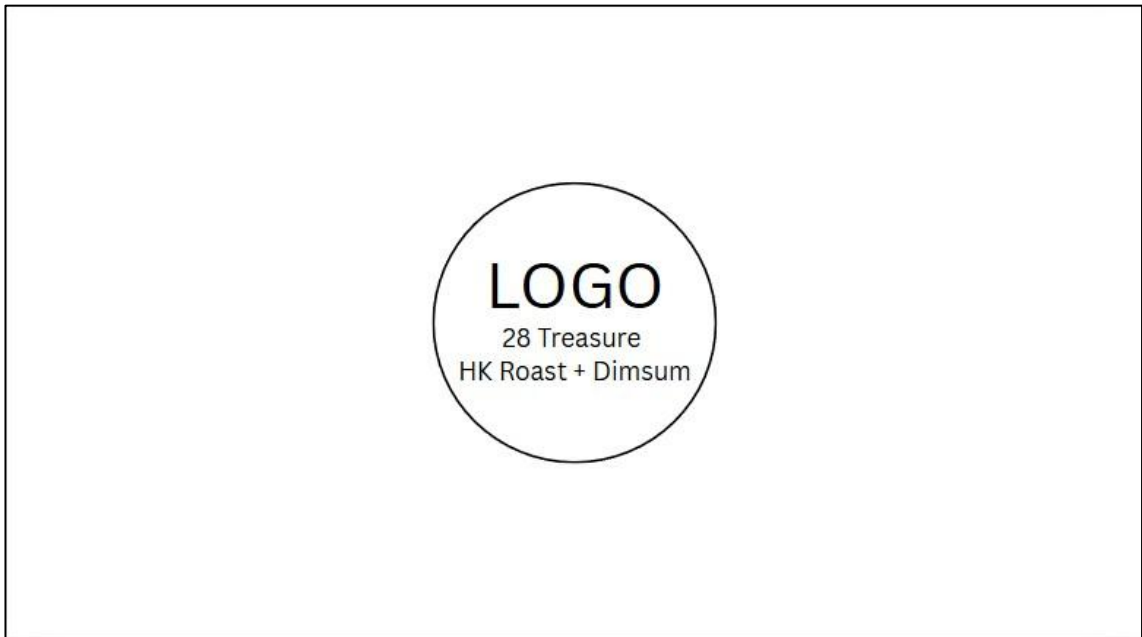


Figure 1. Splash Screen

Splash Screen

- Elements: Logo, restaurant name, and "Start Ordering" button.
- Purpose: Welcomes the user and begins interaction.
- Action: Click → Navigates to Main Menu Page.



Figure 2. Start Ordering Button

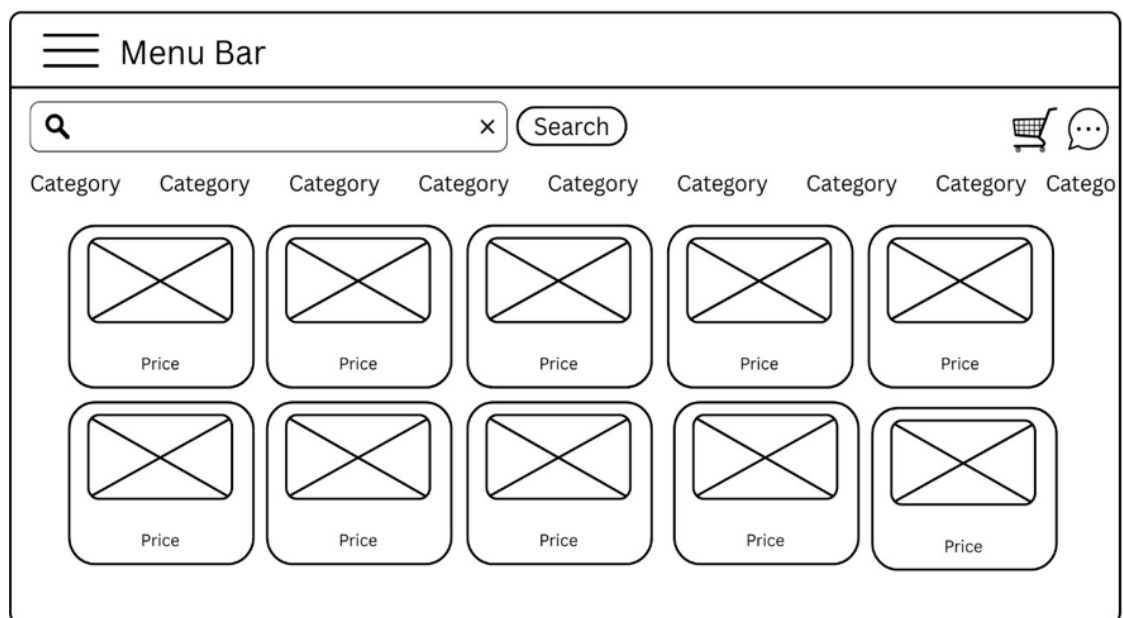


Figure 3. Menu Page

Menu Page Elements:

- Category Tabs (e.g., All, Drinks, Meals, Desserts)
- Search Bar

- Message Icon
- Cart Icon (with item count) **Action:**
- Click on a food item → Opens Food Detail Page
- Search → Filter results
- Click Cart → Opens Cart Page
- Click Message → Opens Message Box

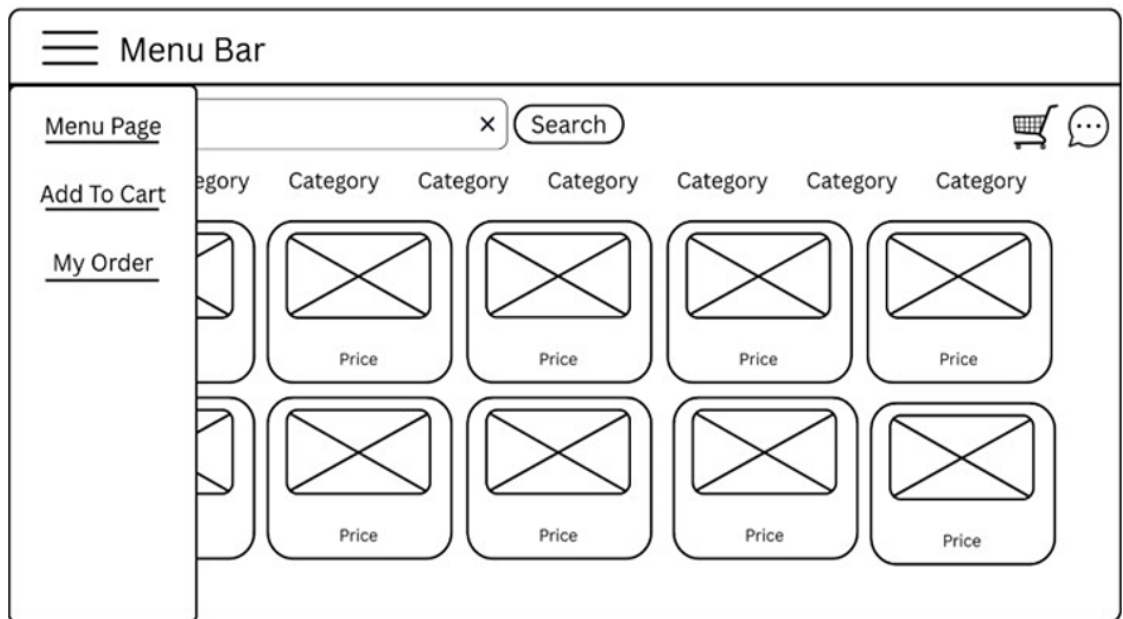


Figure 4. Navigation Bar Messages

Navigation Bar

Elements: Navigation Pages

- Purpose: Navigate Pages Easily

Action:

- Click Menu → Navigates to Main Menu Page.
- Click Add to Cart → Navigates to Add to Cart Page.
- Click My Order → Navigates to My Order Page.

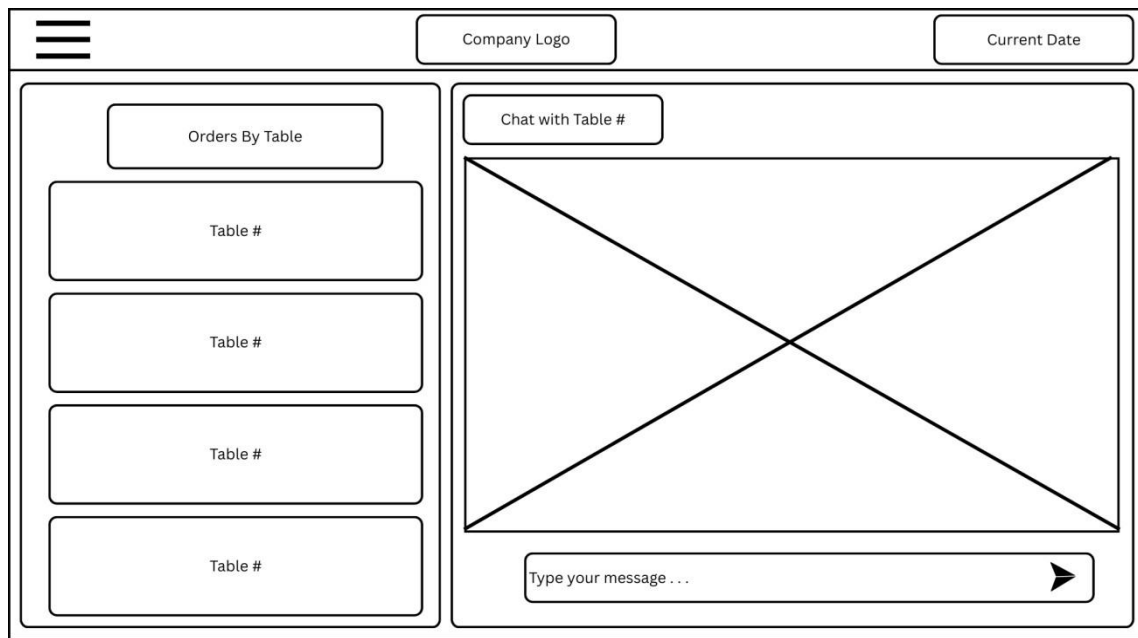


Figure 5. Messages

Elements: Chat Messages Pages

Action:

- View messages per table (e.g., Table 1, Table 2, Takeout, etc.)
- Click on a table to open that chat (conversation with that table)
- Include navigation actions like
 - Menu → go to Main Menu
 - Add to Cart → go to Cart page
 - My Orders → go to Order list
 - Messages → stay on the chat view

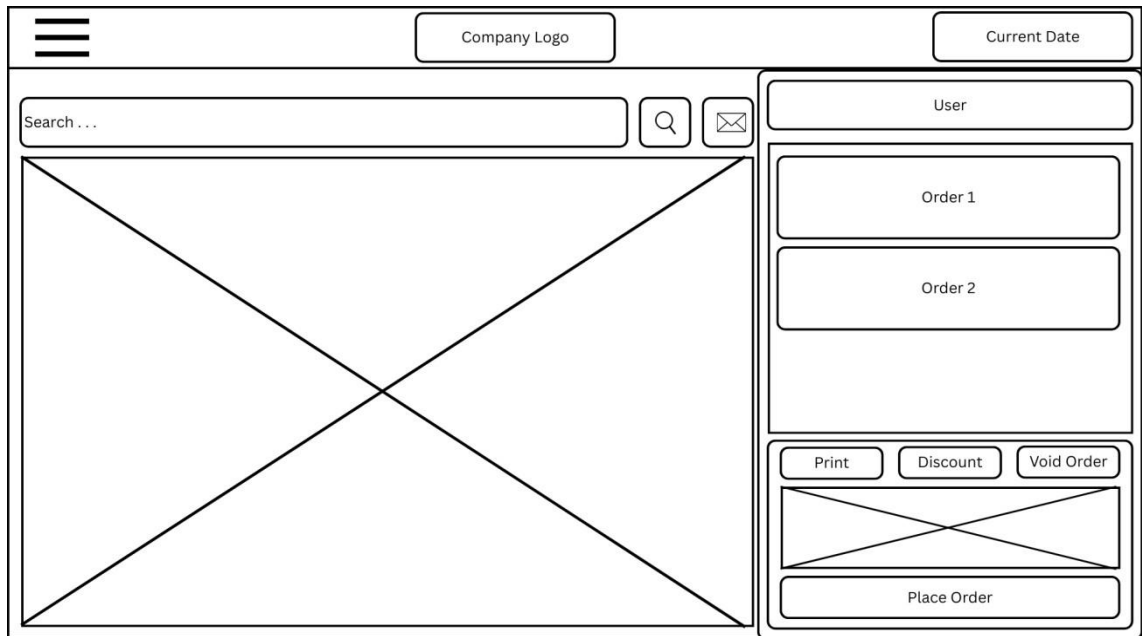


Figure 6. Show Detail Item

Food Detail Page Elements:

- Food Image
- Name, Description, Price
- Quantity Selector
- Add-ons (checkboxes for extras)
- “Add to Cart” Button

Action:

- Add to Cart → Item gets added and navigates back or stays for more edits

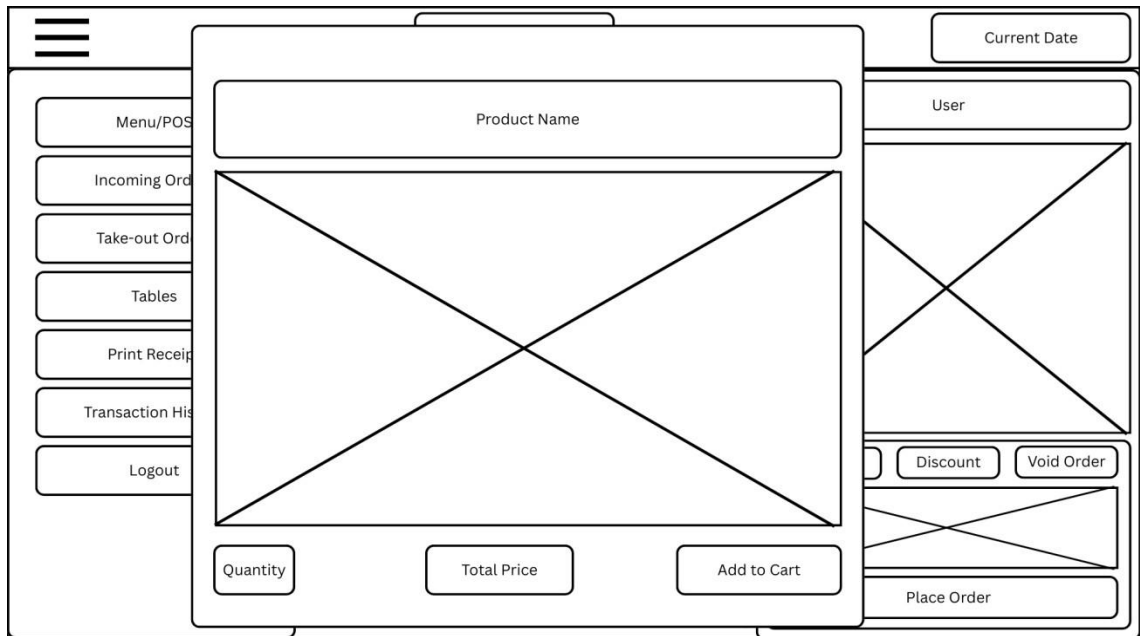


Figure 7. Add to Cart

Cart Page

Elements:

- List of selected items
- Total Price
- “Modify the Quantity” or “Remove” options
- Note to staff
- “Checkout” Button

Actions:

- Checkout → Navigates to Order Confirmation

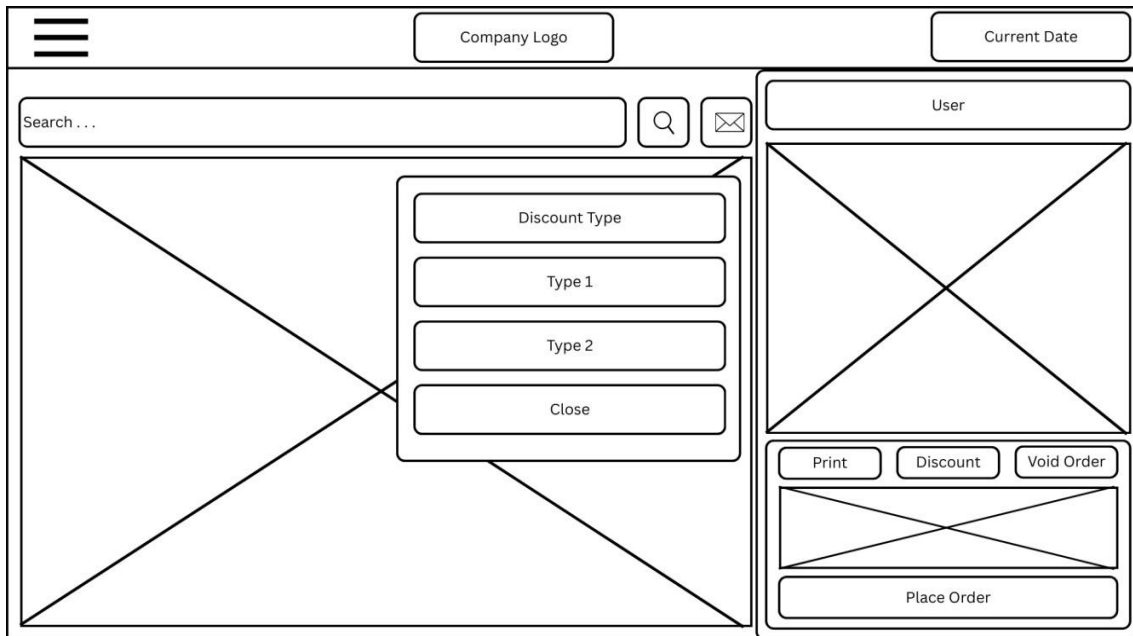


Figure 8. Discount Type

Discount Type

Elements:

- Senior Citizen
- Persons with Disabilities (PWD)

Action:

- Apply Discount

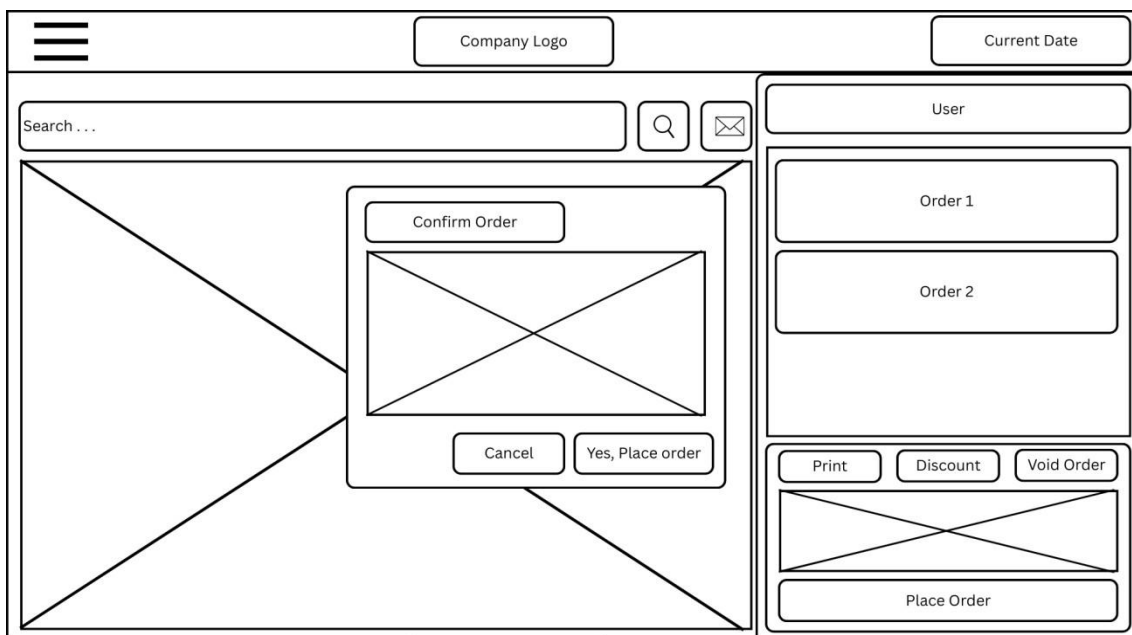


Figure 9. My Order

Order Confirmation Page Elements:

- Order Summary
- Note to Staff
- Status
- Payment Method Selector
- “Place Order” Button

Action: Place Order → Goes to Tracking Page

Time	Transactions	Total Sales

Figure 10. Sales Report

Sales Report

Elements:

- Daily
- Monthly
- Yearly

Action:

- Sales Report Breakdown

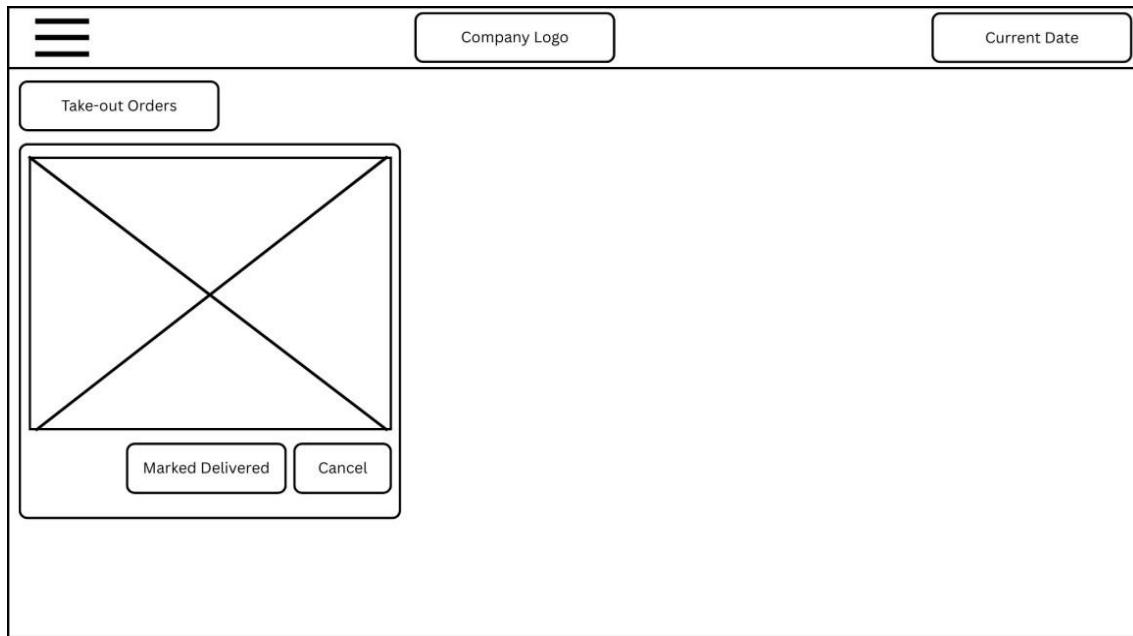


Figure 11. Takeout Orders

Takeout Orders

Elements: Automatic Order Display in KDS

Purpose: Automatically display takeout orders on the Kitchen Display System (KDS) once placed.

Action:

- Customer places a Takeout Order → System marks order as Takeout
- Order is automatically sent through SignalR to the KDS.
- KDS receives and displays the Takeout Order instantly under the “Takeout” section.

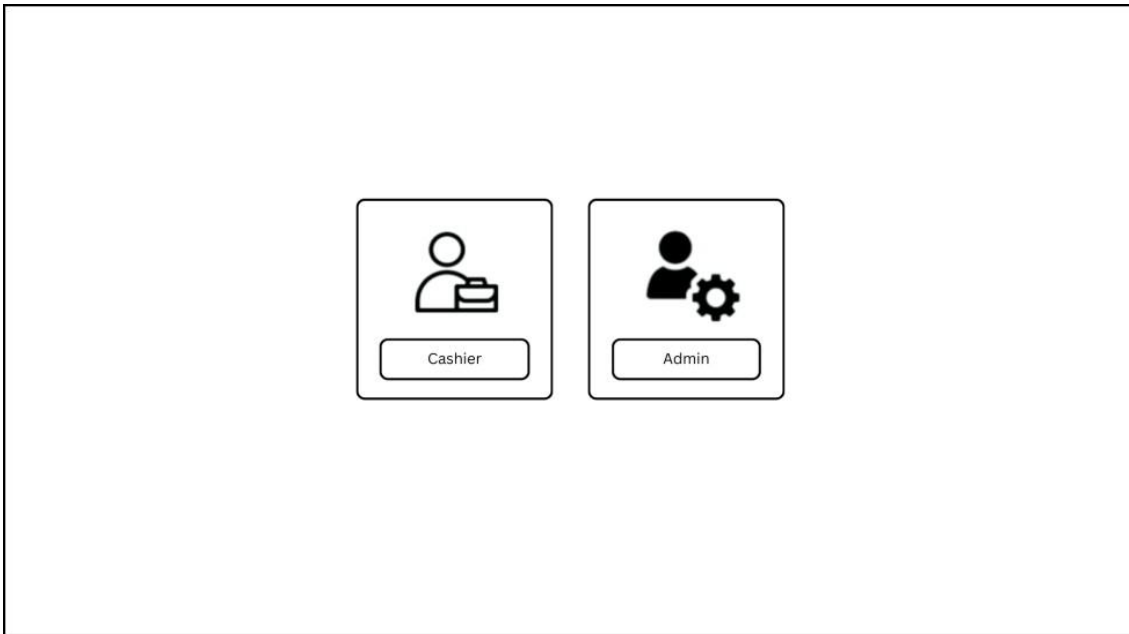


Figure 12. User Selection

User Selection Elements

- *Cashier*
- *Admin*

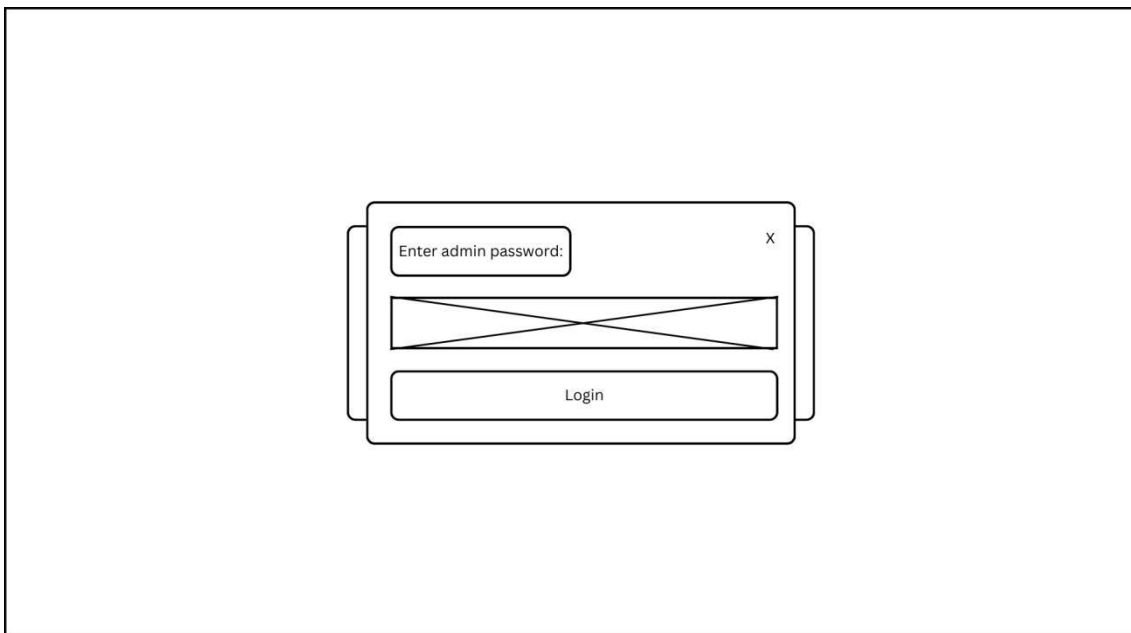


Figure 13. Admin Password

Admin Password Action

- Enter Admin Password

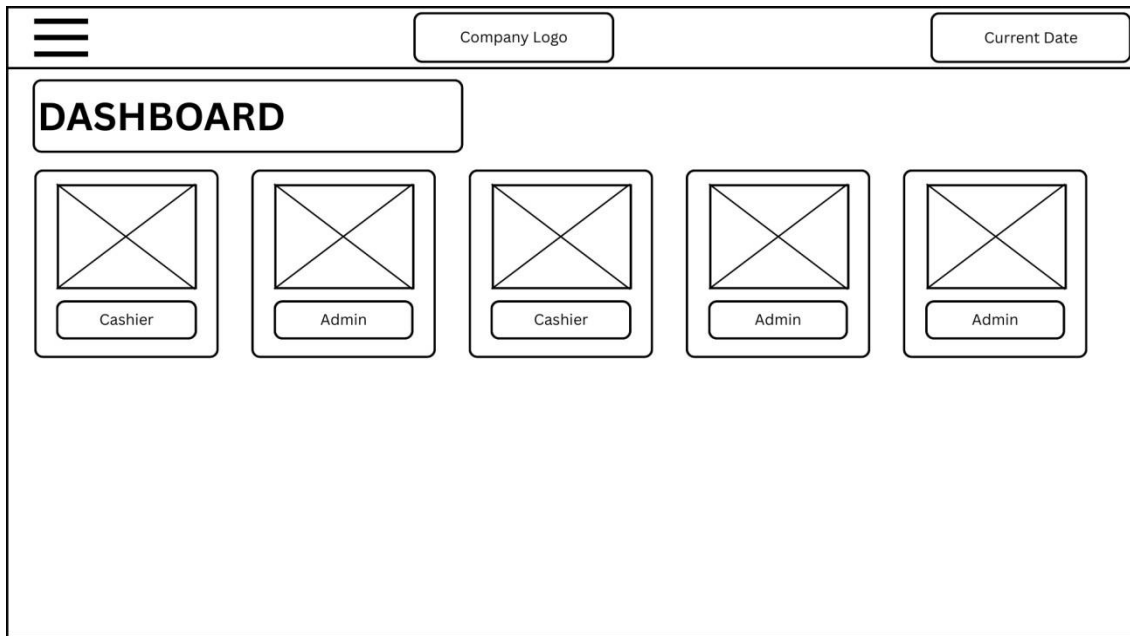


Figure 14. Admin Dashboard

Admin Dashboard

Elements

- Dashboard

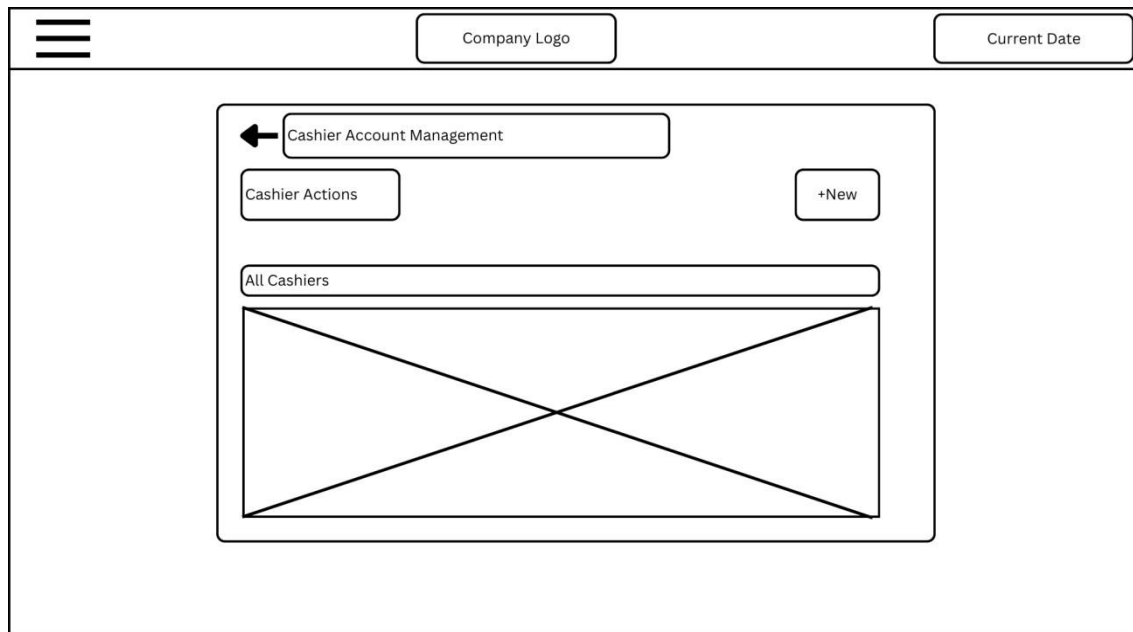


Figure 15. Cashier Account Management

Element

- Create Account

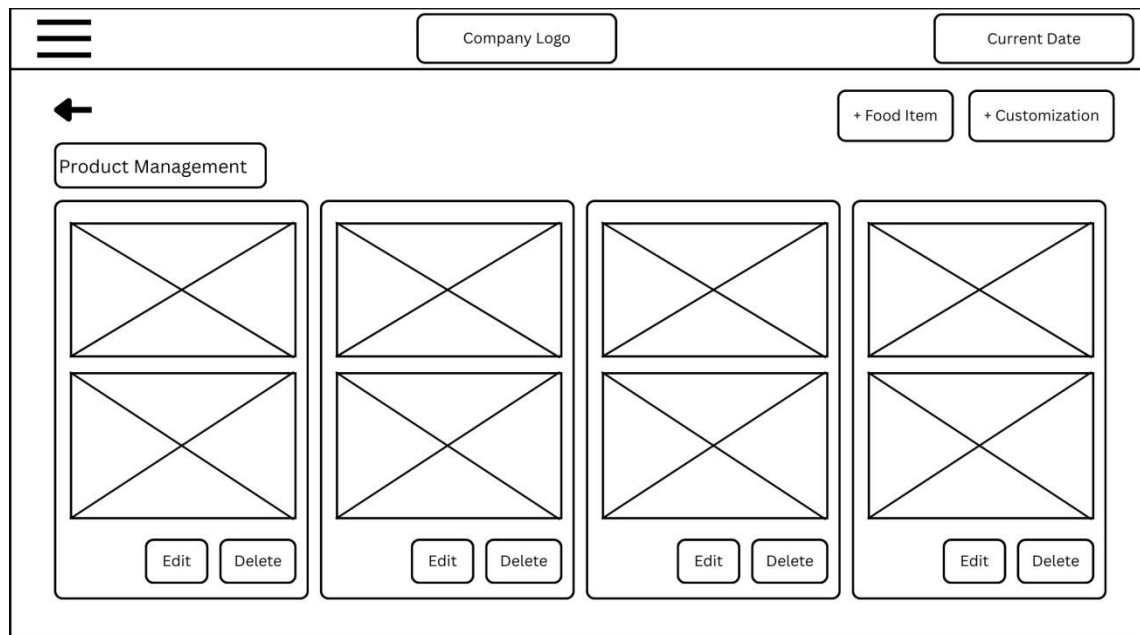


Figure 16. Product Management

Product Management

Elements

- Add Food Item
- Customization

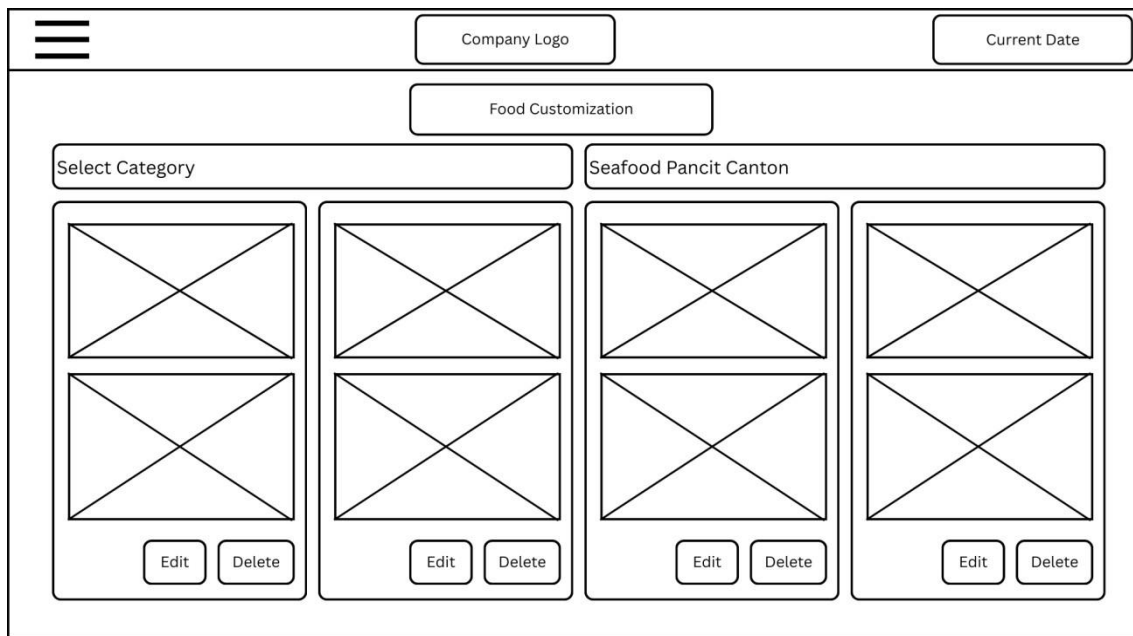


Figure 17. Food Customization

Food Customization

Elements

- Select Category
- Edit
- Delete

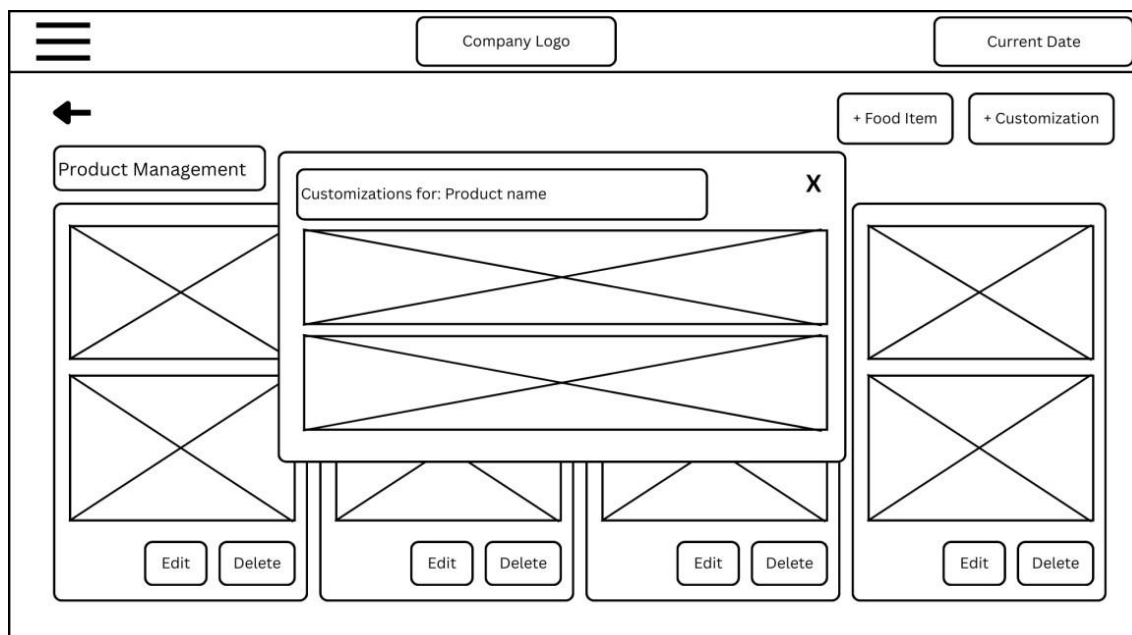


Figure 18: Edit Food Details

Edit Food Details Elements

- Customize Product Name

Wireframe diagram of the 'Add New Food Item' form. The form is a rounded rectangle containing a title bar at the top with a button labeled 'Add New Food Item'. Below the title bar, there are two columns of input fields. The left column contains: 'Category', a text input field, 'Description', a text input field, 'Protien Type', a text input field, and 'Preparation Time (Minutes)', a text input field. The right column contains: 'Food Item Name', a text input field, 'Price', a text input field, 'Spice Level', a text input field, and 'Image/Icon', a text input field. At the bottom right of the form are two buttons: 'Cancel' and 'Save Item'.

Figure 19. Add New Food Item

Add New Food Item Elements

- Add food item

DESIGN OF SOFTWARE, SYSTEM, PRODUCT, AND/OR PROCESSES

In this research project, a data flow diagram and system architectural diagram was used to illustrate the system's logical process flow, while a prototype design was presented to visually demonstrate the user interface and key features of the DigiDine application.

DATABASE DIAGRAM

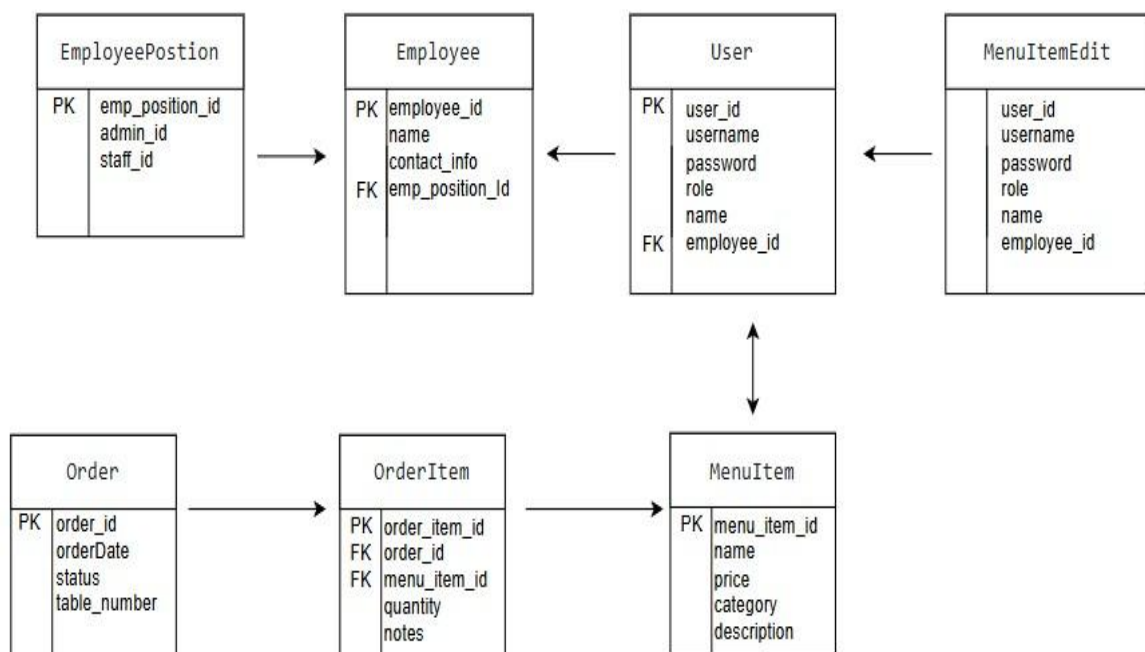


Figure 20. Database Diagram

DATA FLOW DIAGRAM

CONTEXT LEVEL

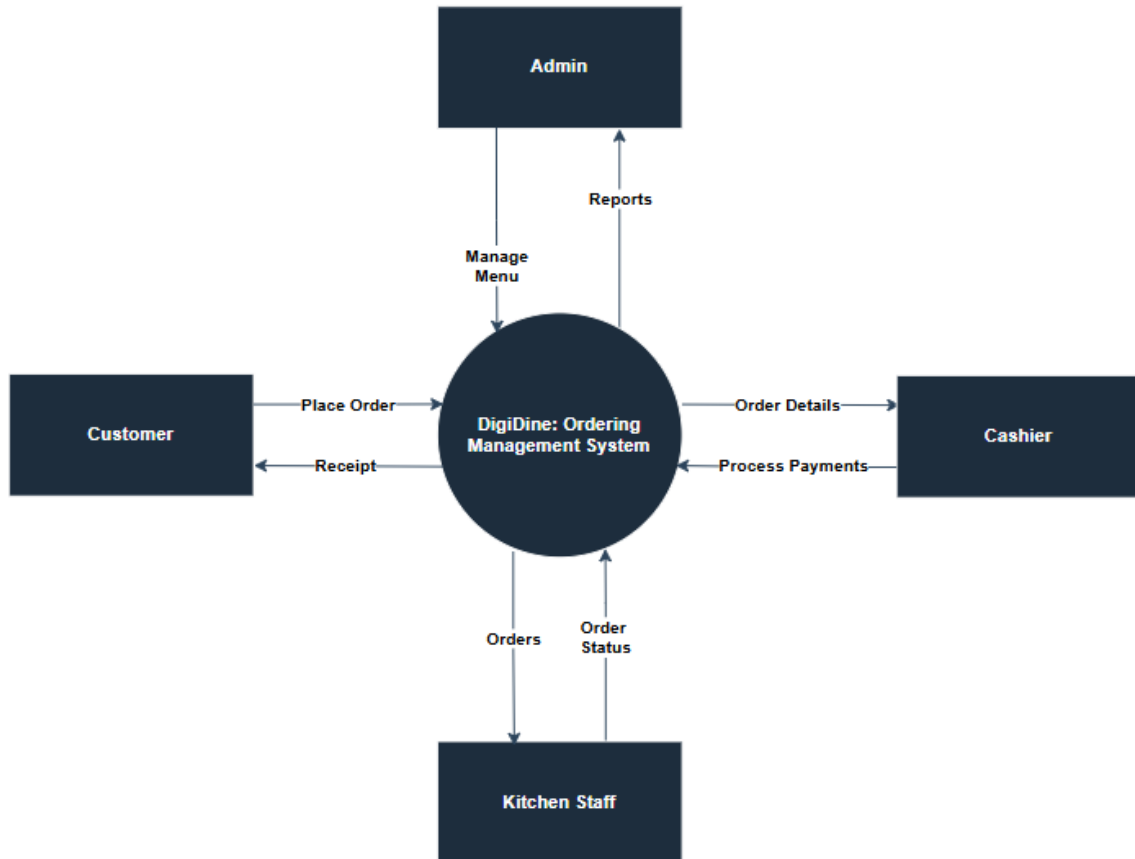


Figure 21. Data Flow Diagram

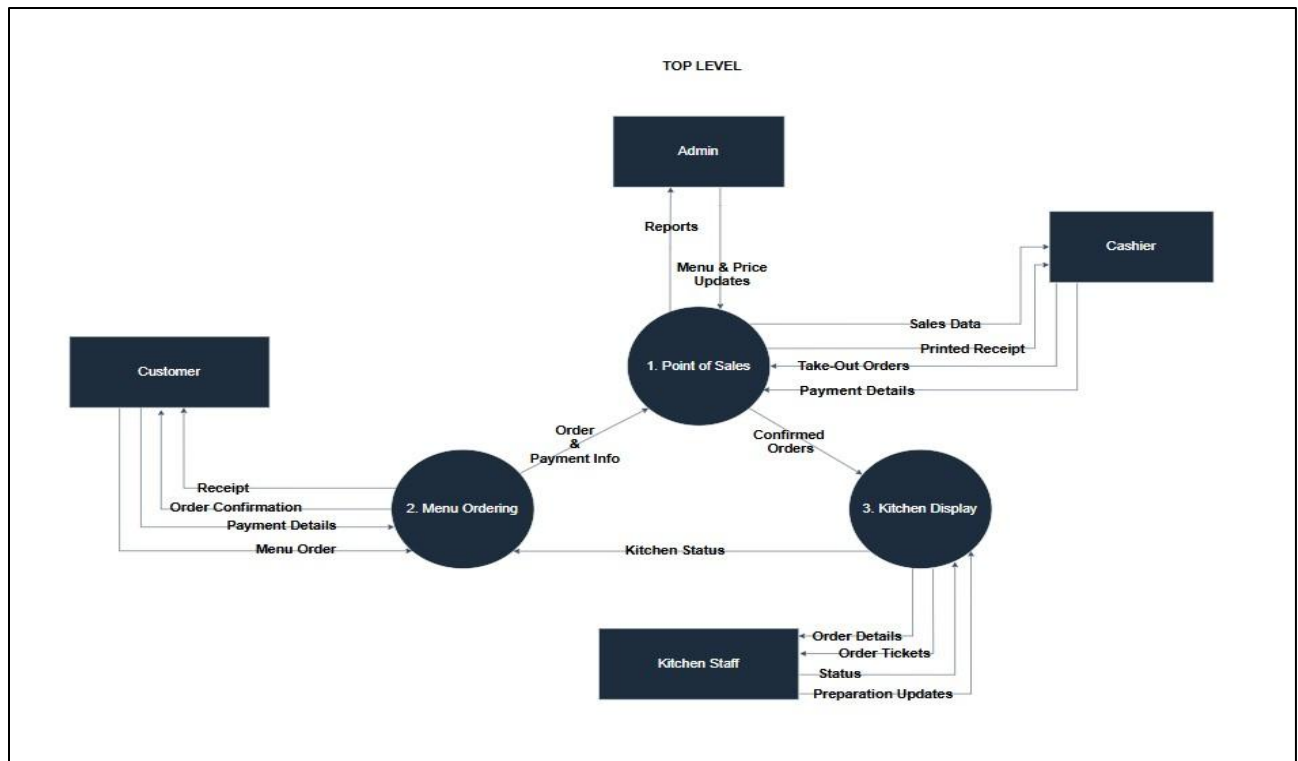


Figure 22. Data Flow Diagram

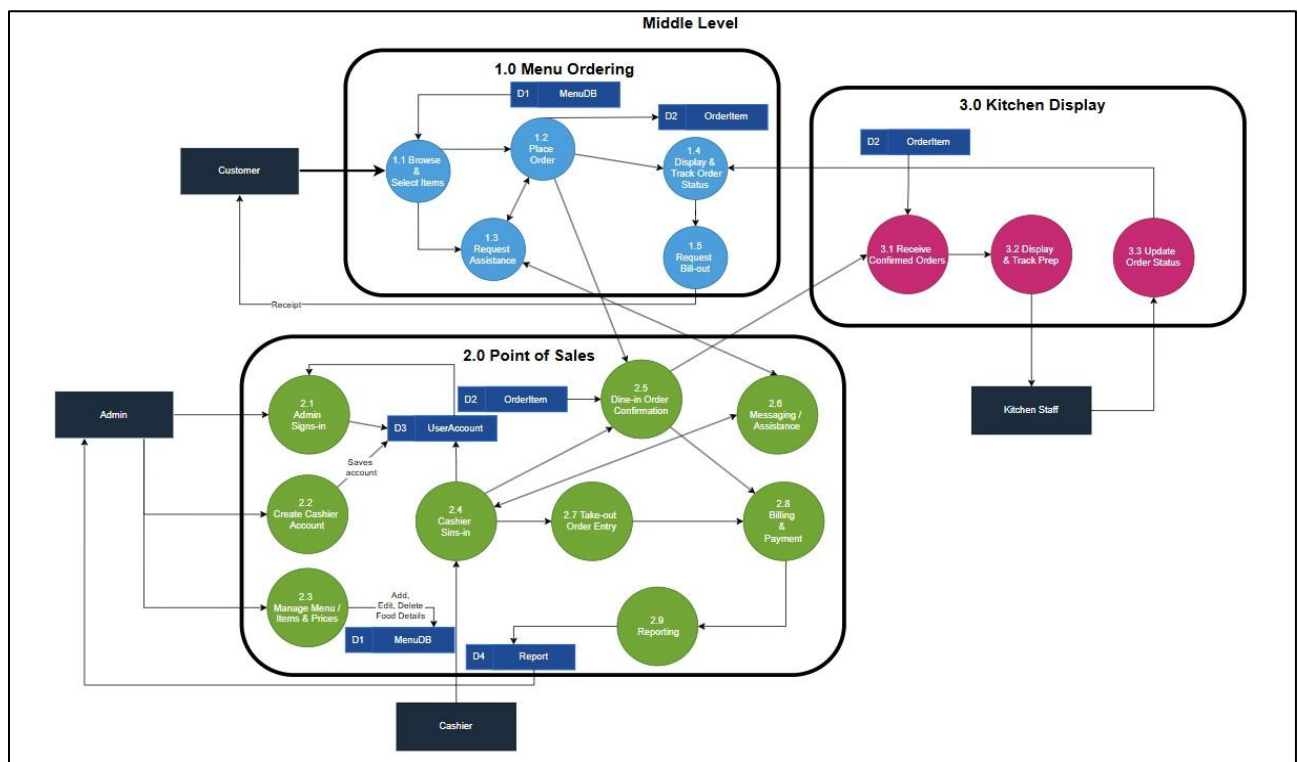


Figure 23. Data Flow Diagram

SYSTEM ARCHITECTURE

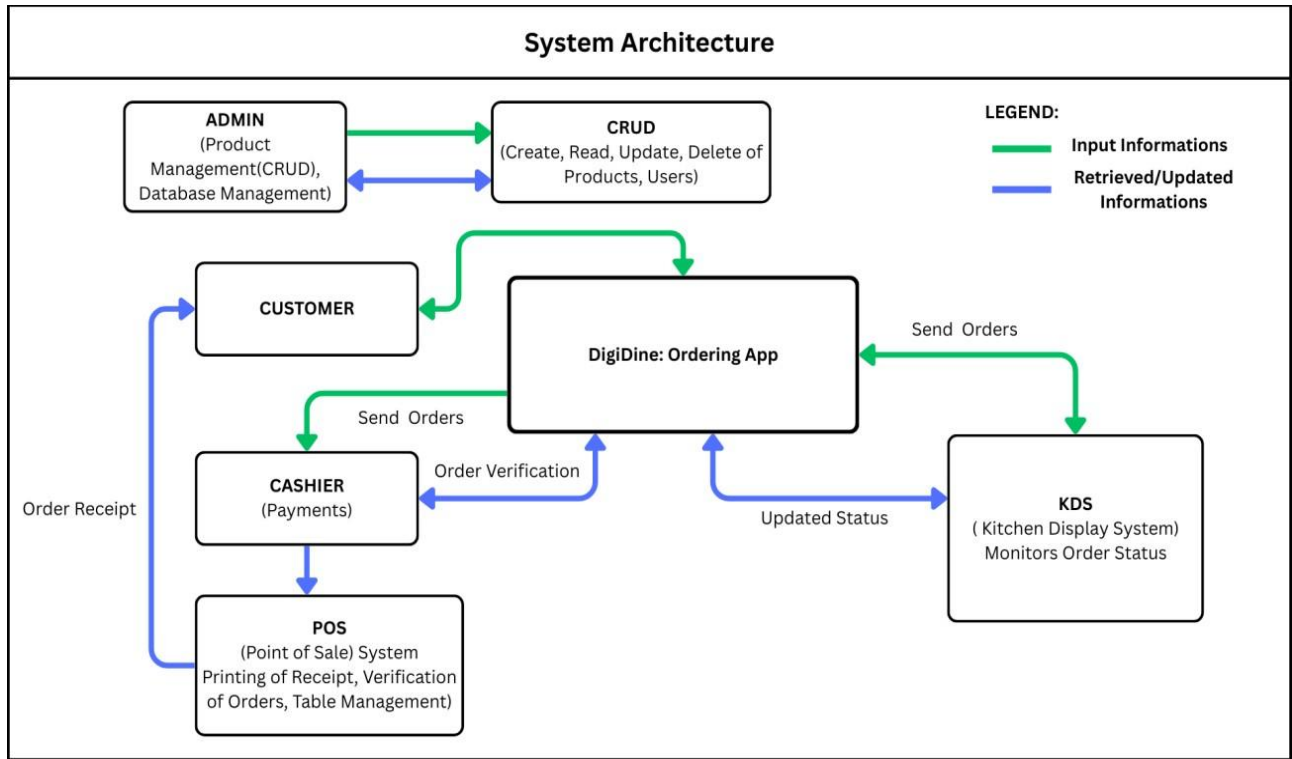


Figure 24. System Architecture

PROTOTYPE DESIGN OF THE DIGIDINE APPLICATION

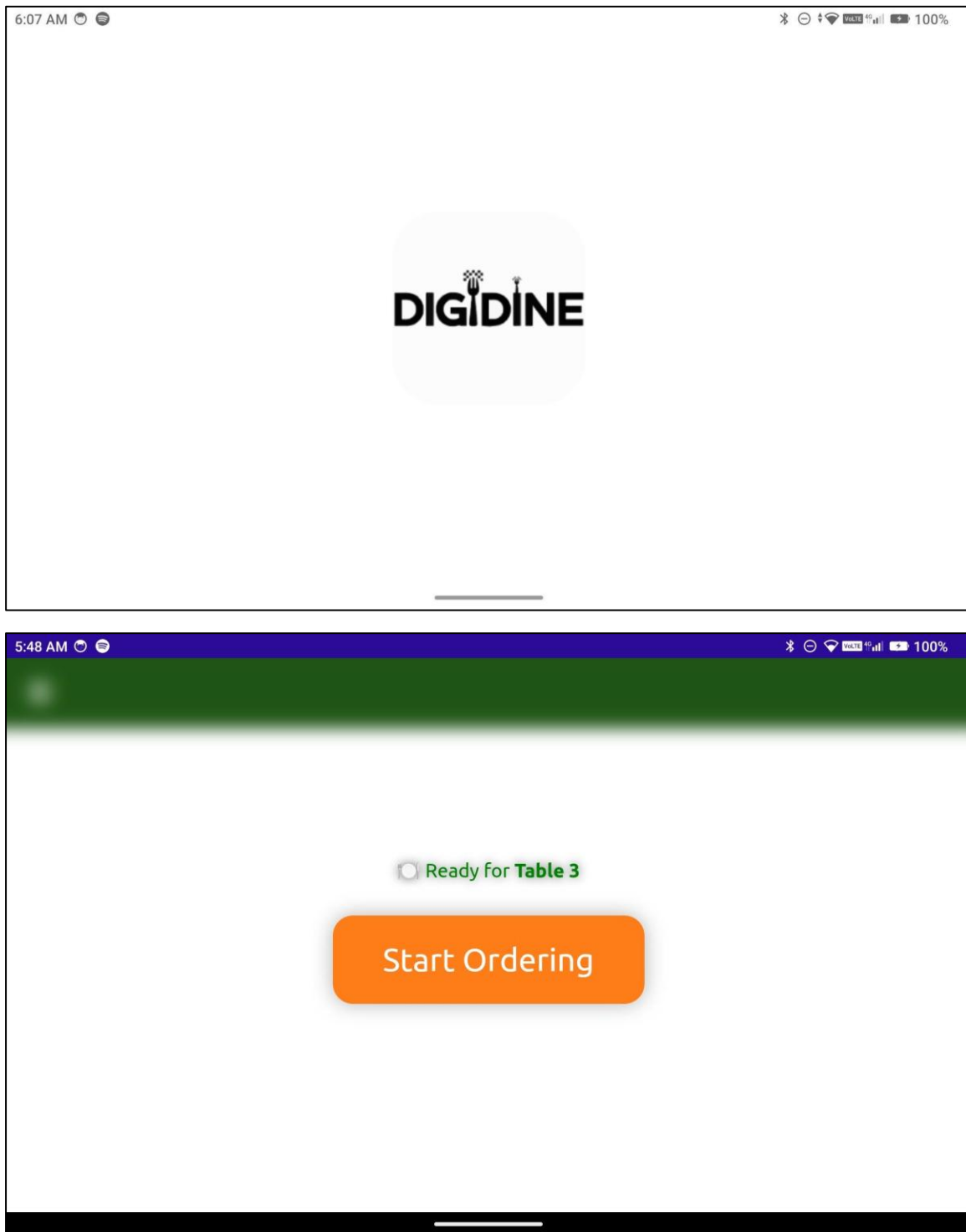


Figure 26. Start Ordering Screen / Logo

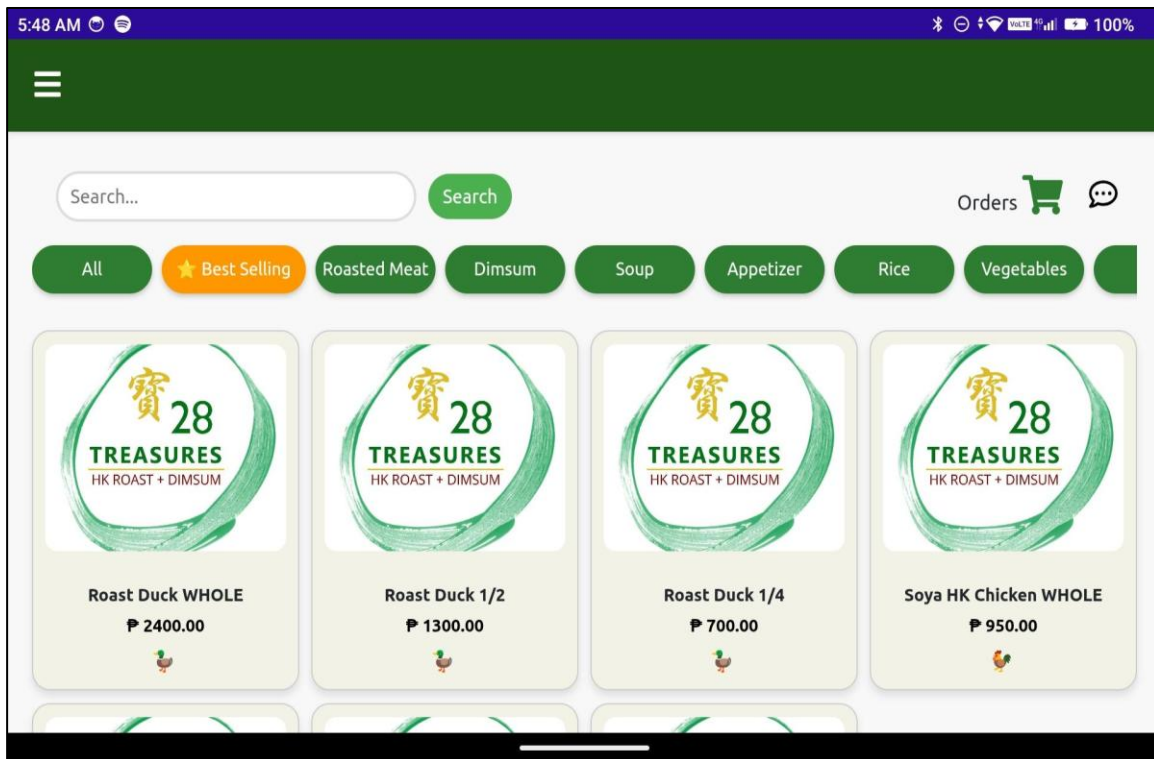


Figure 27. Menu Screen

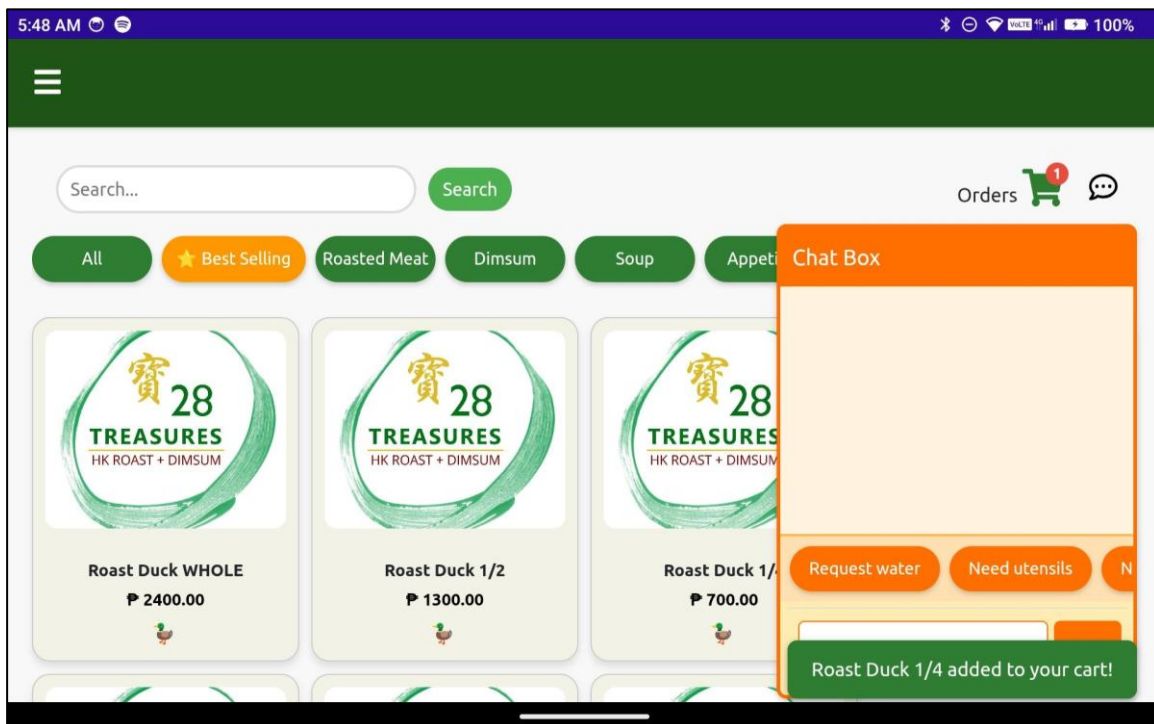


Figure 28. Chat Messaging Screen

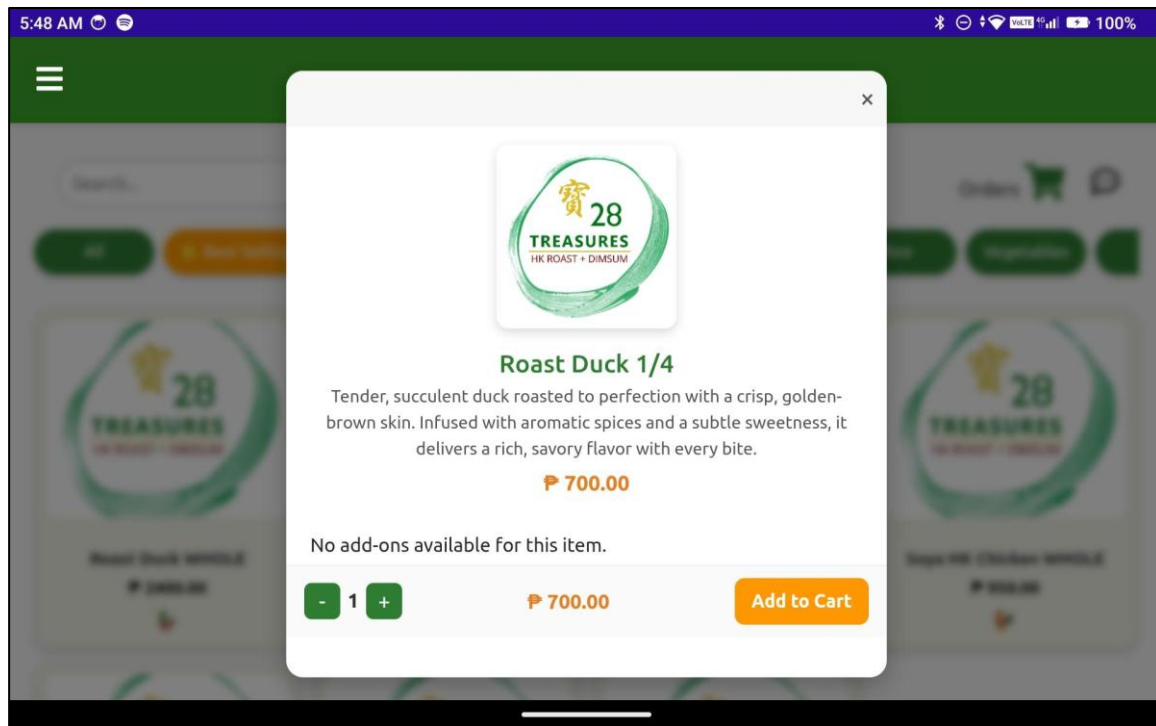


Figure 29. Show Detail Food Item Screen

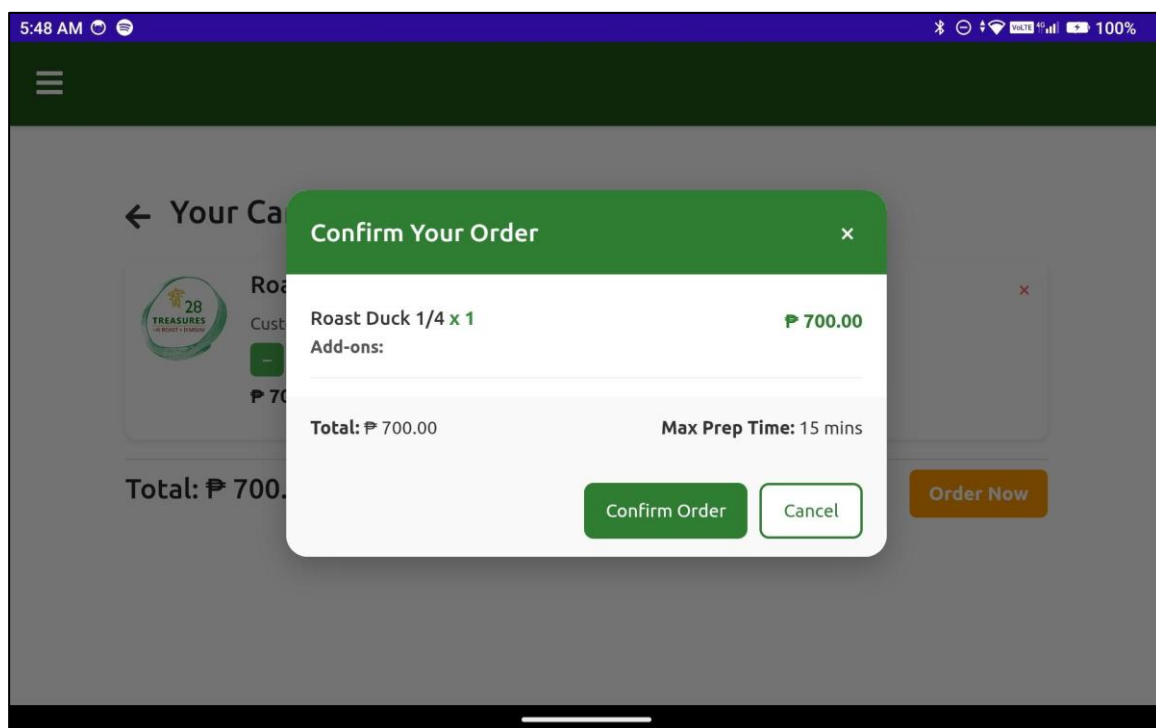


Figure 30. Cart & Note to Staff Screen

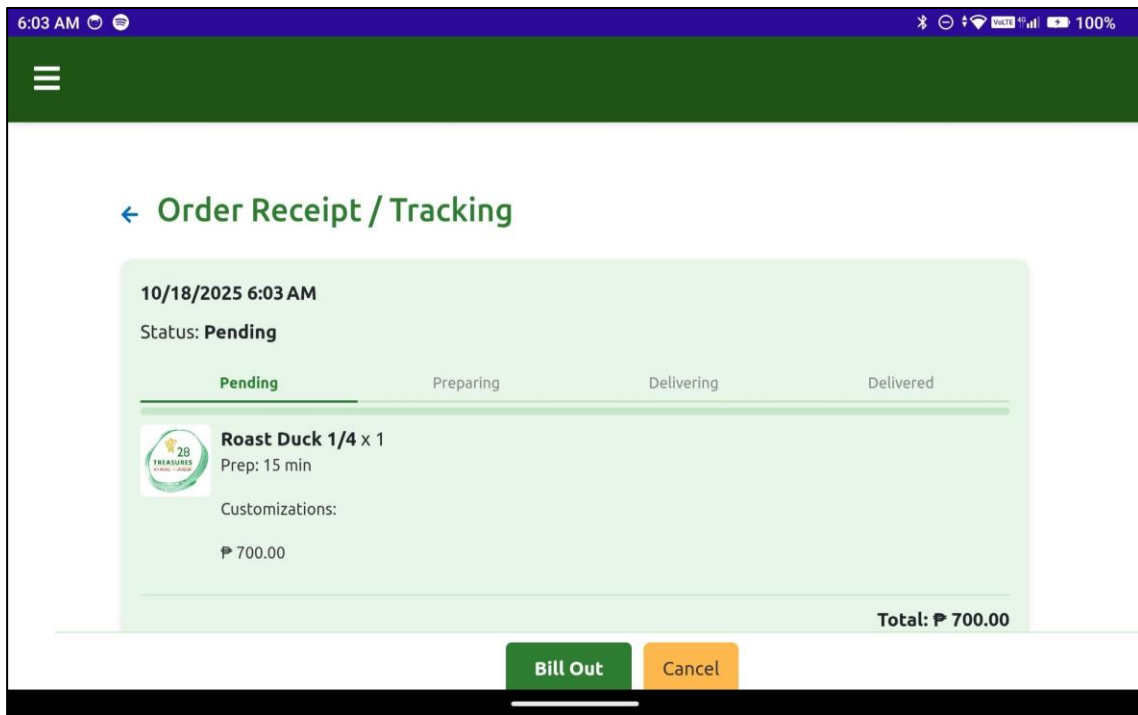


Figure 31. Bill Out Notification Message Screen

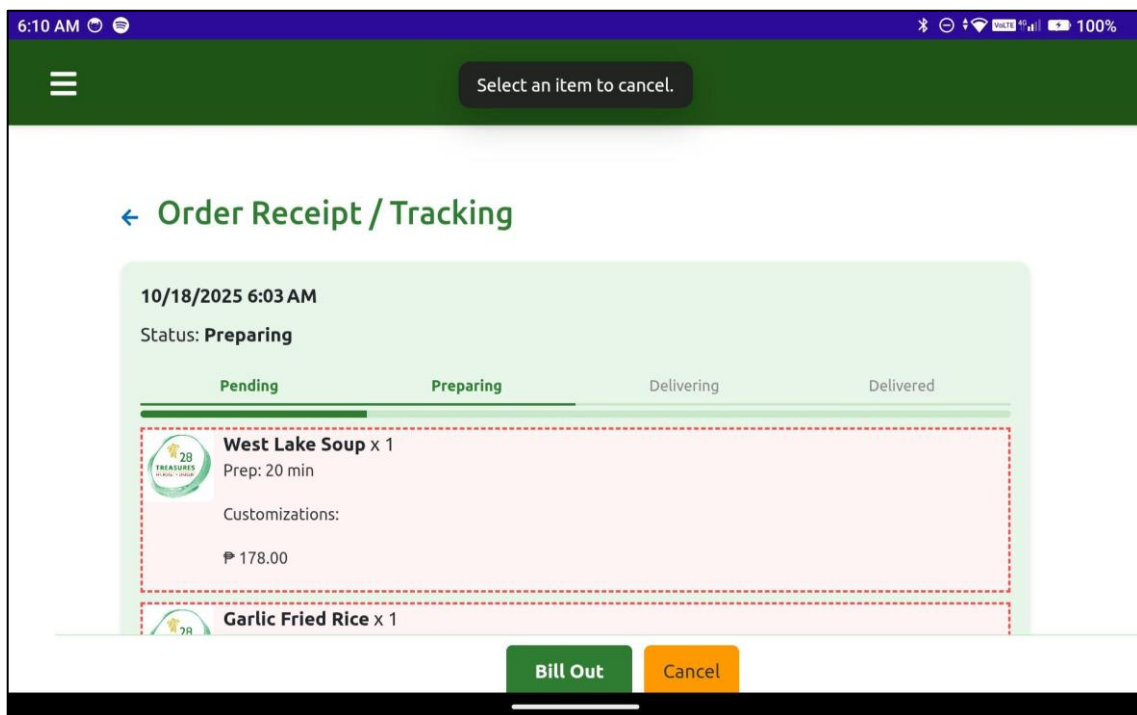


Figure 32. Order Reciept/ Tracking

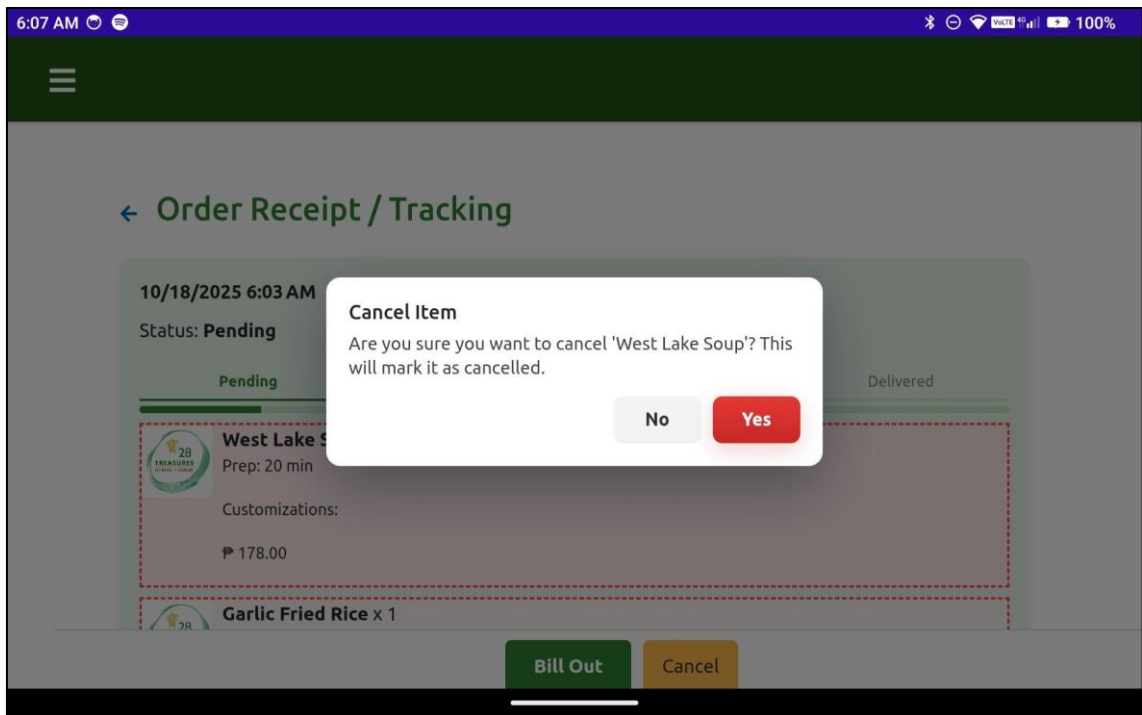


Figure 33. Cancel Item

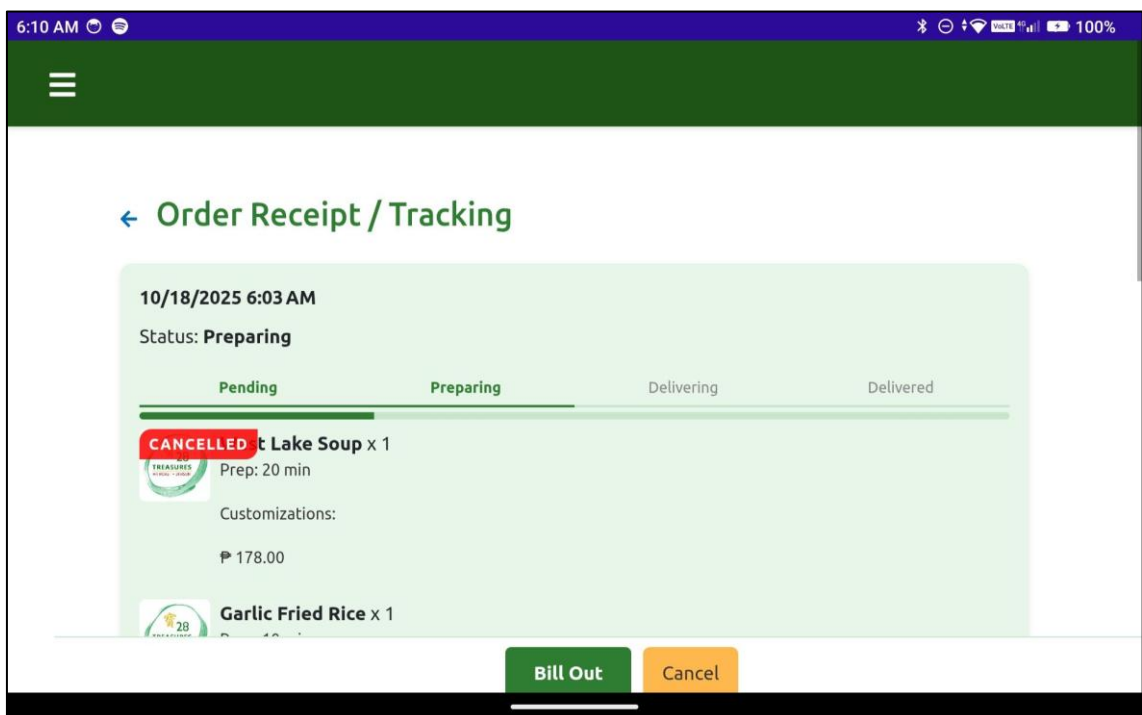


Figure 34. Cancelled item

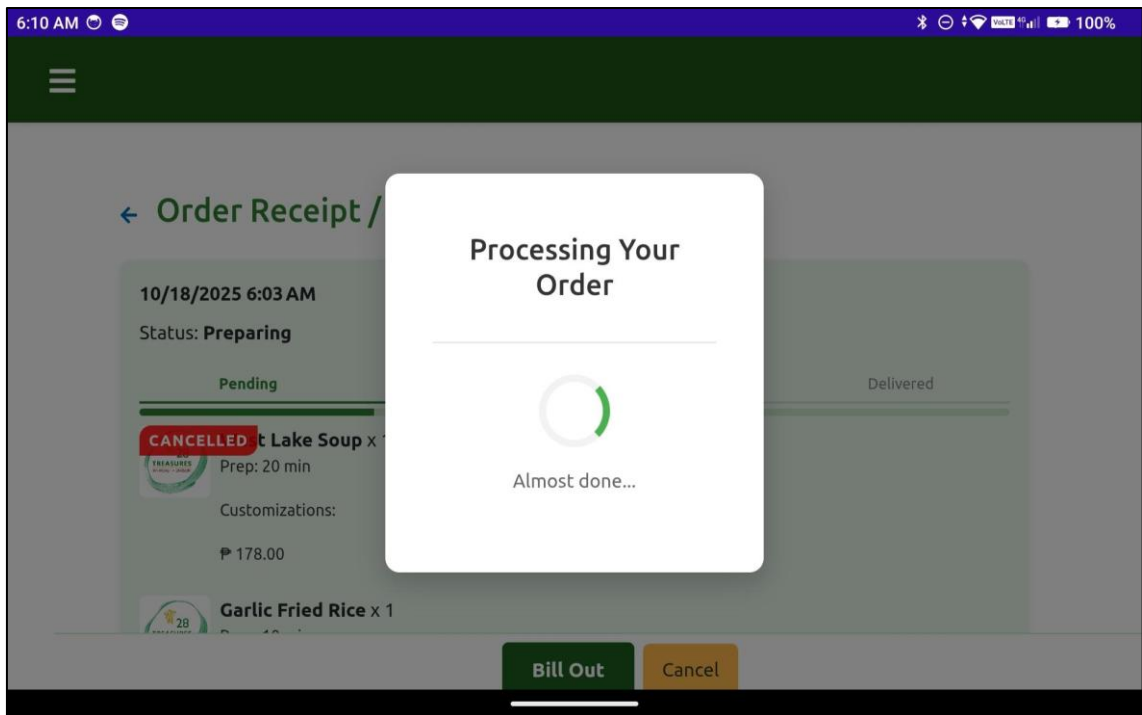


Figure35. Processing order

PROTOTYPE DESIGN OF THE POS CASHIER-SIDE APPLICATION

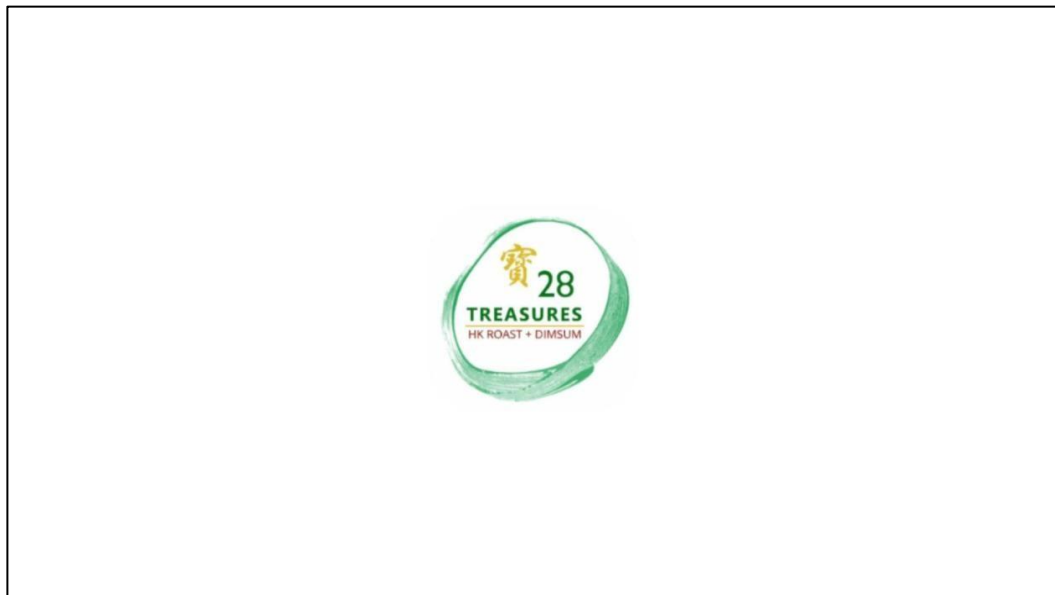


Figure 36: POS splash screen

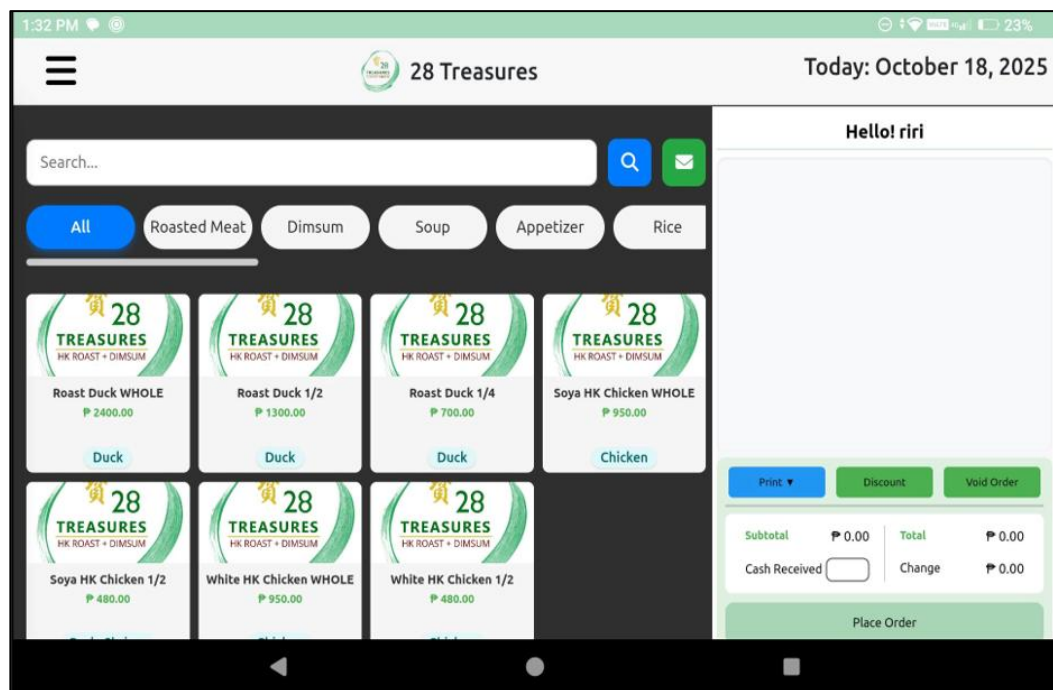


Figure 37. POS/Menu UI

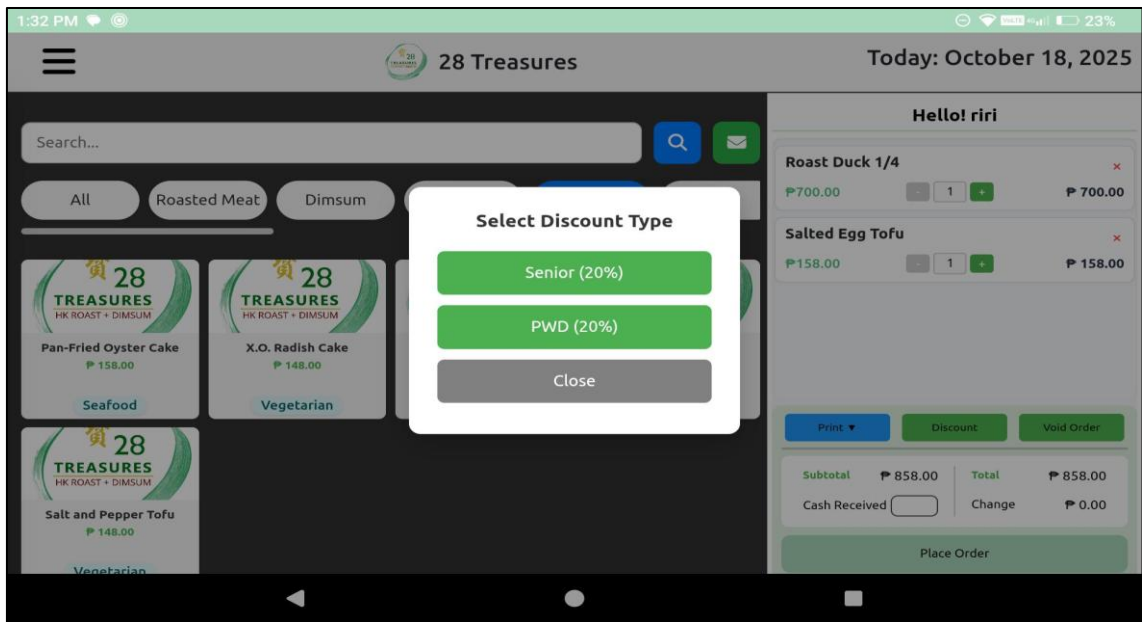


Figure 38. Discount selection

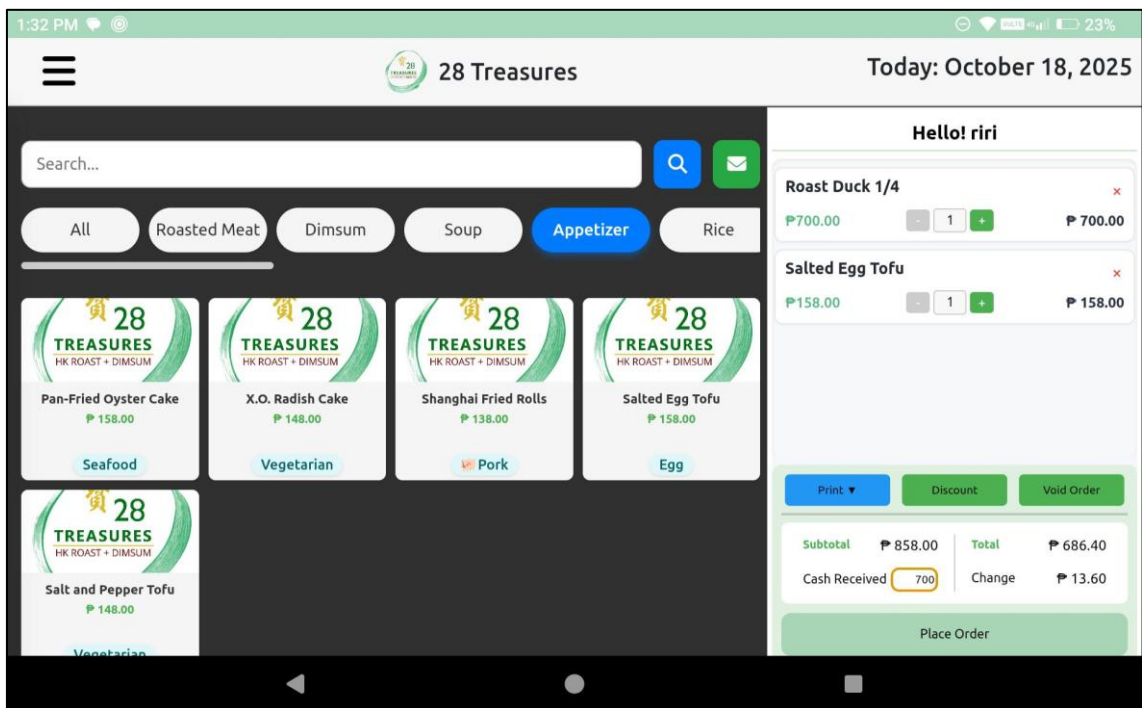


Figure 39. Discount applied

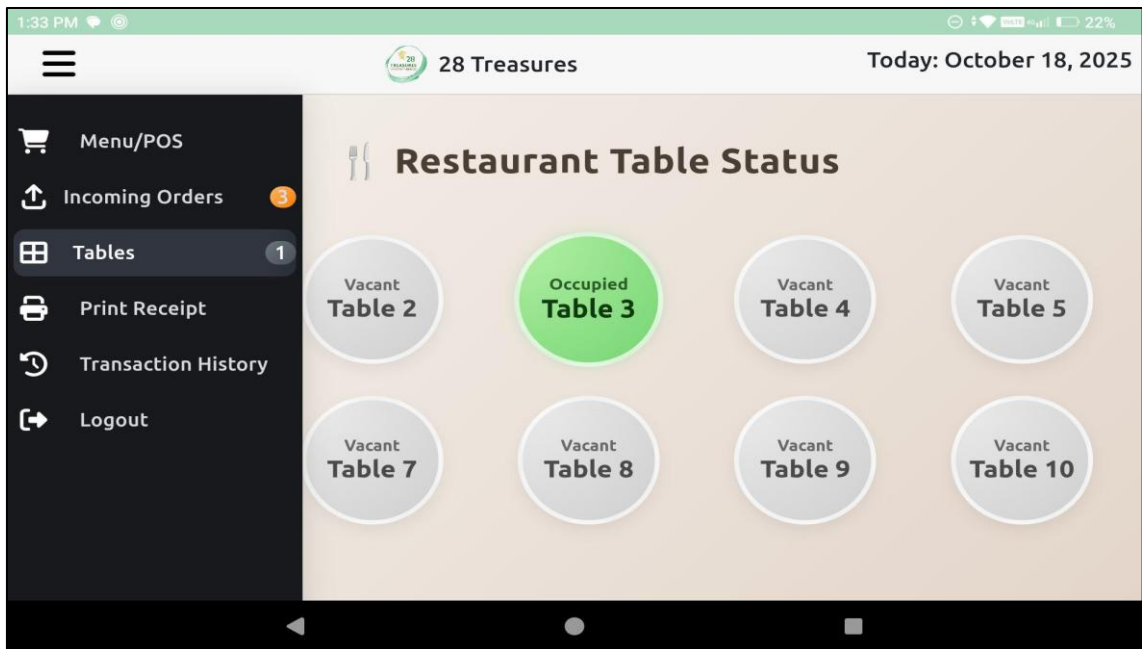


Figure 40. Table status

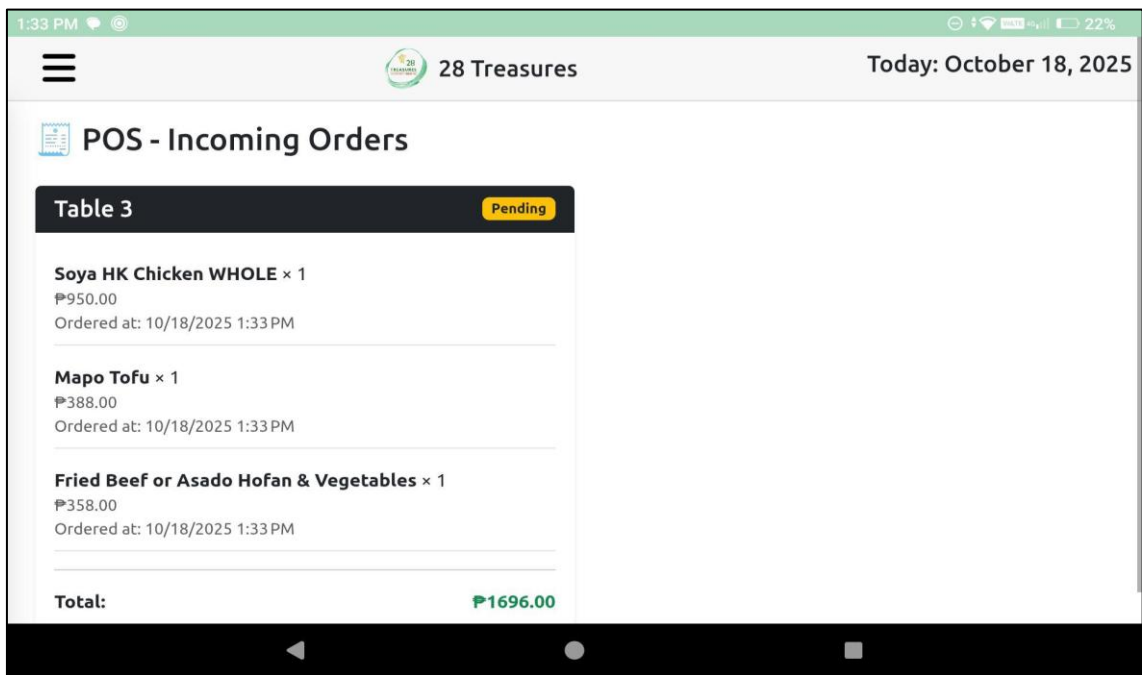


Figure 41. Incoming orders

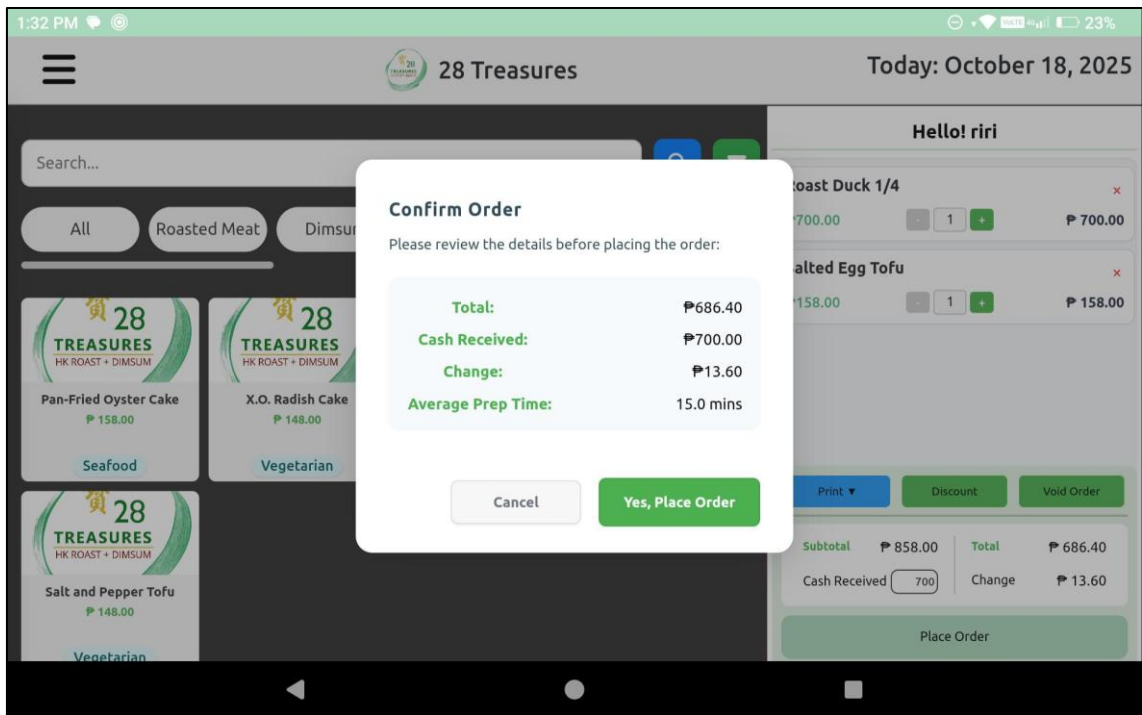


Figure 42. Order confirmation

PROTOTYPE DESIGN OF THE POS ADMIN-SIDE APPLICATION

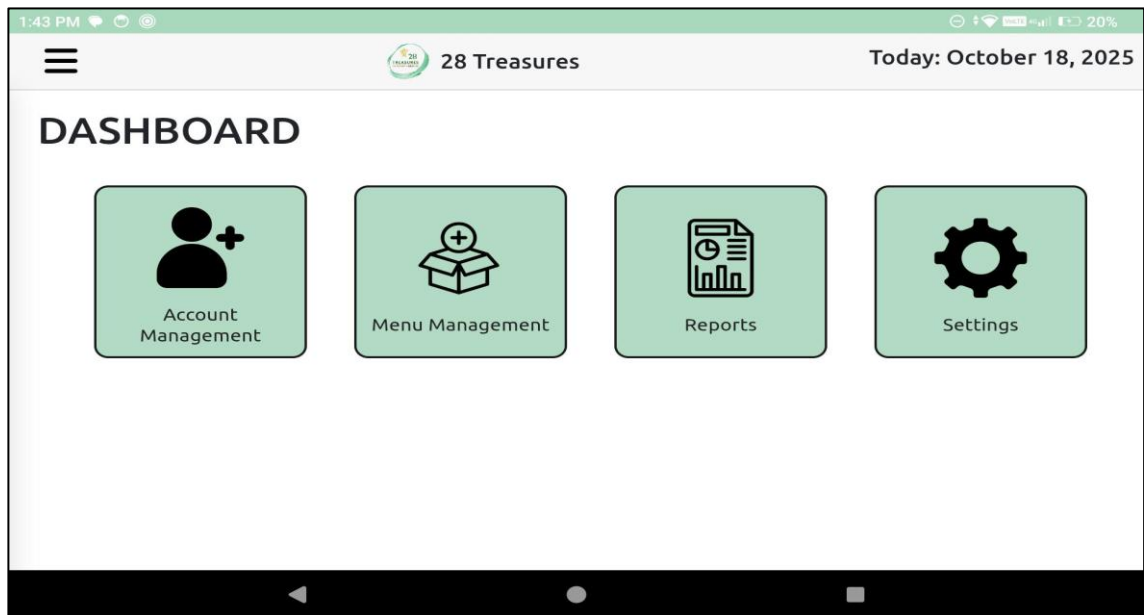


Figure 43. Admin dashboard

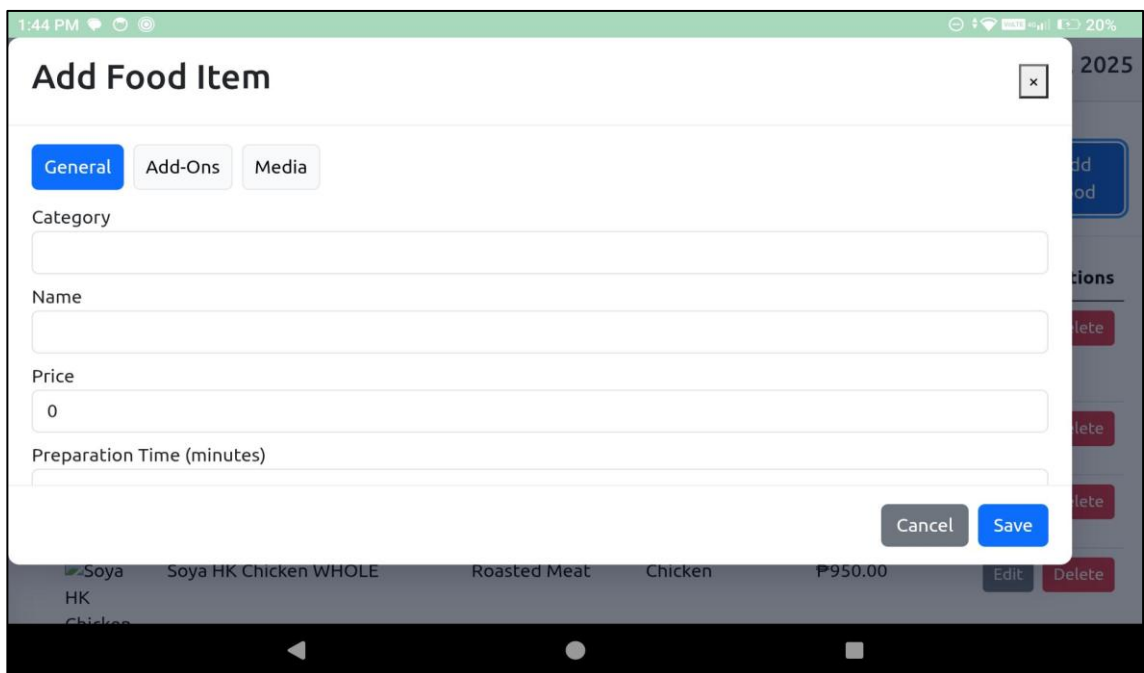


Figure 44. Adding food item

PROTOTYPE OF THE KDS

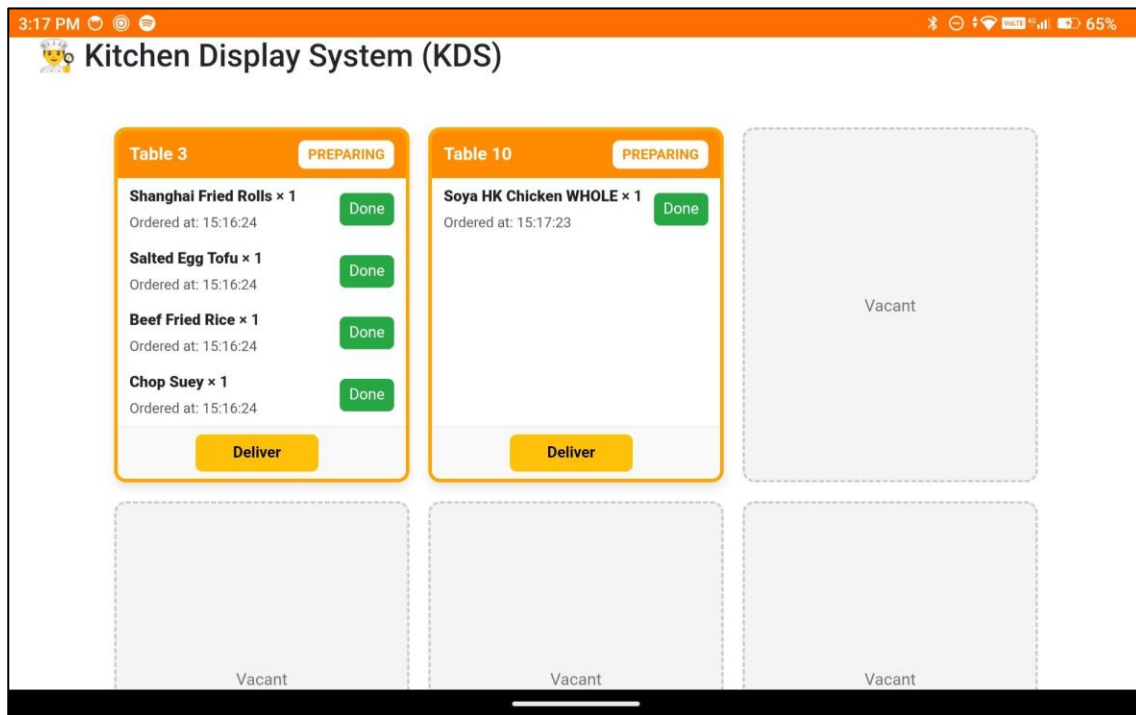


Figure 45. KDS interface

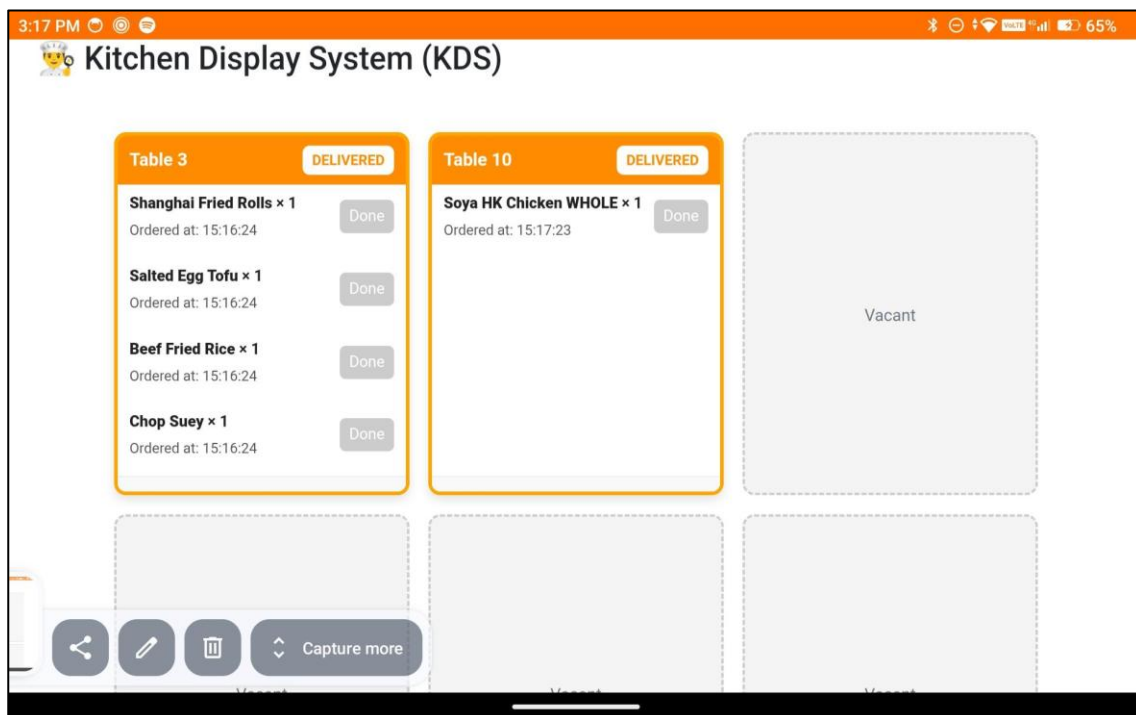


Figure 46. Order Display

Prototype Design of the Receipts:

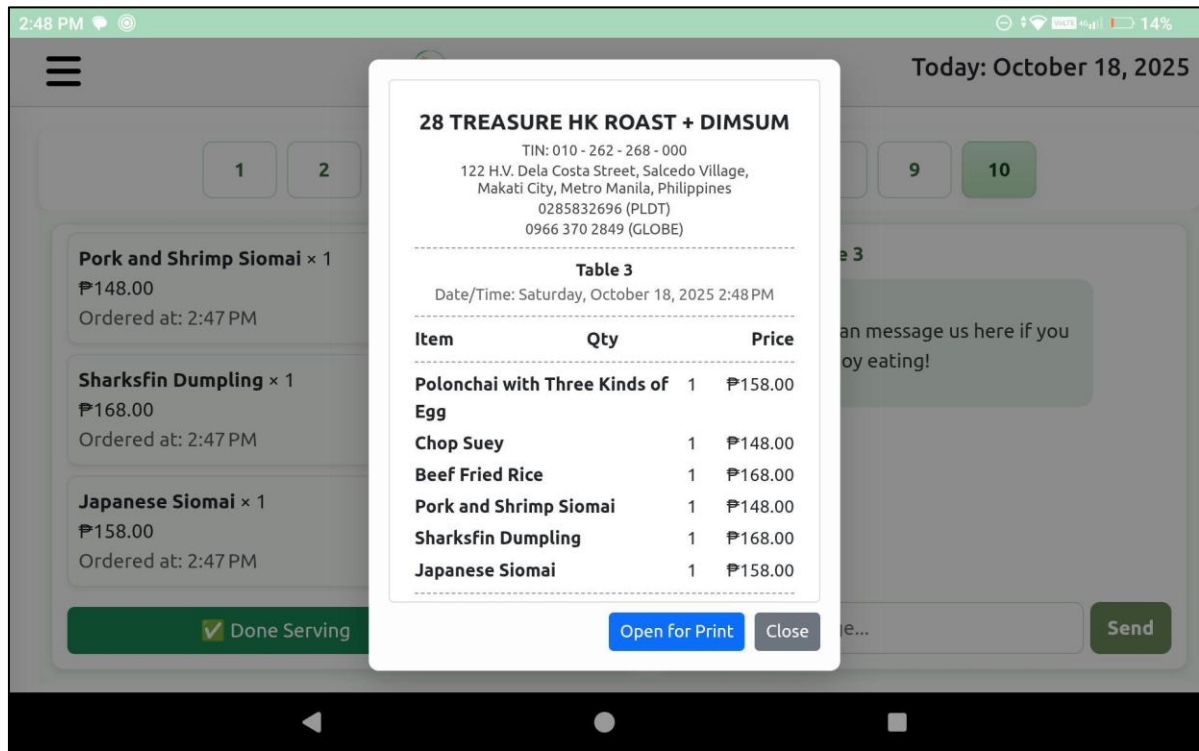


Figure 47. Official receipt display

Development

The development of the mobile ordering application, DigiDine, for 28 Treasures HK Roast + Dimsum-Makati followed a methodical, iterative process that prioritized both technical functionality and user-centered design. The project goal was to put the traditional restaurant ordering system into a modern digital platform to minimize mistakes, provide an easy and convenient ordering process, and enhance customer satisfaction.

The development process began with extensive planning meetings, such as stakeholder discussions and observations of how the restaurant operated. Restaurant management collaborated with the project team to identify their challenges, peak times of day, and table configurations. It was critical information for system functionality such as real-time order tracking, offline use through a local access point (AP), integration with a custom POS and kitchen display system, and support for in-restaurant and outdoor seating. Technical requirements such as Android compatibility, anti-theft mounts, and a secure power supply for tablets were well defined at this point.

The DigiDine system was designed to be modular, stable, and offline-capable. The mobile application communicates with a local server via Access Point to achieve a stable connection irrespective of the internet connectivity. The backend is optimized to manage data of orders, table allocation, and app-POS-KDS interactions. The design enables data to flow seamlessly from the customer's device to the kitchen and cashier with minimal latency.

For user experience (UX) and user interface (UI) design, responsiveness, accessibility, and clarity were given high priority. Large icons, interactive item descriptions, and readable fonts were employed with a clean and intuitive presentation of the digital menu. Special emphasis was on simplifying navigation and reducing the number of taps to place an order, especially for non-tech-savvy individuals.

Prototyping and Feature Implementation

The development team utilized an agile process named Kanban. The process is designed to continuously improve, offer flexible work schedules, and transparently track tasks. They utilized a Kanban board that depicted the development process in columns such as "Backlog," "To Do," "In Progress," "In Review/Testing," and "Done." This enabled the team to easily trace tasks and make changes as necessary.

Key features were progressively integrated:

- **Digital Menu with Customization:** The system has a menu with categories, descriptions, prices, and custom request or special instructions fields. This allows flexibility in ordering and improves order accuracy.
- **Table-Based Identification:** Each tablet is dedicated to one table and labeled with a unique table ID. When customers tap the "Start Ordering" button, the system automatically allocates the order to the proper table, avoiding the need for manual input or a QR scan.
- **Real-Time Order Tracking:** Orders placed through the mobile app can be seen in real-time on the POS and KDS. Updates on status are sent back to the customer's tablet, and they can see progress without having to ask the staff repeatedly.
- **In-App Chat Messaging:** Chatting functionality enables clients to converse directly with the service staff for follow-ups, confirmations, or requests, which supports customized service and prompt responses.
- **Offline Support via Local Access Point:** An access point for local wireless had to be installed to create a local network for the application. This makes ordering feasible without the use of internet connectivity, which is extremely important in outdoor dining areas.
- **POS and KDS Integration:** The backend of the system was integrated with a POS system to handle billing, discounts, and checkout processes. Meanwhile, the Kitchen Display System receives structured order information for food preparation in real-time.

Once the DigiDine Application is completed, the team will perform extensive system testing. Unit testing will ensure the functionality of every feature, and integration testing will ensure that app modules, POS, DigiDine, and KDS work seamlessly together. They'll perform several practice service sessions and simulations to test the system in real-world situations. These will help developers identify usability issues, delays, and unforeseen bugs.

The developers will put tablet mounts and power arrangements to test for their stability and comfort. We will test anti-theft measures to make sure tablets are secure and cannot be accessed. The wired charging system was created to provide uninterrupted power without disrupting customer interaction.

RESULTS AND DISCUSSION

Testing

In this phase, the proponents conducted a series of tests to ensure that DigiDine, the mobile ordering system for 28 Treasures HK Roast + Dimsum - Makati, functioned as expected. The testing process involved checking the system's performance, usability, and reliability under actual restaurant conditions. Each core module—the Menu Application, POS Application, and Kitchen Display System—was tested individually and together as an integrated system.

The proponents performed unit testing to verify that specific features like ordering, cart management, and in-app messaging worked properly. Integration testing followed to confirm that orders placed on the Menu App were accurately reflected on the POS and KDS screens. User acceptance testing was also done with the restaurant's staff and management to evaluate if the system met their operational needs. Finally, offline testing through a local access point (AP) was carried out to ensure the system could still operate even without internet connectivity.

The results showed that the DigiDine system efficiently handled order transactions, displayed accurate real-time status updates, and maintained communication between customers, cashiers, and kitchen staff without delays. The feedback gathered from users helped the developers fine-tune the system's responsiveness and improve the interface for better user experience.

Description of Prototype

The DigiDine prototype was developed using the Prototype Method, which allowed continuous refinement through user feedback. The prototype began as a simple design made with tools like Figma, showing basic screens for menu navigation, order cart, and messaging. As development progressed, interactive prototypes were built using .NET MAUI Blazor, allowing actual simulation of restaurant workflows on mobile devices.

Through multiple iterations, the prototype evolved to include advanced features such as:

- Real-time order status updates (Pending, Preparing, Ready, Delivered)

- Offline functionality via local Wi-Fi access
- Integrated communication between customer and POS staff
- Enhanced cart and menu management features

The prototype helped both developers and restaurant stakeholders visualize the system flow before the final implementation. This stage also made it easier to identify usability issues early, ensuring the final version was efficient and user-friendly.

Implementation Plan

The implementation of DigiDine involved deploying three interconnected subsystems: the Menu Application (customer-side), the POS Application (cashier-side), and the Kitchen Display System (KDS). The system was first installed on Android tablets mounted securely on each dining table. These tablets were connected to a local network via a dedicated Access Point, ensuring smooth communication between the three components even without internet access.

The major tasks included:

1. System Installation and Configuration – Setting up the .NET MAUI Blazor system on tablets and kitchen monitors.
2. Database Integration – Linking the menu, order, and billing data to a centralized database for consistent updates across all modules.
3. User Orientation – Conducting brief training sessions for staff to ensure they understood how to use the Menu App, POS, and KDS.
4. System Testing and Monitoring – Running pilot tests during off-peak hours to identify bugs and measure response times.

The required resources included Android tablets for customers, kitchen monitors for KDS, and a local network router for offline access. The implementation ensured that the system's components were compatible and could run efficiently in a real restaurant setting.

Implementation Results

After installation, DigiDine was tested in the daily operations of 28 Treasures HK Roast + Dimsum - Makati. The results were positive—the restaurant staff reported faster order processing and fewer errors compared to the manual paper-based system. Customers appreciated being able to place and track their orders directly from their tables without waiting for staff.

Some of the major improvements observed during implementation were:

- **Reduced Order Errors:** Orders were transmitted digitally and displayed instantly on the KDS.
- **Improved Communication:** The in-app chat helped staff respond quickly to customer concerns.
- **Faster Service Time:** Real-time coordination between POS and KDS minimized waiting times.
- **Enhanced User Experience:** The app's interface was intuitive, allowing even first-time users to navigate easily.

Feedback from the restaurant's management highlighted that DigiDine streamlined operations, improved coordination among staff, and enhanced customer satisfaction. The system successfully met its goal of digitizing and optimizing the restaurant's order management process, proving that the prototype approach was effective for developing a reliable and user-centered application.

CONCLUSION

The development and implementation of the DigiDine Mobile Ordering System for 28 Treasures HK Roast + Dimsum – Makati successfully addressed the challenges faced in their traditional manual ordering process. Through the use of the Prototype Method, the proponents were able to design, test, and refine a functional system that met the restaurant's operational requirements and improved service efficiency. The system's integration of the Menu Application, POS, and Kitchen Display System resulted in smoother communication between staff and customers, faster order processing, and reduced human error.

The results from testing and implementation demonstrated that DigiDine provided an effective solution for managing orders, monitoring status updates, and ensuring real-time communication within the restaurant. The use of local network access enabled continuous operation even without internet connectivity, ensuring reliability during peak hours. Overall, the system achieved its goal of enhancing the restaurant's workflow and customer experience through digital automation and user-friendly design.

In conclusion, the DigiDine system successfully transformed the restaurant's order management process from manual to digital. It improved operational efficiency, minimized order inaccuracies, and strengthened communication between staff and customers. The study also proved that adopting modern technologies like .NET MAUI Blazor and local access networking can create practical and cost-effective solutions for small to medium-sized restaurants.

Based on the findings, the proponents recommend continuous maintenance and system monitoring to ensure stable performance over time. Future improvements may include integrating online payment options, customer feedback modules, and AI-based food recommendations to further enhance user engagement. Additionally, expanding the system to support multiple branches can help standardize operations across different restaurant locations. With these enhancements, DigiDine can continue to evolve as a complete, smart, and efficient ordering solution for the food service industry

REFERENCES

- A. De. Gabriel, J. L. Caangay, L. M. Pineda and A. S. Vasquez, "Enhancing the Customer Experience in Philippine Food Court Establishments with an Integrated Self-Service Ordering System," 2024 3rd International Conference on Artificial Intelligence and Software Engineering (ICAISE), Singapore, Singapore, 2024, pp. 33-37, doi: 10.1109/ICAISE65384.2024.00015.
- Borbon, N. M. D. (2025). Kiosk Revolution: The Transformation of Fast Food in the Philippines. In Technological Innovations in the Food Service Industry (pp. 69-80). IGI Global Scientific Publishing. Retrieved from <https://www.igi-global.com/chapter/kioskrevolution/363548>
- Buenaventura, R. U., Ignacio, A. E., & Laspoña, J. A. S. (2021). Mobile ordering application for a generic fast-food restaurant. *International Journal of Multidisciplinary: Applied Business and Education Research*, 2(5). Retrieved from <https://ejournals.ph/article.php?id=16818>
- Castillo, A. M. M., Salonga, L. J. L., Sia, J. A. L., & Young, M. N. (2020). Improving Fast-Food Restaurants' Method of Operation: Automated Drive-Through Ordering System. *Proceedings of the International Conference on Industrial Engineering and Operations Management*, Dubai, UAE, March 10-12, 2020. Retrieved from <https://ieomsociety.org/ieom2020/papers/375.pdf>
- Davidbritch. (n.d.). *What is .NET MAUI? - .NET MAUI*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui?view=net-maui9.0&utm_source=
- Dolan, E., & Widayanti, R. (2022). Implementation Of Authentication Systems On Hotspot Network Users To Improve Computer Network Security. *International Journal of Cyber and IT Service Management (IJCITSM)*, 2(1), 88-94. Retrieved from <https://iiastjournal.org/ijcitsm/index.php/IJCITSM/article/view/93>

GeeksforGeeks. (2024, August 23). *Introduction to SQLite*. GeeksforGeeks.
<https://www.geeksforgeeks.org/introduction-to-sqlite>

Nguyen, D., Nguyen, V., Trinh, T., Ho, T., & Le, H. (2024). A personalized product recommendation model in e-commerce based on retrieval strategy. *Journal of Open Innovation: Technology, Market, and Complexity*, 10(2), 100303.
<https://doi.org/10.1016/j.joitmc.2024.100303>

Sam, Yet Huat & Leong, Pui & Ku, Cheng. (2023). The Implementation of Mobile Application Ordering System to Optimize The User Experience of Food and Beverage Industry. 22-26.
10.1109/ICSGRC57744.2023.10215430.

Scott Taylor, Jr. (2020): Campus dining goes mobile: Intentions of college students to adopt a mobile food-ordering app, *Journal of Foodservice Business Research*, DOI: 10.1080/15378020.2020.1814087

Shah, Z., & Rai, S. (2022). A research paper on the effects of customer feedback on business. *International Journal of Advanced Research in Science, Communication and Technology*, 672-675. <https://doi.org/10.48175/IJARST-5743>

Shahril, Z., Zulkafly, H. A., Ismail, N. S., & Sharif, N. U. N. M. (2021). Customer Satisfaction Towards Self-Service Kiosks for Quick Service Restaurants (QSRs) in Klang Valley. *International Journal of Academic Research in Business and Social Sciences*, 11(13), 54–72.

Statista. (2023). Mobile App Usage in the Hospitality Industry. Retrieved from www.statista.com

Tamondong, D. R. A., Cabus, E. A. S., Echalar, A. R. B., Santos, J. J. A., & Bernarte, R. (20222023). Assessment of self-service kiosk as food service tool among customers in top fastfood chains in Manila. *LATHALA e-Journal*. Retrieved from <https://manila.lpu.edu.ph/publications/lathala-e-journal/assessment-of-self-service-kiosk-as-food-service-tool-among-customers-in-top-fast-food-chains-in-manila/>

- Yeung S. G. D. et al., "Expedition of Bakery Processes for Increased In-Store Productivity Through a POS Object Detector Module for Pastries," 2023 IEEE 15th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM), Coron, Palawan, Philippines, 2023, pp. 1-6, doi: 10.1109/HNICEM60674.2023.10589009.
- Yong, J. (2022). Factors influencing consumers' intention to use self-ordering kiosk in restaurant (Final Year Project, Universiti Tunku Abdul Rahman). UTAR Institutional Repository

APPENDICES

APPENDIX A. RESOURCE PERSONS

Owner of 28 Treasure HK Roast + Dimsum:

Chester Chua

Restaurant Owner

Ronaldo Convento Restaurant

Owner

APPENDIX B. RELEVANT SOURCE CODE

User Menu Interface

```

61 <p class="instructions">Instructions: @item.Instructions</p>
62 </div>
63 <p class="item-total">P @FormatPrice(item.FoodPrice * item.Quantity)</p>
64 </div>
65 </div>
66 </div>
67 </div>
68 <div class="order-footer">
69 <strong>Total:</strong> P @FormatPrice(order.Items.Sum(i => i.FoodPrice * i.Quantity))
70 </div>
71 </div>
72 </div>
73 </div>
74 </div>
75 </div>
76 </div>
77 <div class="checkout-footer">
78 <button class="billout-btn" @onclick="BillOut">Bill Out</button>
79 <button class="cancel-btn" @onclick="ToggleCancelMode">Cancel</button>
80 </div>
81 <div>
82 <div class="waiting-modal-overlay">
83 <div class="waiting-modal">
84 <div class="modal-header">
85 <h3>Processing Your Order</h3>
86 </div>
87 <div class="modal-body">
88 <div class="spinner"></div>
89 <p>@waitingMessage</p>
90 </div>
91 </div>
92 </div>

```

Figure 50. Displaying order summary, billing options, and processing modal.

```

91 <div class="spinner"></div>
92 <p>@waitingMessage</p>
93 </div>
94 </div>
95 </div>
96 </div>
97 </div>
98 @code {
99 private List<TrackedOrderDTO> orders;
100 private bool showWaitingModal = false;
101 private string waitingMessage = "Waiting for POS to complete serving...";
102 private System.Threading.Timer statusTimer;
103 private bool cancelMode = false;
104 private TaskCompletionSource<bool> posDoneTcs;
105
106 protected override async Task OnInitializedAsync()
107 {
108     await LoadOrdersAsync();
109
110     statusTimer = new System.Threading.Timer(async _ =>
111     {
112         await UpdateOrdersStatusAsync();
113     }, null, 3000, 3000);
114
115     await SignalRService.InitializeAsync();
116     SignalRService.OnDoneServing += OnPosDoneServing;
117     SignalRService.OnOrderStatusChanged += OnOrderStatusChanged;
118 }
119
120 private async Task LoadOrdersAsync()
121 {
122     var allOrders = await SQLiteCartService.GetAllOrdersAsync();

```

Figure 51. Initializing asynchronous order loading and status update

```

118     }
119
120     private async Task LoadOrdersAsync()
121     {
122         var allOrders = await SQLiteCartService.GetAllOrdersAsync();
123         orders = allOrders.Select(MapToTracked).ToList();
124         await InvokeAsync(StateHasChanged);
125     }
126
127     private async Task UpdateOrdersStatusAsync()
128     {
129         if (orders != null)
130         {
131             foreach (var order in orders)
132             {
133                 order.Status = GetOrderStatus(order);
134                 order.StatusIndex = GetOrderStatusIndex(order);
135             }
136             await InvokeAsync(StateHasChanged);
137         }
138     }
139
140     private void OnPosDoneServing(int tableNumber) => posDoneTcs?.TrySetResult(true);
141
142     private async void OnOrderStatusChanged(int tableNumber, string status)
143     {
144         if (orders == null) return;
145
146         foreach (var order in orders)
147         {
148             if (order.TableNumber == tableNumber)
149             {

```

Figure 52. Loading orders, updating statuses, and handling serving completion events.

```

145         }
146     }
147     {
148         if (order.TableNumber == tableNumber)
149         {
150             order.Status = status;
151             order.StatusIndex = GetOrderStatusIndex(order);
152             await SQLiteCartService.UpdateOrderStatusAsync(order.Id, status);
153         }
154     }
155     await InvokeAsync(StateHasChanged);
156 }
157
158 // -----
159 // CANCEL FEATURE
160 // -----
161 private void ToggleCancelMode()
162 {
163     cancelMode = !cancelMode;
164 }
165
166 private async void OnItemClicked(int orderId, int itemId)
167 {
168     if (!cancelMode) return;
169
170     bool confirm = await JS.InvokeAsync<bool>("confirm", "Cancel this item?");
171     if (confirm)
172         await CancelOrderItem(orderId, itemId);
173
174     cancelMode = false;
175     await LoadOrdersAsync();
176 }

```

Figure 53. Updating order status and implementing the cancel feature.


```

172      await CancelOrderItem(orderId, itemId);
173
174      cancelMode = false;
175      await LoadOrdersAsync();
176    }
177
178    private async Task CancelOrderItem(int orderId, int itemId)
179    {
180      await SQLiteCartService.RemoveItemFromOrderAsync(orderId, itemId);
181
182      // Optional: notify POS if connected
183      if (SignalRService.IsConnected)
184      {
185        await SignalRService.SendItemCancellationAsync(orderId, itemId);
186      }
187    }
188
189    // -----
190    // BILL OUT
191    // -----
192    private async Task BillOut()
193    {
194      try
195      {
196        if (!SignalRService.IsConnected)
197          await SignalRService.InitializeAsync();
198
199        showWaitingModal = true;
200        waitingMessage = "Sending request to POS...";
201        StateHasChanged();
202
203        posDoneTcs = new TaskCompletionSource<bool>();

```

Figure 54. Implementing order cancellation and bill-out request functions.

```

190    // BILL OUT
191    // -----
192    private async Task BillOut()
193    {
194      try
195      {
196        if (!SignalRService.IsConnected)
197          await SignalRService.InitializeAsync();
198
199        showWaitingModal = true;
200        waitingMessage = "Sending request to POS...";
201        StateHasChanged();
202
203        posDoneTcs = new TaskCompletionSource<bool>();
204        SignalRService.OnDoneServing += OnPosDoneServing;
205
206        await SignalRService.EndSessionAsync();
207
208        _ = ShowWaitingMessagesAsync();
209        await posDoneTcs.Task;
210
211        var allOrders = await SQLiteCartService.GetAllOrdersAsync();
212        if (allOrders.Any())
213        {
214          ReceiptService.ReceiptOrders = allOrders.Select(MapToTracked).ToList();
215          ReceiptService.TotalAmount = allOrders.Sum(o => o.Items.Sum(i => i.FoodPrice * i.Quantity));
216        }
217
218        await SQLiteCartService.ClearAllOrdersAsync();
219
220        showWaitingModal = false;
221        NavigationManager.NavigateTo("/receipt");

```

Figure 55. Executing the bill-out process and handling receipt generation.

```

190 // BILL OUT
191 // -----
192 private async Task BillOut()
193 {
194     try
195     {
196         if (!SignalRService.IsConnected)
197             await SignalRService.InitializeAsync();
198
199         showWaitingModal = true;
200         waitingMessage = "Sending request to POS...";
201         StateHasChanged();
202
203         posDoneTcs = new TaskCompletionSource<bool>();
204         SignalRService.OnDoneServing += OnPosDoneServing;
205
206         await SignalRService.EndSessionAsync();
207
208         _ = ShowWaitingMessagesAsync();
209         await posDoneTcs.Task;
210
211         var allOrders = await SQLiteCartService.GetAllOrdersAsync();
212         if (allOrders.Any())
213         {
214             ReceiptService.ReceiptOrders = allOrders.Select(MapToTracked).ToList();
215             ReceiptService.TotalAmount = allOrders.Sum(o => o.Items.Sum(i => i.FoodPrice * i.Quantity));
216         }
217
218         await SQLiteCartService.ClearAllOrdersAsync();
219
220         showWaitingModal = false;
221         NavigationManager.NavigateTo("/receipt");
222     }
223 }

```

Figure 56. Executing the bill-out process and handling receipt generation.

```

214 ReceiptService.ReceiptOrders = allOrders.Select(MapToTracked).ToList();
215 ReceiptService.TotalAmount = allOrders.Sum(o => o.Items.Sum(i => i.FoodPrice * i.Quantity));
216 }
217
218 await SQLiteCartService.ClearAllOrdersAsync();
219
220 showWaitingModal = false;
221 NavigationManager.NavigateTo("/receipt");
222 }
223 catch (Exception ex)
224 {
225     Console.WriteLine($"✗ BillOut failed: {ex.Message}");
226     showWaitingModal = false;
227 }
228 finally
229 {
230     SignalRService.OnDoneServing -= OnPosDoneServing;
231 }
232 }
233
234 private async Task ShowWaitingMessagesAsync()
235 {
236     var messages = new[]
237     {
238         "Waiting for POS to confirm serving...",
239         "Printing receipt...",
240         "Almost done..."
241     };
242
243     int i = 0;
244     while (!posDoneTcs.Task.IsCompleted)
245     {

```

Figure 57. Executing the bill-out process, clearing cart data, and handling receipt generation with asynchronous status messages.

```

244 while (!posDoneTcs.Task.IsCompleted)
245 {
246     waitingMessage = messages[i % messages.Length];
247     i++;
248     await InvokeAsync(StateHasChanged);
249     await Task.Delay(1500);
250 }
251
252 // -----
253 // HELPERS
254 // -----
255 private string FormatPrice(decimal value) => value.ToString("F2");
256 private string GetImageUrl(CartItemDTO item) => string.IsNullOrEmpty(item.ImageUrl) ? "images/placeholder.p
257
258 private List<string> GetCustomizations(CartItemDTO item)
259 {
260     if (string.IsNullOrEmpty(item.CustomizationsJson)) return new List<string>();
261     try
262     {
263         var list = System.Text.Json.JsonSerializer.Deserialize<List<AddOnOptionDTO>>(item.CustomizationsJson);
264         return list?.Select(a => a.Name).ToList() ?? new List<string>();
265     }
266     catch { return new List<string>(); }
267 }
268
269 private string GetOrderStatus(OrderDTO order)
270 {
271     var minutes = (DateTime.Now - order.OrderDate).TotalMinutes;
272     return minutes < 5 ? "Pending" : minutes < 10 ? "Preparing" : minutes < 20 ? "Delivering" : "Delivered";
273 }
274
275

```

Figure 58. Displaying dynamic waiting messages and defining helper methods for formatting prices, retrieving images, parsing customizations, and determining real-time order status.

```

275
276 private int GetOrderStatusIndex(OrderDTO order)
277 {
278     var status = GetOrderStatus(order);
279     return status switch { "Pending" => 0, "Preparing" => 1, "Delivering" => 2, "Delivered" => 3, _ => 0 };
280 }
281
282 private int GetProgressPercentage(TrackedOrderDTO order)
283 {
284     var minutes = (DateTime.Now - order.OrderDate).TotalMinutes;
285     return (int)Math.Min(100, (minutes / 25.0) * 100);
286 }
287
288 private TrackedOrderDTO MapToTracked(OrderDTO order)
289 {
290     return new TrackedOrderDTO
291     {
292         Id = order.Id,
293         OrderDate = order.OrderDate,
294         Items = order.Items.Select(i => new AnimatedCartItemDTO
295         {
296             Id = i.Id,
297             FoodName = i.FoodName,
298             FoodPrice = i.FoodPrice,
299             Quantity = i.Quantity,
300             PreparationTime = i.PreparationTime,
301             CustomizationsJson = i.CustomizationsJson,
302             Instructions = i.Instructions,
303             ImageUrl = i.ImageUrl
304         }).ToList(),
305         Status = GetOrderStatus(order),
306         StatusIndex = GetOrderStatusIndex(order)
307     }
308 }

```

Figure 59. Calculating order status index and progress percentage while mapping order details into a tracked order model for real-time status monitoring and display.

POS INTERFACE

```

1  @using SenderApp.Services
2  @using SenderApp.Dtos
3  @inject ChatService ChatService
4  @inject IJSRuntime JSRuntime
5  @inject NavigationManager Navigation
6  @inject AuthService AuthService
7  @inherits LayoutComponentBase
8  @inject OrderService OrderService
9  @inject PosSignalRService SignalRService
10 @implements IDisposable
11
12 <!-- o Toast Notification -->
13 <div id="toast-container" class="toast-container"></div>
14 <audio id="notifySound" src="sounds/incoming-notif.mp3" preload="auto"></audio>
15
16 <div class="sidebar-overlay @(sidebarOpen ? "active" : "")" @onclick="ToggleSidebar"></div>
17
18 <div class="nav-menu @(sidebarOpen ? "open" : "mini")">
19   <nav class="flex-column">
20     @if (AuthService.Role == "Cashier")
21     {
22       <NavLink class="nav-link d-flex align-items-center text-white" href="/menu" Match="NavLinkMatch.All" @onclick="CloseS
23       <i class="fa-solid fa-cart-shopping me-2"></i>
24       @if (sidebarOpen) { <span>Menu/POS</span> }
25     }
26   }
27
28   <NavLink class="nav-link d-flex align-items-center text-white justify-content-between" href="/incomingorders" @oncl
29   <div class="d-flex align-items-center gap-2">
30     <i class="fa-solid fa-arrow-up-from-bracket"></i>
31     @if (sidebarOpen) { <span>Incoming Orders</span> }
32   }
33   @if (OrderService.OrdersByTable.Any())
34   {

```

Figure 60. Defining the main layout structure with injected services, sidebar navigation, and role-based access for cashier functionalities in the POS system.

```

37   }
38   </NavLink>
39
40   <NavLink class="nav-link d-flex align-items-center text-white justify-content-between" href="/pos" @onclick="CloseSid
41   <div class="d-flex align-items-center gap-2">
42     <i class="fa-solid fa-table-cells-large me-2"></i>
43     @if (sidebarOpen) { <span>Tables</span> }
44   }
45
46   <div class="d-flex align-items-center gap-1">
47     @if (occupiedCount > 0)
48     {
49       <span class="badge tables-badge @(sidebarOpen ? "expanded" : "collapsed")">@occupiedCount</span>
50     }
51     @if (totalUnreadAcrossTables > 0)
52     {
53       <span class="badge chat-unread-badge @(sidebarOpen ? "expanded" : "collapsed")" title="@($"{totalUnreadA
54       } @totalUnreadAcrossTables > 9 ? "9+" : totalUnreadAcrossTables.ToString())"
55     }
56   }
57   </div>
58   </NavLink>
59
60   <NavLink class="nav-link d-flex align-items-center text-white" href="/print" @onclick="CloseSidebar">
61     <i class="fa-solid fa-print me-2"></i>
62     @if (sidebarOpen) { <span>Print Receipt</span> }
63   }
64   </NavLink>
65
66   <NavLink class="nav-link d-flex align-items-center text-white" href="/history" @onclick="CloseSidebar">
67     <i class="fa-solid fa-clock-rotate-left me-2"></i>
68     @if (sidebarOpen) { <span>Transaction History</span> }
69   }
70   </NavLink>

```

Figure 61. Creating sidebar navigation links for table management, receipt printing, and transaction history, with dynamic badges showing occupied tables and unread chat messages.

```

64
65 <NavLink class="nav-link d-flex align-items-center text-white" href="history" @onclick="CloseSidebar">
66   <i class="fa-solid fa-clock-rotate-left me-2"></i>
67   @if (sidebarOpen) { <span>Transaction History</span> }
68 </NavLink>
69
70 @if (AuthService.Role == "Admin")
71 {
72   <NavLink class="nav-link d-flex align-items-center text-white" href="admin-dashboard" @onclick="CloseSidebar">
73     <i class="fa-solid fa-gear me-2"></i>
74     @if (sidebarOpen) { <span>Admin Dashboard</span> }
75   </NavLink>
76 }
77
78 <button class="nav-link d-flex align-items-center text-white bg-transparent border-0" @onclick="ConfirmLogout">
79   <i class="fa-solid fa-right-from-bracket me-2"></i>
80   @if (sidebarOpen) { <span>Logout</span> }
81 </button>
82 </div>
83 </nav>
84 </div>
85
86 <ChatPersistence />
87
88 <main class="d-flex flex-column align-items-start">
89   <div class="top-row ps-4 pe-2 d-flex justify-content-between align-items-center w-100">
90     <button class="sidebar-toggle me-2" @onclick="ToggleSidebar" aria-label="Toggle sidebar">
91       <i class="fa-solid fa-bars"></i>
92     </button>
93
94     <div class="d-flex align-items-center gap-2">
95       
96       <h5 class="mb-0">28 Treasures</h5>
97     </div>
98   </div>
99 </main>

```

Figure 62. Implementing sidebar links for transaction history, admin dashboard, and logout options, along with a responsive main layout header displaying the restaurant logo and navigation controls.

```

100 </div>
101
102 <article class="content ps-4 w-100">
103   @Body
104 </article>
105
106 <OrderNotifications />
107 </main>
108
109 @if (ShowLogoutConfirmation)
110 {
111   <div class="position-fixed top-0 start-0 w-100 h-100 d-flex align-items-center justify-content-center bg-dark bg-opacity-50">
112     <div class="bg-white p-4 rounded shadow-lg text-center" style="max-width: 300px; width: 100%;">
113       <p class="mb-3">Are you sure you want to logout?</p>
114       <div class="d-flex justify-content-center gap-2">
115         <button @onclick="Logout" class="btn btn-danger">Yes, Logout</button>
116         <button @onclick="CancelLogout" class="btn btn-secondary">Cancel</button>
117       </div>
118     </div>
119   </div>
120 }
121
122 @code {
123   private bool sidebarOpen = false;
124   private bool ShowLogoutConfirmation = false;
125   private int occupiedCount = 0;
126   private int totalUnreadAcrossTables = 0;
127
128   private Action<int, string, string, Guid>? _signalRChatHandler;
129   private Func<Task>? _chatServiceHandler;
130   private readonly HashSet<Guid> _processedMessageIds = new();
131   private readonly HashSet<int> _recentOccupiedTables = new();
132 }

```

Figure 63. Managing the main layout's interactive components, including logout confirmation modal, dynamic sidebar states, and real-time tracking of occupied tables and unread messages using SignalR integration

```

133 // Toast queue mechanism
134 private bool _toastActive = false;
135 private readonly Queue<string> _toastQueue = new();
136
137 private void ToggleSidebar() => sidebarOpen = !sidebarOpen;
138 private void CloseSidebar() => sidebarOpen = false;
139 private void ConfirmLogout() => ShowLogoutConfirmation = true;
140 private void Logout()
141 {
142     ShowLogoutConfirmation = false;
143     Navigation.NavigateTo("/Login");
144 }
145 private void CancelLogout() => ShowLogoutConfirmation = false;
146
147 protected override async Task OnInitializedAsync()
148 {
149     try
150     {
151         if (!SignalRService.IsConnected)
152             await SignalRService.ConnectAsync();
153     }
154     catch (Exception ex)
155     {
156         Console.WriteLine($"[MainLayout] SignalR connect failed: {ex.Message}");
157     }
158
159     UpdateOccupiedCount();
160     UpdateTotalUnread();
161
162     SignalRService.TableOccupiedEvent += OnTableStatusChanged;
163     SignalRService.TableVacantEvent += OnTableStatusChanged;
164
165     _signalRChatHandler = (table, sender, message, messageId) =>

```

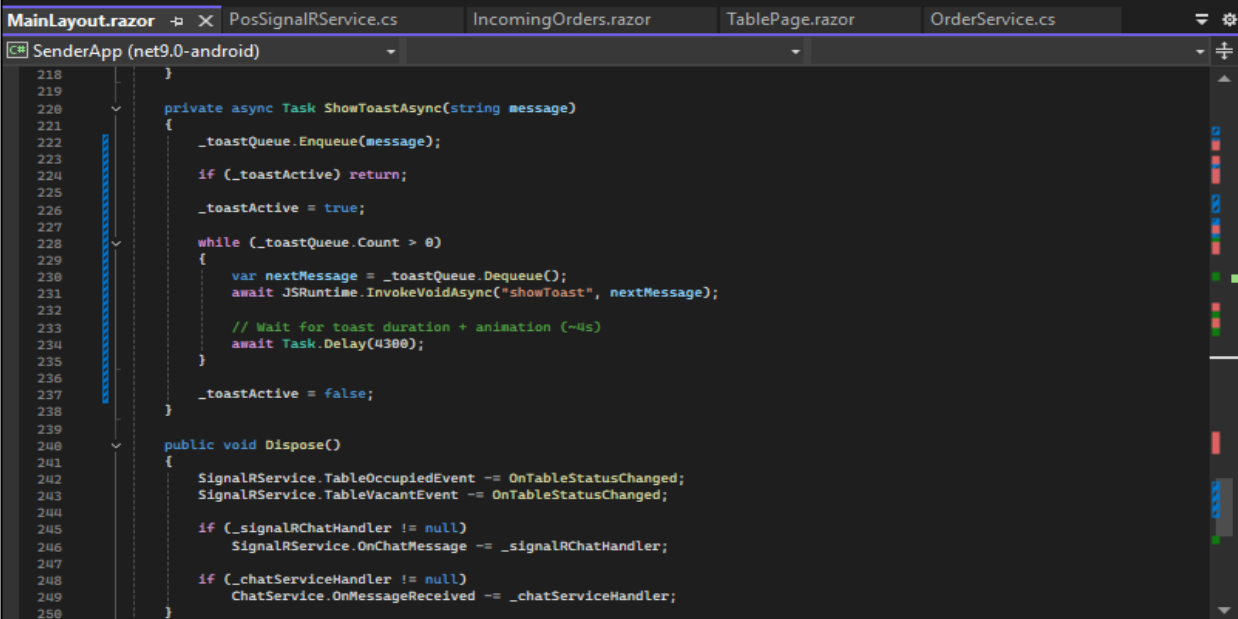
Figure 64. Initializing real-time SignalR connections, handling sidebar and logout actions, and setting up event-driven updates for table occupancy and chat notifications within the POS main layout\

```

161     SignalRService.TableOccupiedEvent += OnTableStatusChanged;
162     SignalRService.TableVacantEvent += OnTableStatusChanged;
163
164     _signalRChatHandler = (table, sender, message, messageId) =>
165     {
166         if (_processedMessageIds.Contains(messageId)) return;
167         _processedMessageIds.Add(messageId);
168
169         var chatMsg = new TableChatMessage
170         {
171             MessageId = messageId,
172             Sender = sender,
173             Content = message,
174             Timestamp = DateTime.Now
175         };
176
177         _ = ChatService.SendMessageToTable(table, chatMsg);
178     };
179     SignalRService.OnChatMessage += _signalRChatHandler;
180
181     _chatServiceHandler = async () =>
182     {
183         UpdateTotalUnread();
184         await InvokeAsync(StateHasChanged);
185     };
186     ChatService.OnMessageReceived += _chatServiceHandler;
187
188
189     private async void OnTableStatusChanged(int tableNumber)
190     {
191         UpdateOccupiedCount();
192     }
193

```

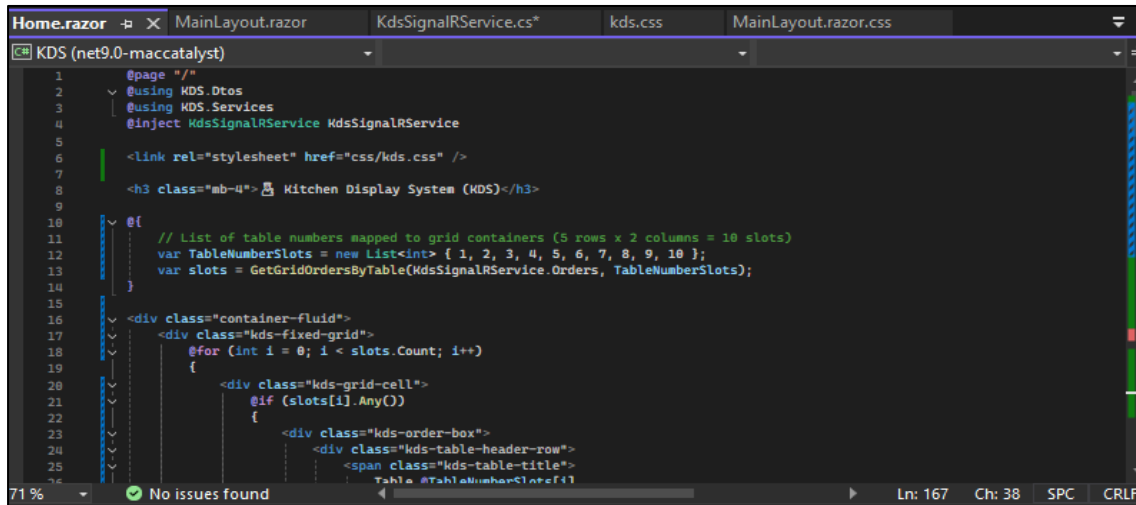
Figure 65. Handling real-time chat and table status updates through SignalR, processing messages to avoid duplication, and updating the POS interface asynchronously for live communication.



```
218     }
219
220     private async Task ShowToastAsync(string message)
221     {
222         _toastQueue.Enqueue(message);
223
224         if (!_toastActive) return;
225
226         _toastActive = true;
227
228         while (_toastQueue.Count > 0)
229         {
230             var nextMessage = _toastQueue.Dequeue();
231             await JSRuntime.InvokeVoidAsync("showToast", nextMessage);
232
233             // Wait for toast duration + animation (~4s)
234             await Task.Delay(4300);
235         }
236
237         _toastActive = false;
238     }
239
240     public void Dispose()
241     {
242         SignalRService.TableOccupiedEvent -= OnTableStatusChanged;
243         SignalRService.TableVacantEvent -= OnTableStatusChanged;
244
245         if (_signalRChatHandler != null)
246             SignalRService.OnChatMessage -= _signalRChatHandler;
247
248         if (_chatServiceHandler != null)
249             ChatService.OnMessageReceived -= _chatServiceHandler;
250     }
```

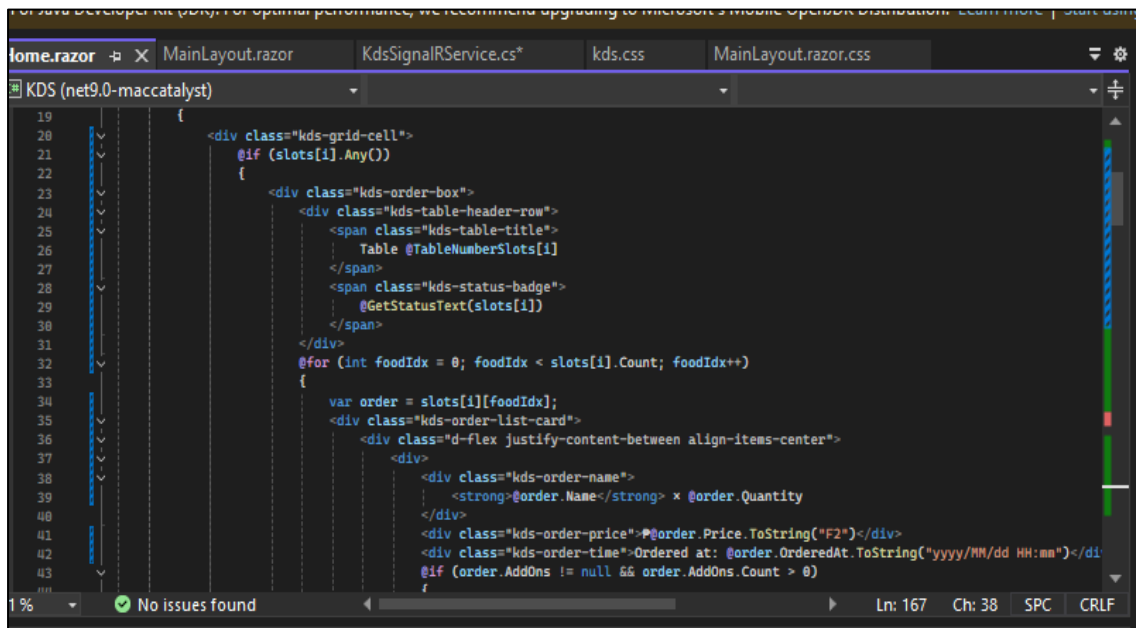
Figure 66. Displays queued toast messages and handles cleanup of SignalR and chat events.

KITCHEN DISPLAY SYSTEM (KDS)



```
1 @page "/"
2 @using KDS.Dtos
3 @using KDS.Services
4 @inject KdsSignalRService KdsSignalRService
5
6 <link rel="stylesheet" href="css/kds.css" />
7
8 <h3 class="mb-4"> Kitchen Display System (KDS)</h3>
9
10 @[
11 // List of table numbers mapped to grid containers (5 rows x 2 columns = 10 slots)
12 var TableNumberSlots = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
13 var slots = GetGridOrdersByTable(KdsSignalRService.Orders, TableNumberSlots);
14 ]
15
16 <div class="container-fluid">
17 <div class="kds-fixed-grid">
18 @for (int i = 0; i < slots.Count; i++)
19 {
20 <div class="kds-grid-cell">
21 @if (slots[i].Any())
22 {
23 <div class="kds-order-box">
24 <div class="kds-table-header-row">
25 <span class="kds-table-title">
26 Table @TableNumberSlots[i]
27 </span>
28 <span class="kds-status-badge">
29 @GetStatusText(slots[i])
30 </span>
31 </div>
32 @for (int foodIdx = 0; foodIdx < slots[i].Count; foodIdx++)
33 {
34 var order = slots[i][foodIdx];
35 <div class="kds-order-list-card">
36 <div class="d-flex justify-content-between align-items-center">
37 <div>
38 <div class="kds-order-name">
39 <strong>@order.Name</strong> x @order.Quantity
40 </div>
41 <div class="kds-order-price">@order.Price.ToString("F2")</div>
42 <div class="kds-order-time">Ordered at: @order.OrderedAt.ToString("yyyy/MM/dd HH:mm")</div>
43 @if (order.AddOns != null && order.AddOns.Count > 0)
44 {
45 <div class="kds-add-ons">
46 @foreach (var addOn in order.AddOns)
47 {
48 <div class="kds-add-on">
49 <div class="kds-add-on-name">@addOn.Name</div>
50 <div class="kds-add-on-price">@addOn.Price.ToString("F2")</div>
51 </div>
52 }
53 </div>
54 </div>
55 </div>
56 </div>
57 </div>
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

Figure 67. Displays the Kitchen Display System (KDS) layout showing orders mapped to table slots for real-time kitchen monitoring



```
19 {
20 <div class="kds-grid-cell">
21 @if (slots[i].Any())
22 {
23 <div class="kds-order-box">
24 <div class="kds-table-header-row">
25 <span class="kds-table-title">
26 Table @TableNumberSlots[i]
27 </span>
28 <span class="kds-status-badge">
29 @GetStatusText(slots[i])
30 </span>
31 </div>
32 @for (int foodIdx = 0; foodIdx < slots[i].Count; foodIdx++)
33 {
34 var order = slots[i][foodIdx];
35 <div class="kds-order-list-card">
36 <div class="d-flex justify-content-between align-items-center">
37 <div>
38 <div class="kds-order-name">
39 <strong>@order.Name</strong> x @order.Quantity
40 </div>
41 <div class="kds-order-price">@order.Price.ToString("F2")</div>
42 <div class="kds-order-time">Ordered at: @order.OrderedAt.ToString("yyyy/MM/dd HH:mm")</div>
43 @if (order.AddOns != null && order.AddOns.Count > 0)
44 {
45 <div class="kds-add-ons">
46 @foreach (var addOn in order.AddOns)
47 {
48 <div class="kds-add-on">
49 <div class="kds-add-on-name">@addOn.Name</div>
50 <div class="kds-add-on-price">@addOn.Price.ToString("F2")</div>
51 </div>
52 }
53 </div>
54 </div>
55 </div>
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

Figure 68. Displays table orders in the KDS grid, showing order details, quantities, and status for each table slot.


```

37 <div>
38   <div class="kds-order-name">
39     <strong>@order.Name</strong> x @order.Quantity
40   </div>
41   <div class="kds-order-price">@order.Price.ToString("F2")</div>
42   <div class="kds-order-time">Ordered at: @order.OrderedAt.ToString("yyyy/MM/dd HH:mm")</div>
43   @if (order.AddOns != null && order.AddOns.Count > 0)
44   {
45     <div class="kds-addon">
46       Add-ons: @string.Join(" ", order.AddOns.Select(a => a.Name))
47     </div>
48   }
49   @if (!string.IsNullOrEmpty(order.CustomizationsJson))
50   {
51     <div class="kds-addon">
52       Customizations:
53       @if (
54         List<AddOnOptionDTO>? customizations = null;
55         try
56         {
57           customizations = System.Text.Json.JsonSerializer.Deserialize<List<AddOnOp
58         }
59         catch { }
60         if (customizations != null && customizations.Count > 0)
61         {
62           <span>@string.Join(" ", customizations.Select(r => r.Name))</span>
63         }
64       }
65     </div>
66   }
67 </div>

```

Figure 69. Displays detailed order information in the KDS, including item name, quantity, price, time ordered, and add-ons or customizations.

```

67   }
68   }
69   </div>
70   }
71   @if (!string.IsNullOrEmpty(order.Instructions))
72   {
73     <div class="kds-note">Note: @order.Instructions</div>
74   }
75   </div>
76   <button class="kds-done-btn ms-2"
77     @onclick="() => MarkDone(order)"
78     disabled="@((order.Status == "Delivered" || order.Status == "Done"))">
79     Done
80   </button>
81 </div>
82 </div>
83 @if (foodIdx < slots[i].Count - 1)
84 {
85   <hr class="kds-order-divder" />
86 }
87 </div>
88 <div class="kds-table-footer">
89   <button class="btn btn-warning kds-footer-btn me-2" @onclick="() => DeliverTable(TableNumbersSlots[i])>
90   <button class="btn btn-success kds-footer-btn" @onclick="() => MarkTableDelivered(TableNumbersSlots[i])>
91 </div>

```

Figure 70. Adds order notes, completion buttons, and table action controls for marking orders as done or delivered in the KDS interface.

```

91         </div>
92     </div>
93 }
94 else
95 {
96     <div class="kds-order-card kds-empty-card">
97         <div class="kds-table-header kds-empty-header">
98             Vacant (@TableNumberSlots[i])
99         </div>
100        <div class="kds-order-body kds-empty-body">
101            <span class="text-muted">No order</span>
102        </div>
103    </div>
104 }
105 </div>
106 }
107 </div>
108 </div>
109
110 @code {
111     // Returns a fixed grid of slots, each slot contains all orders for that table number
112     List<List<KdsOrderDTO>> GetGridOrdersByTable(IEnumerable<KdsOrderDTO> orders, List<int> TableNumberSlots)
113     {
114         var slotOrders = new List<List<KdsOrderDTO>>();
115         foreach (var tableNumber in TableNumberSlots)
116     {
117         }
118     }
119 }

```

Figure 71. Displays vacant table slots with a “No orders” message and includes logic to group orders by table number in the KDS system.

```

106     }
107 </div>
108 </div>
109
110 @code {
111     // Returns a fixed grid of slots, each slot contains all orders for that table number
112     List<List<KdsOrderDTO>> GetGridOrdersByTable(IEnumerable<KdsOrderDTO> orders, List<int> TableNumberSlots)
113     {
114         var slotOrders = new List<List<KdsOrderDTO>>();
115         foreach (var tableNumber in TableNumberSlots)
116         {
117             var ordersForTable = orders.Where(o => o.TableNumber == tableNumber).ToList();
118             slotOrders.Add(ordersForTable); // If no order, will be empty list
119         }
120         return slotOrders;
121     }
122
123     private string GetStatusText(List<KdsOrderDTO> orders)
124     {
125         if (orders.Any(o => o.Status == "Pending")) return "Pending";
126         if (orders.Any(o => o.Status == "Ready")) return "Ready";
127         return "Delivered";
128     }
129
130     private Action<KdsOrderDTO>? _onOrderReceivedHandler;
131 }

```

Figure 72. Displays backend code that groups orders by table and shows their status as Pending, Ready, or Delivered.

```

127         return "Delivered";
128     }
129
130     private Action<KdsOrderDTO>? _onOrderReceivedHandler;
131
132     protected override async Task OnInitializedAsync()
133     {
134         _onOrderReceivedHandler = order => InvokeAsync(StateHasChanged);
135         KdsSignalRService.OnOrderReceived += _onOrderReceivedHandler;
136         await KdsSignalRService.ConnectAsync();
137     }
138
139     public void Dispose()
140     {
141         if (_onOrderReceivedHandler != null)
142         {
143             KdsSignalRService.OnOrderReceived -= _onOrderReceivedHandler;
144         }
145     }
146
147     // Mark single food item as done
148     private async Task MarkDone(KdsOrderDTO order)
149     {
150         order.Status = "Done";
151         await KdsSignalRService.UpdateOrderStatusAsync(order.OrderId, "Done");
152         StateHasChanged();
153     }

```

Figure 73. Orders are grouped per table and shown with their status.

```

145     }
146
147     // Mark single food item as done
148     private async Task MarkDone(KdsOrderDTO order)
149     {
150         order.Status = "Done";
151         await KdsSignalRService.UpdateOrderStatusAsync(order.OrderId, "Done");
152         StateHasChanged();
153     }
154
155     // Mark all orders for a table as "Ready"
156     private async Task DeliverTable(int tableNumber)
157     {
158         var orders = KdsSignalRService.Orders.Where(o => o.TableNumber == tableNumber);
159         foreach (var order in orders)
160         {
161             order.Status = "Ready";
162             await KdsSignalRService.UpdateOrderStatusAsync(order.OrderId, "Ready");
163         }
164         StateHasChanged();
165     }
166
167     // Mark all orders for a table as "Delivered"
168     private async Task MarkTableDelivered(int tableNumber)
169     {
170         var orders = KdsSignalRService.Orders.Where(o => o.TableNumber == tableNumber);

```

Figure 74. Updates the status of order marks one as **Done**, all table orders as **Ready**, or all as **Delivered**, then refreshes the UI.

```

160     {
161         order.Status = "Ready";
162         await KdsSignalRService.UpdateOrderStatusAsync(order.OrderId, "Ready");
163     }
164     StateHasChanged();
165 }
166
167 // Mark all orders for a table as "Delivered"
168 private async Task MarkTableDelivered(int tableNumber)
169 {
170     var orders = KdsSignalRService.Orders.Where(o => o.TableNumber == tableNumber);
171     foreach (var order in orders)
172     {
173         order.Status = "Delivered";
174         await KdsSignalRService.UpdateOrderStatusAsync(order.OrderId, "Delivered");
175     }
176     StateHasChanged();
177 }
178

```

71 % No issues found Ln: 165 Ch: 6 SPC CRLF

Figure 75. Sets all orders for a table to **Delivered** and updates the UI.

API SERVER

```

1 using DigiDineAPI.Dtos;
2 using Microsoft.AspNetCore.SignalR.Client;
3 using System.Collections.Generic;
4 using System.Threading.Tasks;
5
6 namespace DigiDineAPI.Services
7 {
8     1 reference
9     public class SignalRService
10     {
11         private HubConnection hubConnection;
12
13         1 reference
14         public bool IsConnected => hubConnection?.State == HubConnectionState.Connected;
15
16         0 references
17         public async Task InitializeAsync()
18         {
19             if (hubConnection != null && IsConnected)
20                 return;
21
22             hubConnection = new HubConnectionBuilder()
23                 .WithUrl("http://192.168.254.140:5001/orderHub") // [X] your Web API URL
24                 .WithAutomaticReconnect()
25                 .Build();
26         }
27     }
28 }

```

78 % 0 1 Ln: 8 Ch: 4 SPC CRLF

Figure 76. Initializes a Signal connection to the Web API hub for real-time order communication.

APPENDIX C. EVALUATION TOOLS/TEST DOCUMENTS

The researchers employed surveys as the primary evaluation instrument to assess the functionality, usability, and overall effectiveness of the DIGIDINE Ordering Management System. The surveys were administered to three key user groups—restaurant staff, administrators, and customers—to obtain comprehensive feedback and insights from multiple perspectives. Through these instruments, the researchers were able to systematically gather data concerning the participants' experiences, satisfaction levels, and perceptions regarding the system's efficiency, reliability, and user-friendliness. This evaluative approach facilitated an understanding of how the DIGIDINE system contributes to operational improvement, service quality enhancement, and customer satisfaction. The collected data served as a substantial foundation for the analysis of the system's performance and for the formulation of evidence-based conclusions and recommendations aimed at further development and refinement of the system.

For Users/Employees:

1. How easy is it to learn and use the DIGIDINE system?
2. How well does DIGIDINE help you process customer orders?
3. How often does the system respond quickly without lag or errors?
4. Which DIGIDINE feature do you use the most?
5. How accurate are the orders displayed or printed by the system?
6. How easy is it to communicate with the kitchen or cashier through DIGIDINE?
7. How helpful is DIGIDINE in managing your daily tasks?
8. How satisfied are you with the design and layout of the screens?
9. How reliable is the system during busy hours?
10. Overall, how satisfied are you with DIGIDINE as a work tool?

For Admin:

1. How easy is it to manage and configure the DIGIDINE system?
2. How effective is DIGIDINE in monitoring sales and transactions?
3. How satisfied are you with the reporting and analytics features?
4. How reliable is the system in handling multiple users and devices at once?
5. How secure do you feel the system is in protecting business and customer data?
6. How easy is it to update menus, prices, and product details in the system?
7. How useful are the admin control features (user access, logs, reports)?
8. How responsive is the system during peak hours or heavy transactions?
9. How satisfied are you with the accuracy of sales and inventory records?
10. Overall, how satisfied are you with DIGIDINE's performance and management tools?

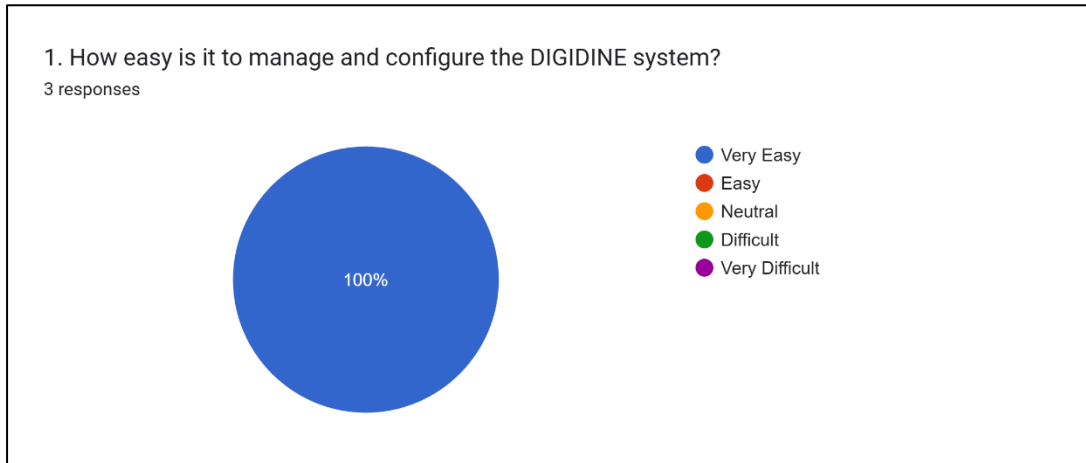
For Customers:

1. How easy is it to place an order using the DIGIDINE app?
2. How satisfied are you with the speed of the ordering process?
3. How accurate was your order upon receiving it?
4. How would you rate the design and layout of the DIGIDINE app?
5. How easy is it to browse the menu and find items you want to order?
6. How satisfied are you with the payment options available in the app?
7. How reliable is the system in confirming and updating your order status?
8. How would you rate the overall speed of service from order to delivery/pickup?
9. How likely are you to use DIGIDINE again for your next order?
10. Overall, how satisfied are you with your DIGIDINE experience?

APPENDIX D. SAMPLE INPUT/OUTPUT/REPORTS

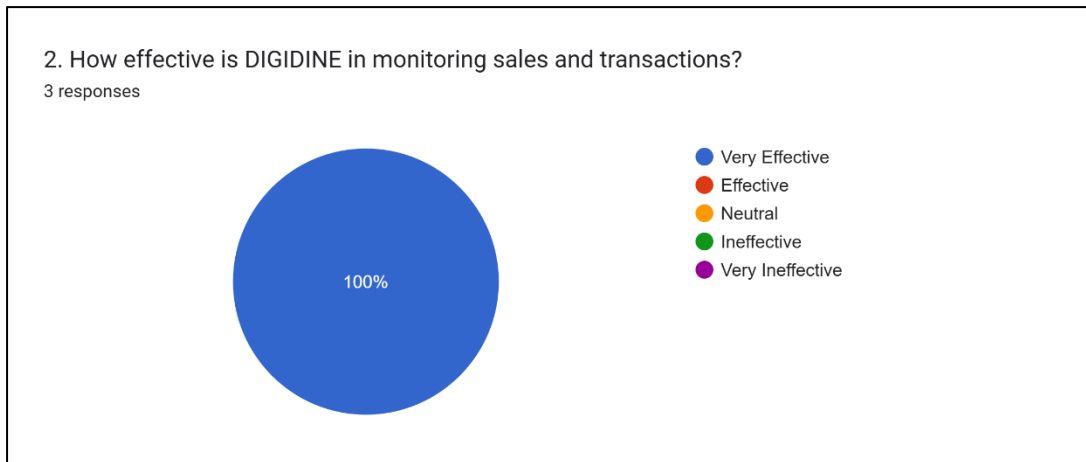
For Admin:

Question #1:



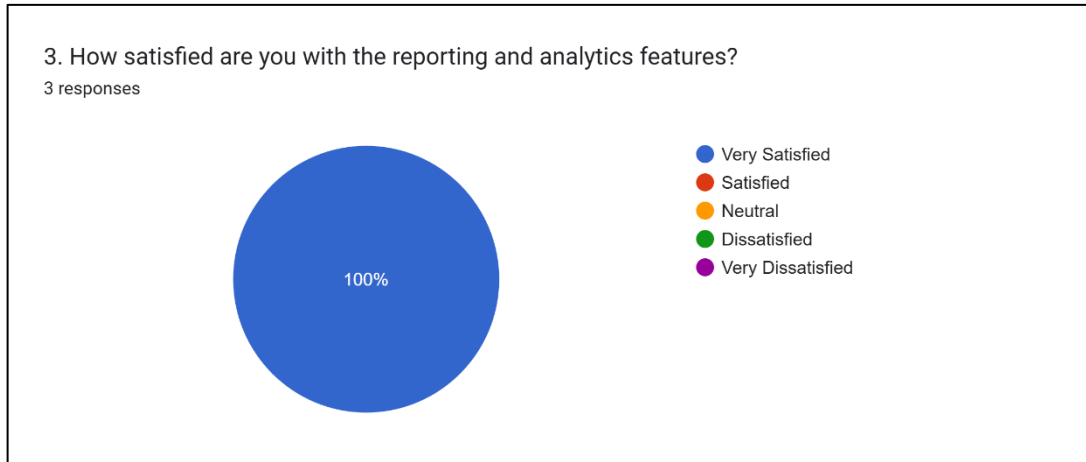
The survey results show that 100% of respondents rated the DIGIDINE system as “Very Easy” to manage and configure. This indicates that the system is highly user-friendly, efficient, and easy to operate, demonstrating its effectiveness in providing a smooth user experience.

Question #2:



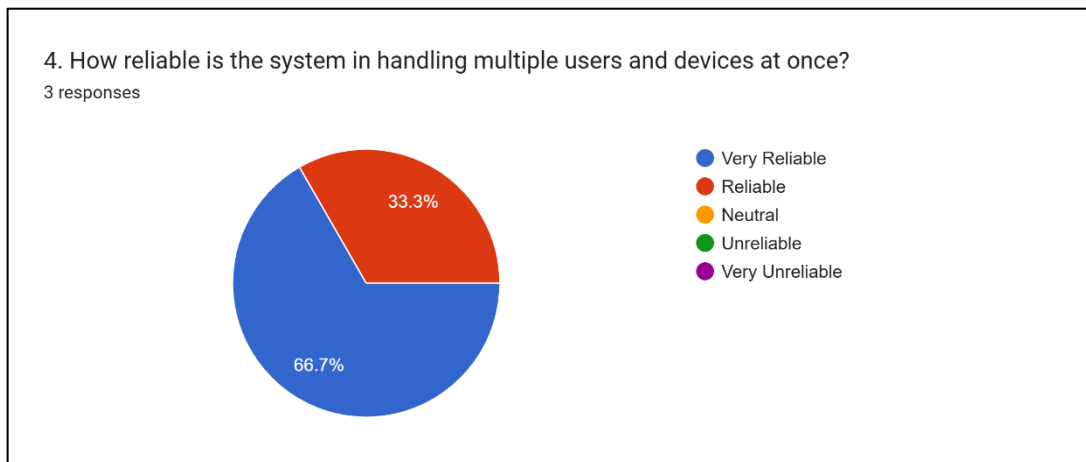
The figure shows that 100% of respondents rated DIGIDINE as “**Very Effective**” in monitoring sales and transactions. This result indicates that users find the system highly reliable and efficient in tracking sales and managing transaction data.

Question #3:



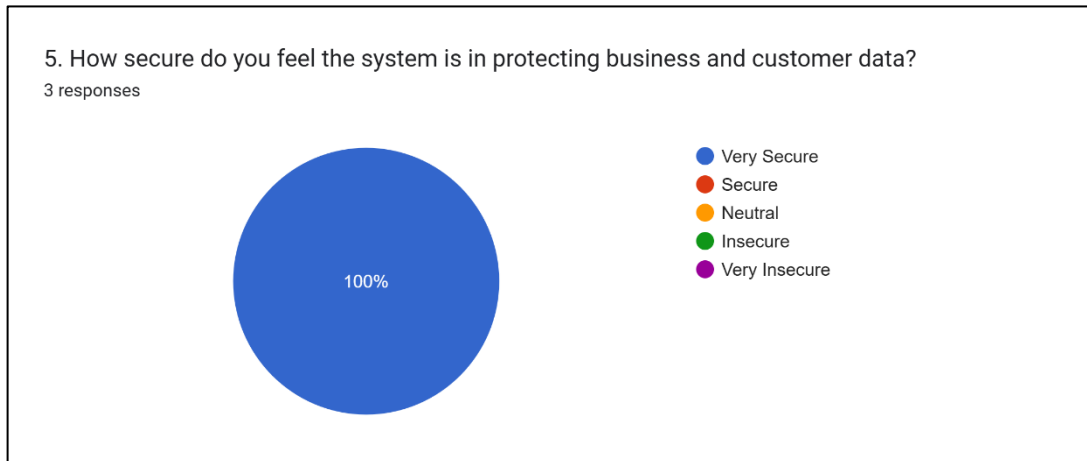
The figure shows that 100% of respondents are **“Very Satisfied”** with DIGIDINE’s reporting and analytics features. This indicates that users find these functions highly effective in providing accurate insights and useful data for business monitoring.

Question #4:



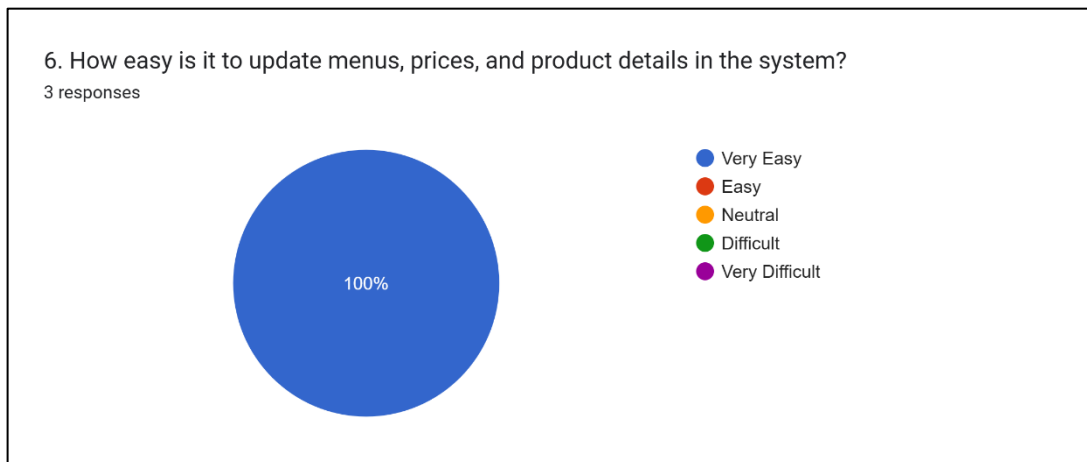
The figure shows that **66.7%** of respondents rated the system as **“Very Reliable”**, while **33.3%** rated it as **“Reliable.”** This indicates that users find the system dependable when handling multiple users and devices simultaneously.

Question #5:



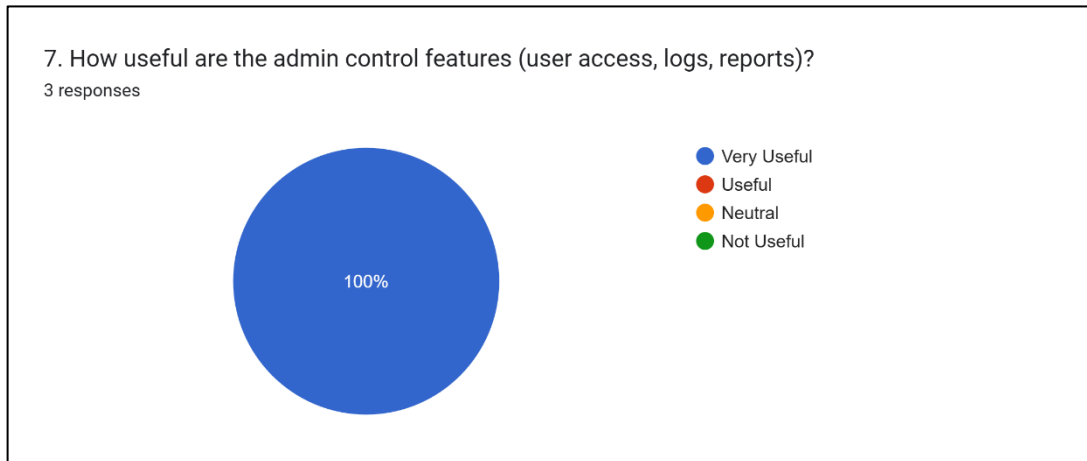
The figure shows that **100%** of respondents rated the system as “**Very Secure.**” This means all users trust the system’s ability to protect both business and customer data effectively.

Question #6:



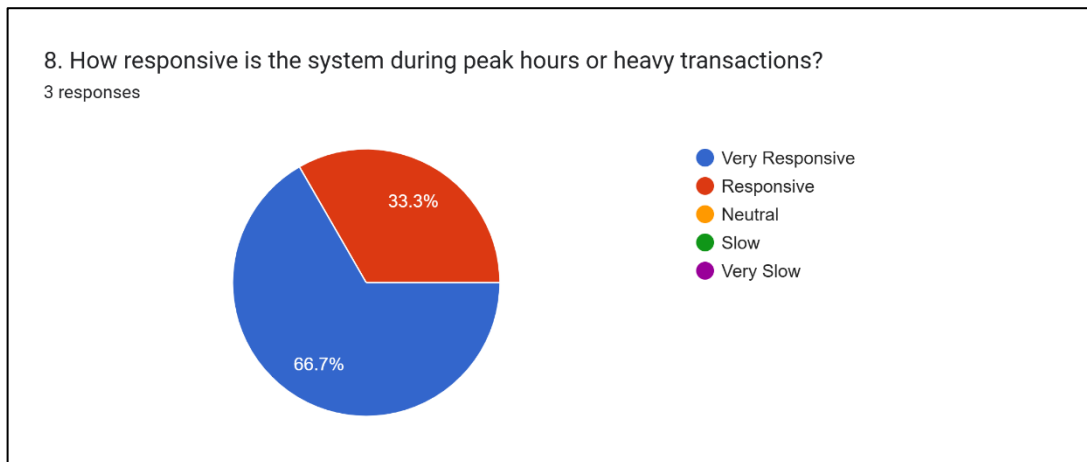
The figure shows that **100%** of respondents rated the system as “**Very Easy**” to update menus, prices, and product details. This means users find the system simple and convenient to manage and maintain.

Question #7:



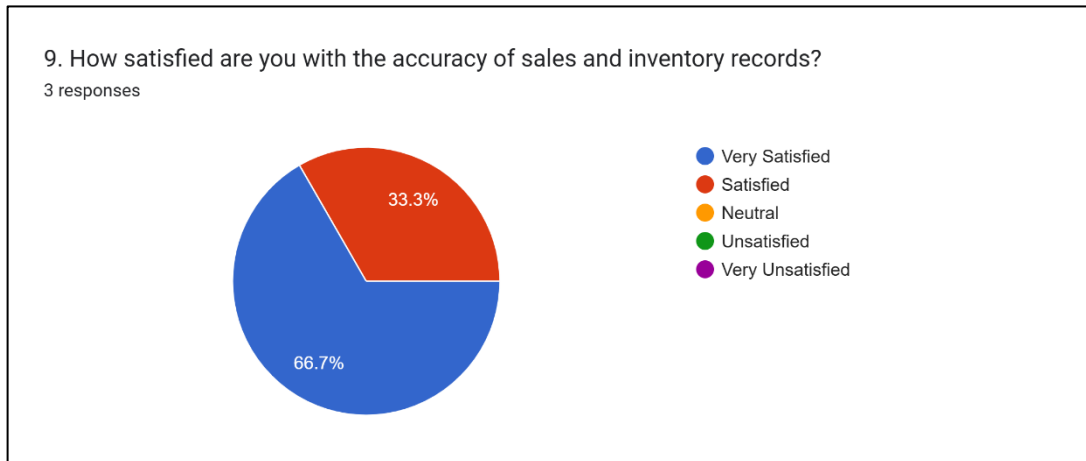
The figure shows that **100%** of respondents rated the admin control features as **“Very Useful.”** This indicates that users find the tools for user access, logs, and reports highly beneficial for system management.

Question #8:



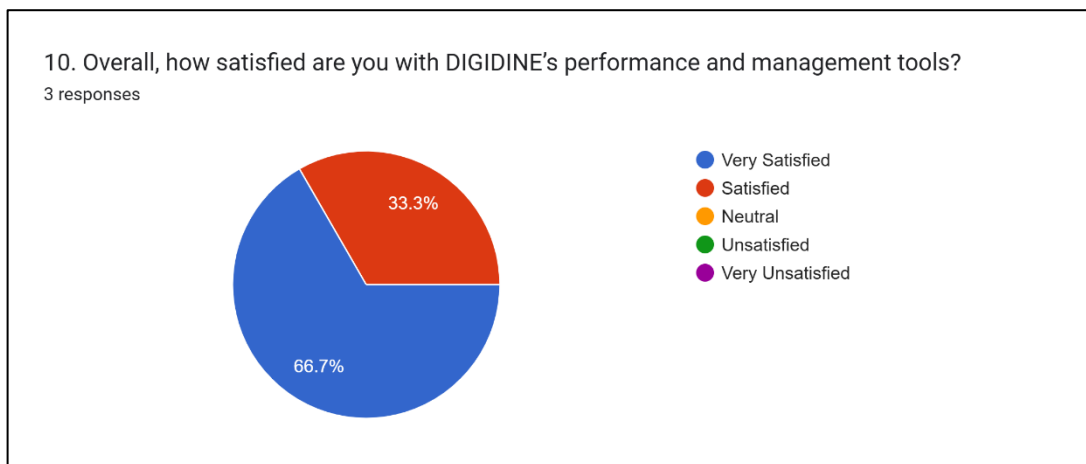
All employees (100%) reported being very satisfied with DIGIDINE’s design and screen layout, indicating a clear, visually appealing, and user-friendly interface.

Question #9:



Most employees (**87.5%**) found DIGIDINE **very reliable** during busy hours, while **12.5%** rated it **reliable**, showing that the system performs consistently well even under high workload.

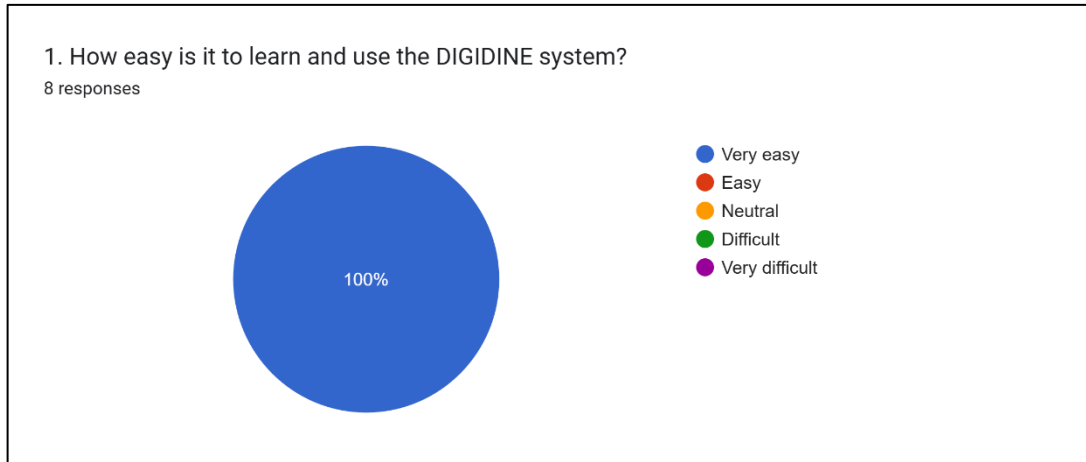
Question #10:



The figure shows that **66.7%** of respondents are “**Very Satisfied**” and **33.3%** are “**Satisfied**” with DIGIDINE’s performance and management tools. This means all users are pleased with how the system performs and supports management tasks.

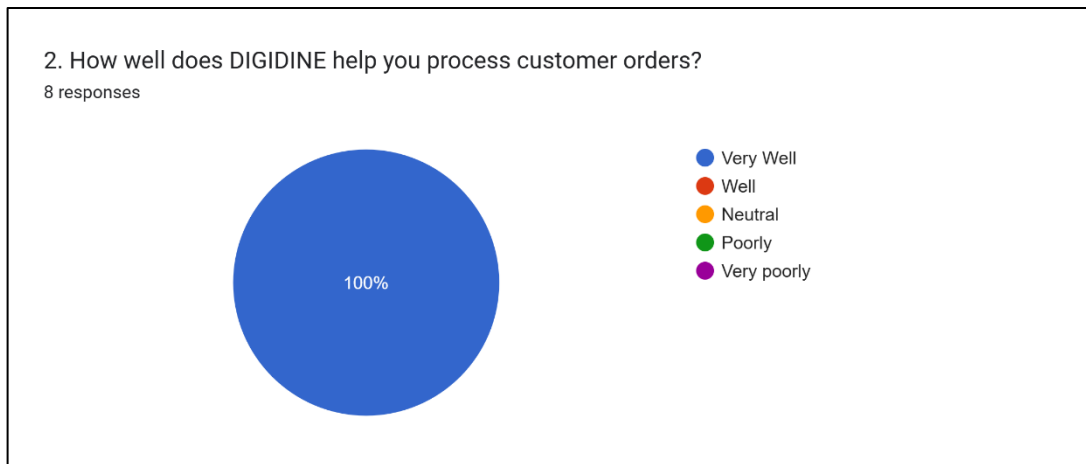
For Employees:

Question #1:



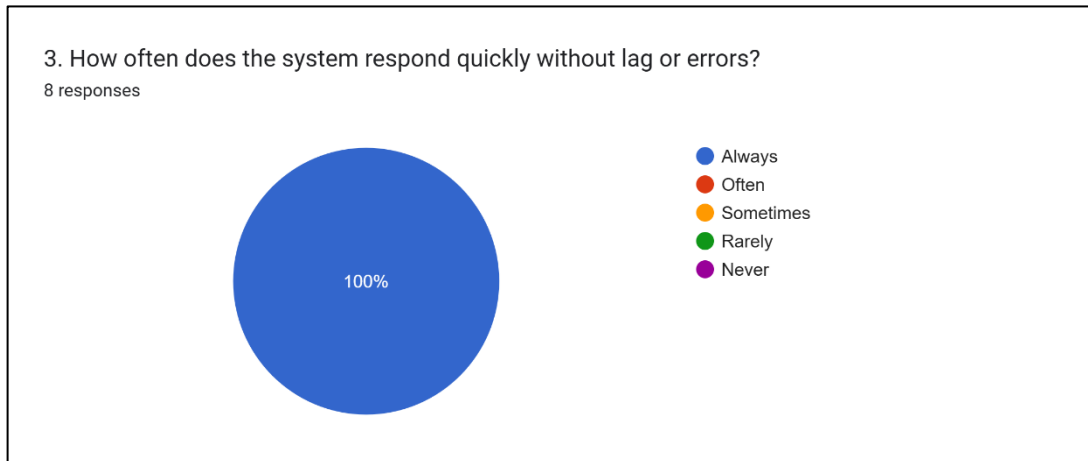
All respondents (100%) found the DIGIDINE system **very easy** to learn and use, showing excellent user-friendliness and accessibility.

Question #2:



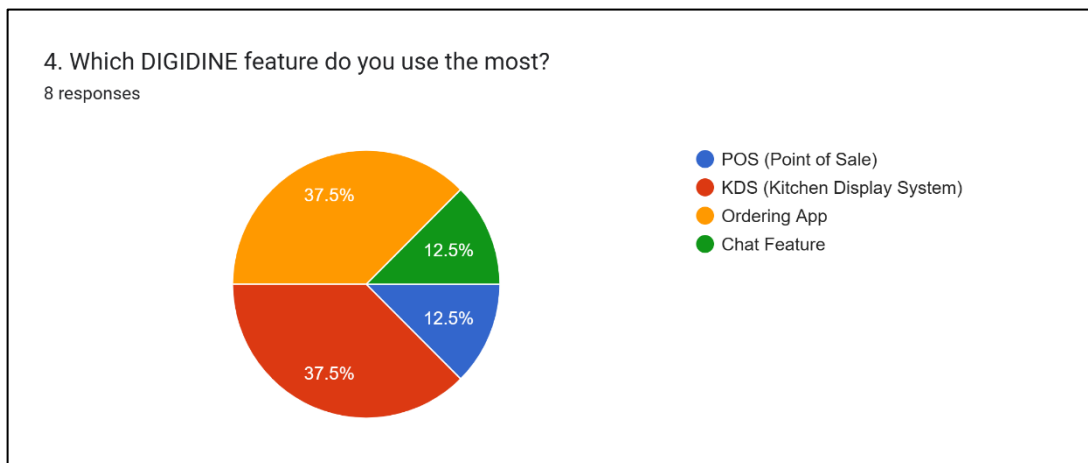
All respondents (100%) said DIGIDINE helps them **process customer orders very well**, showing excellent performance and reliability in order management.

Question #3:



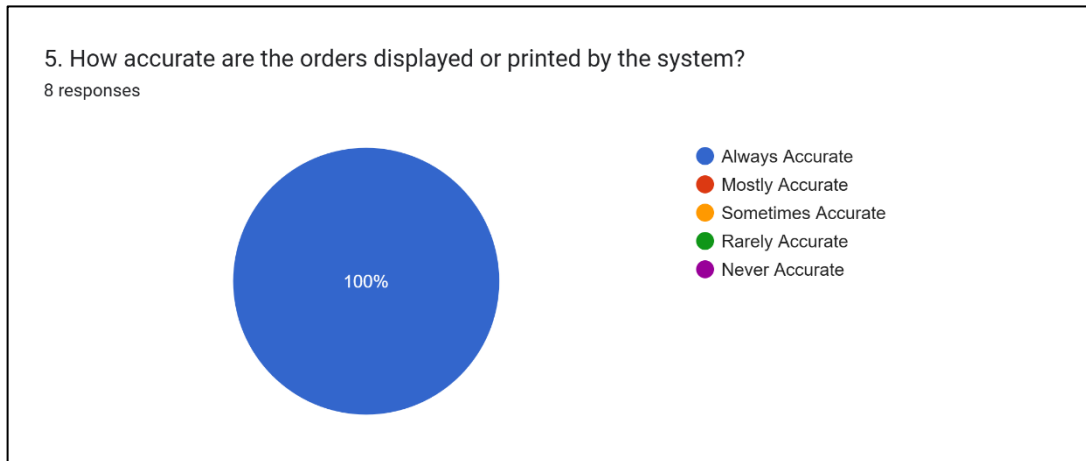
All employees (100%) reported that the system **always responds quickly without lag or errors**, showing excellent system reliability and performance.

Question #4:



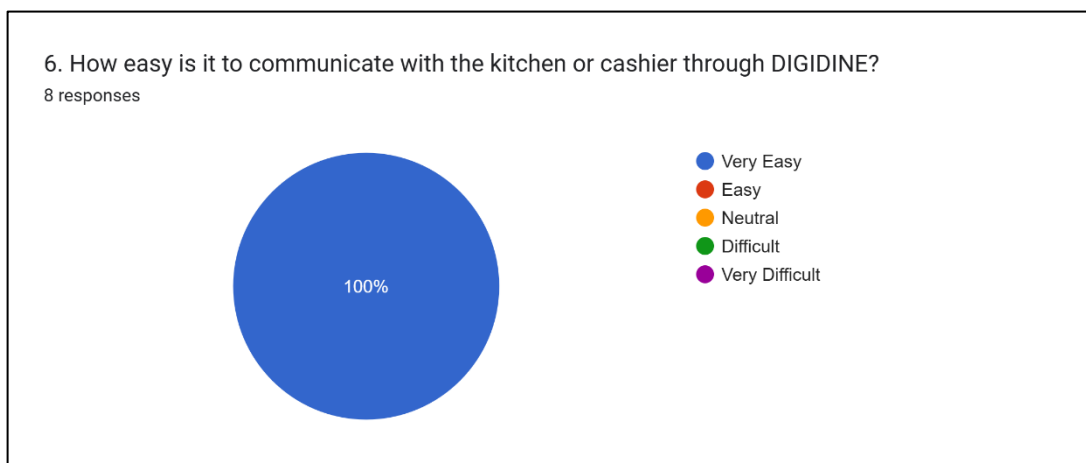
Employees mostly use the **KDS (37.5%)** and **Ordering App (37.5%)**, while **POS** and **Chat Feature** are used less, each at **12.5%**. This shows higher reliance on order management tools.

Question #5:



All employees (100%) stated that orders are **always accurate** when displayed or printed by the system, showing excellent reliability and precision.

Question #6:



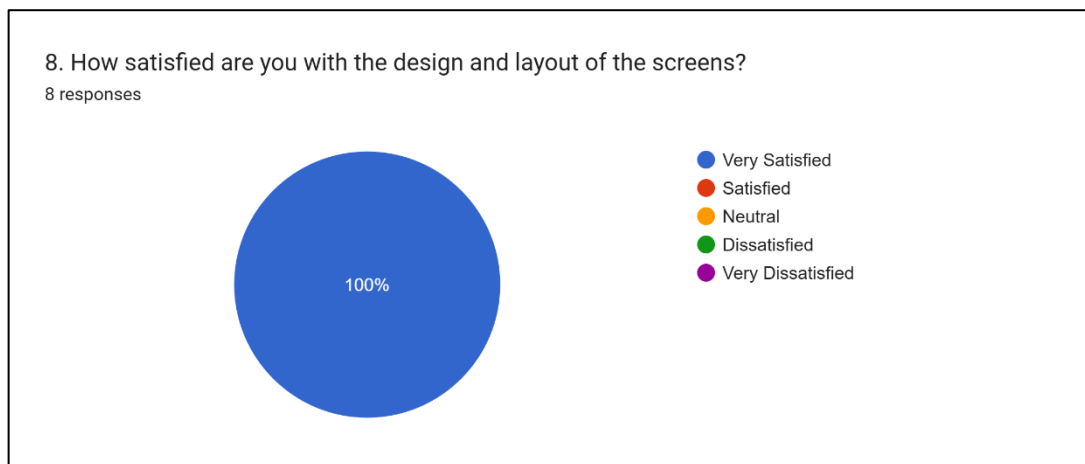
All employees (100%) found it **very easy** to communicate with the kitchen or cashier through DIGIDINE, indicating the system is **highly effective and user-friendly** for internal communication.

Question #7:



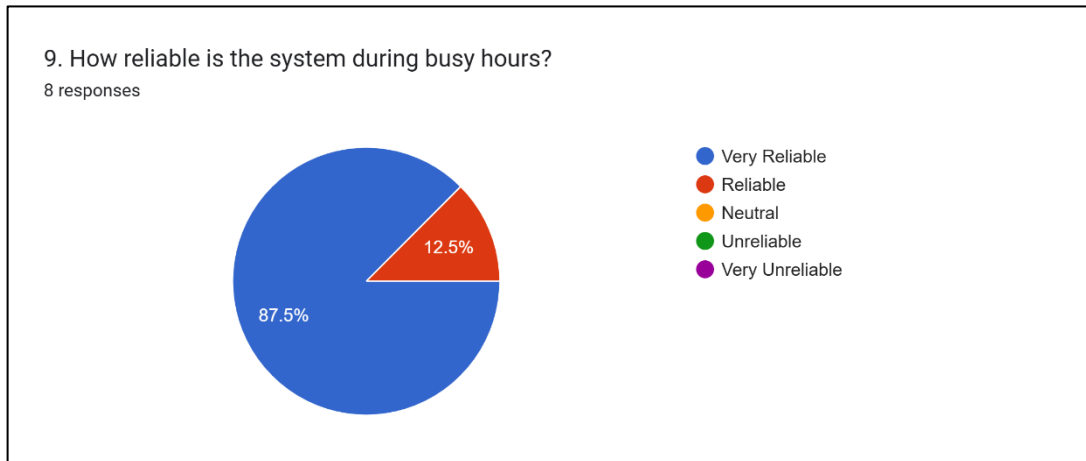
All employees (100%) rated DIGIDINE as **very helpful** in managing their daily tasks, showing that the system greatly improves **efficiency and task organization** in their workflow.

Question #8:



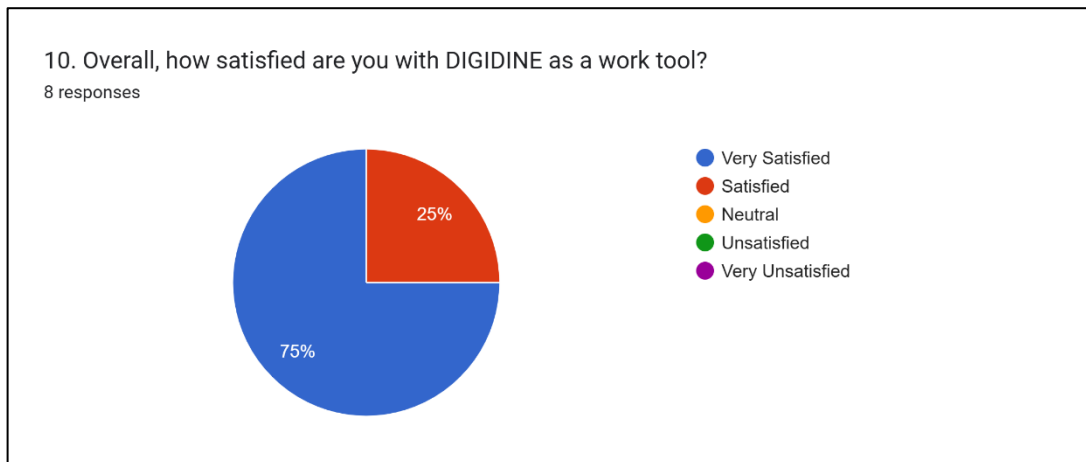
All employees (100%) reported being **very satisfied** with DIGIDINE's design and screen layout, indicating a **clear, visually appealing, and user-friendly interface**.

Question #9:



Most employees (87.5%) found DIGIDINE **very reliable** during busy hours, while 12.5% rated it **reliable**, showing that the system performs **consistently well even under high workload**.

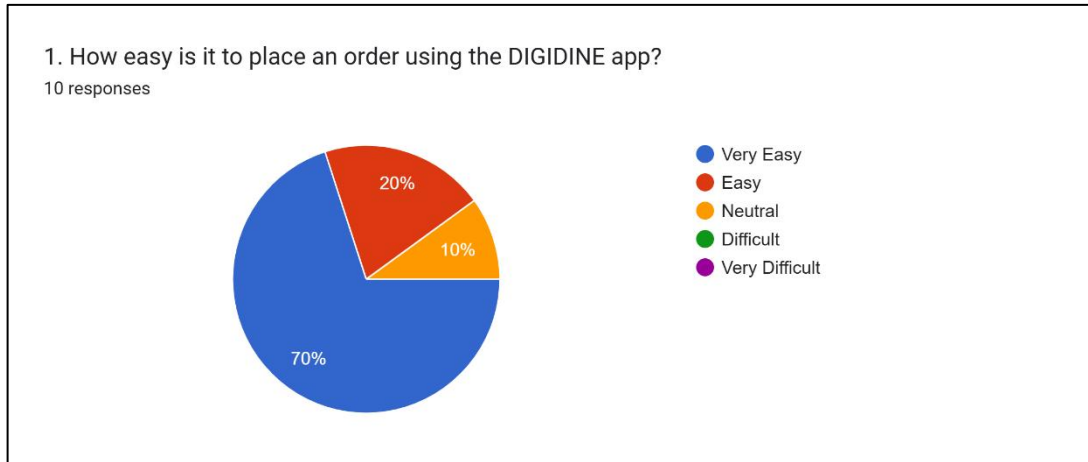
Question #10:



The chart shows that **75%** of respondents are **very satisfied** and **25%** are **satisfied** with DIGIDINE as a work tool, indicating strong overall satisfaction.

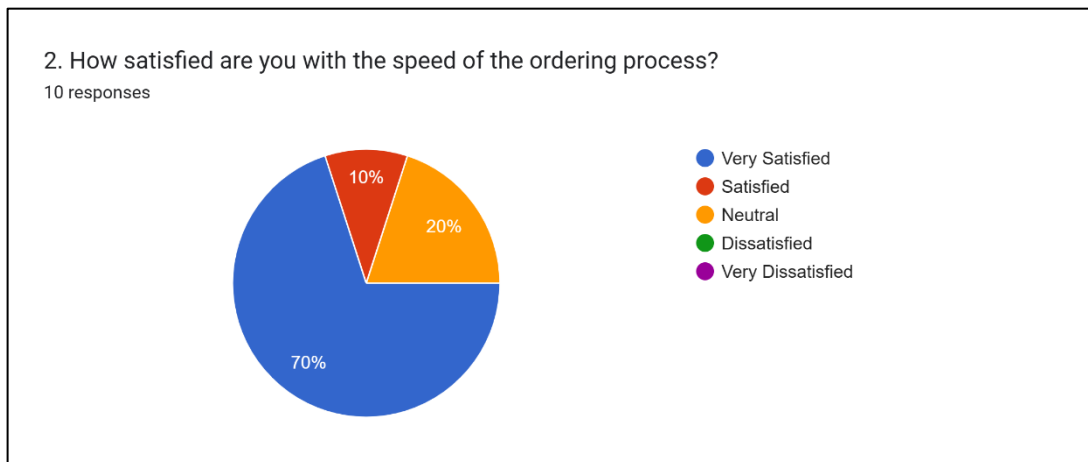
For Customers:

Question #1:



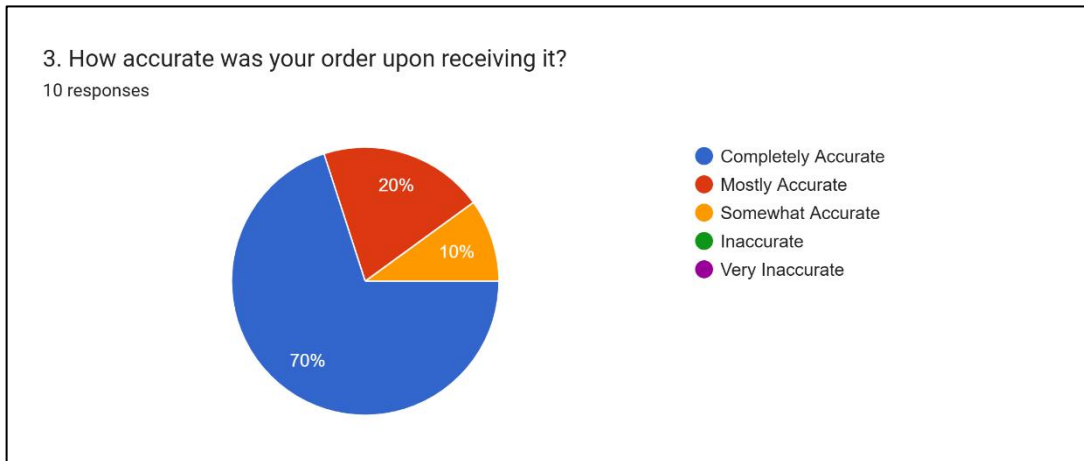
The chart shows that most customers find the DIGIDINE app **easy to use** — **70% said “Very Easy,” 20% “Easy,” and 10% “Neutral.”** None found it difficult, indicating a smooth ordering experience.

Question #2:



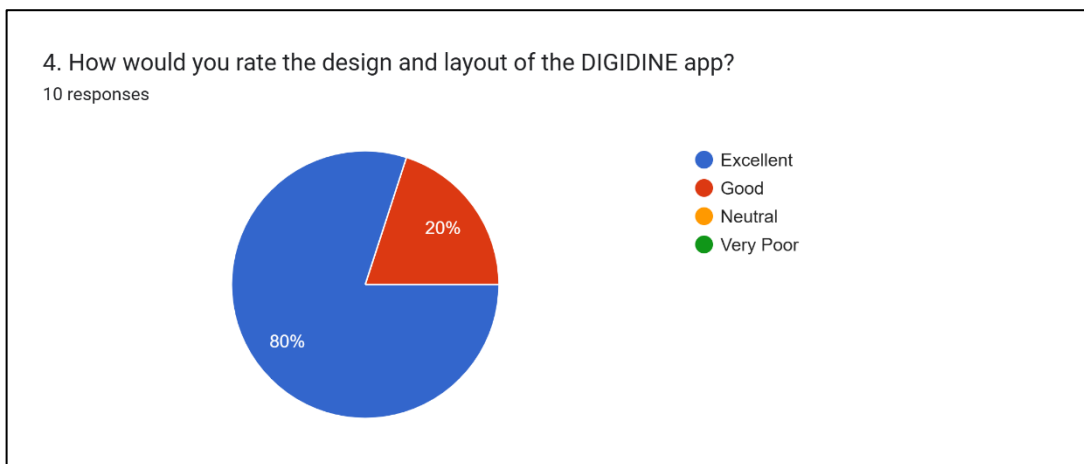
The chart shows that most customers are pleased with the ordering speed — **70% are “Very Satisfied,” 20% “Satisfied,” and 10% “Neutral.”** Overall, the process is viewed as fast and efficient.

Question #3:



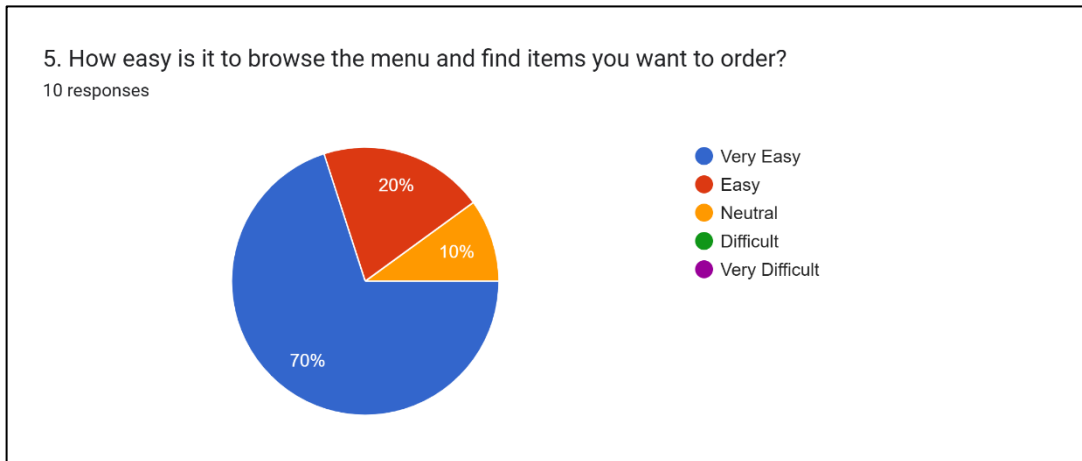
Most customers (70%) said their orders were **completely accurate**, 20% found them **mostly accurate**, and 10% said **somewhat accurate**, indicating that DIGIDINE provides **high order accuracy** with minor room for improvement.

Question #4:



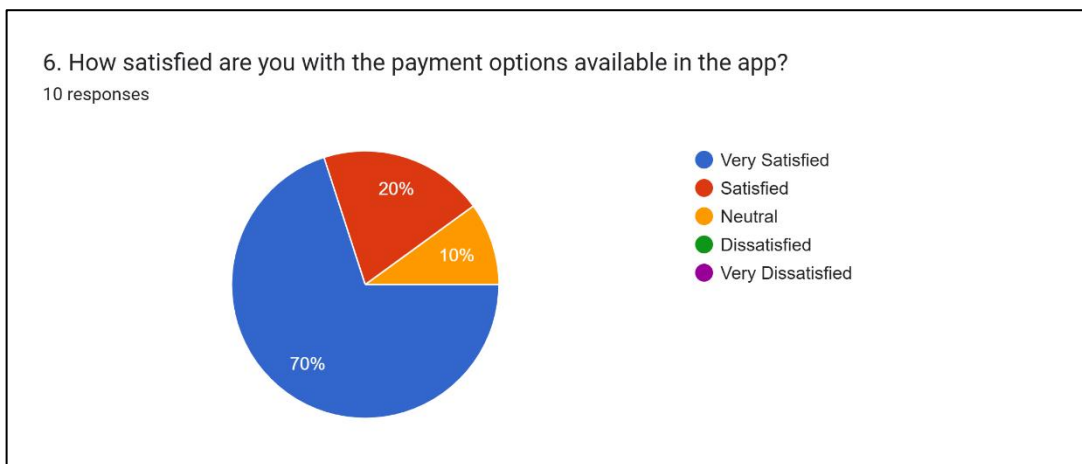
Most customers (80%) rated the DIGIDINE app's design and layout as **excellent**, while 20% rated it **good**, showing that users find the interface **visually appealing and easy to use**.

Question #5:



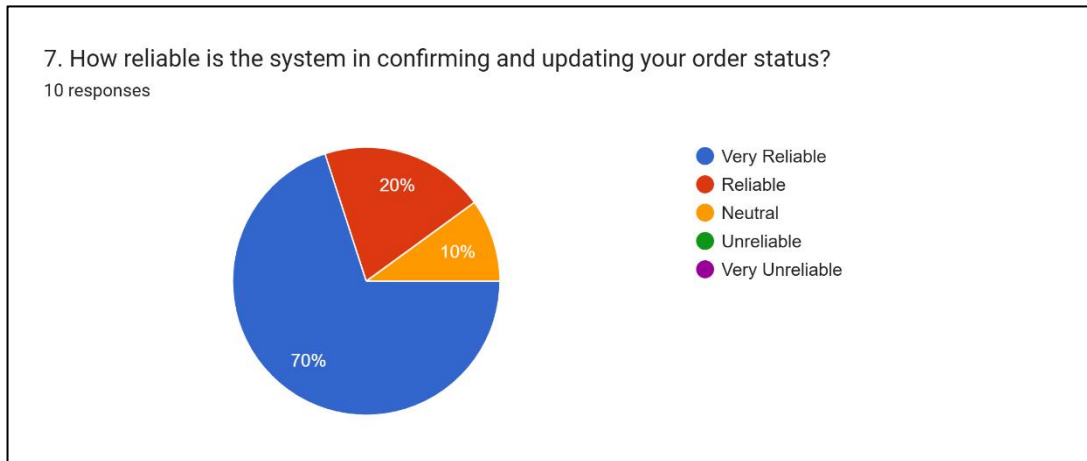
Most customers (70%) found it **very easy** to browse the menu and find items, 20% said it was **easy**, and 10% were **neutral**, showing that DIGIDINE offers a **smooth and user-friendly menu navigation experience**.

Question #6:



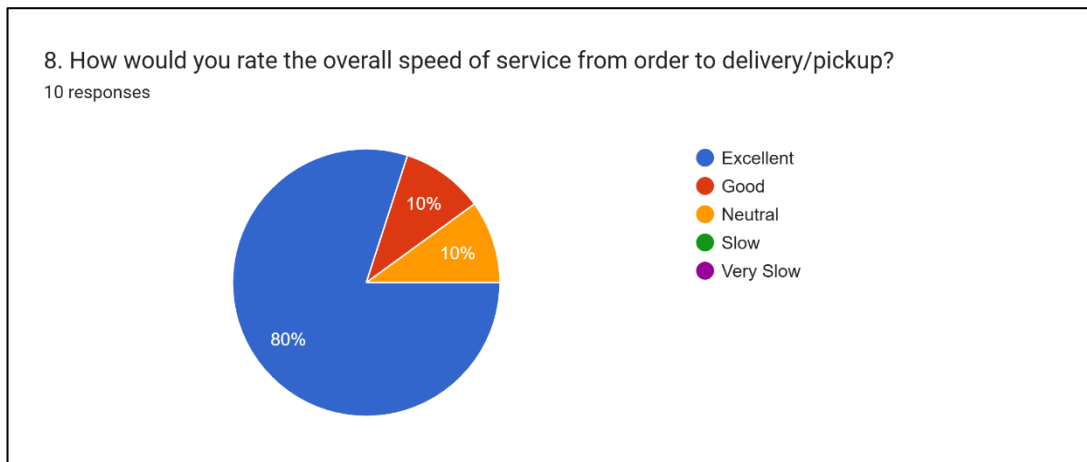
Most customers (70%) were **very satisfied** with DIGIDINE's payment options, 20% were **satisfied**, and 10% were **neutral**, showing that the app provides **convenient and reliable payment choices** for users.

Question #7:



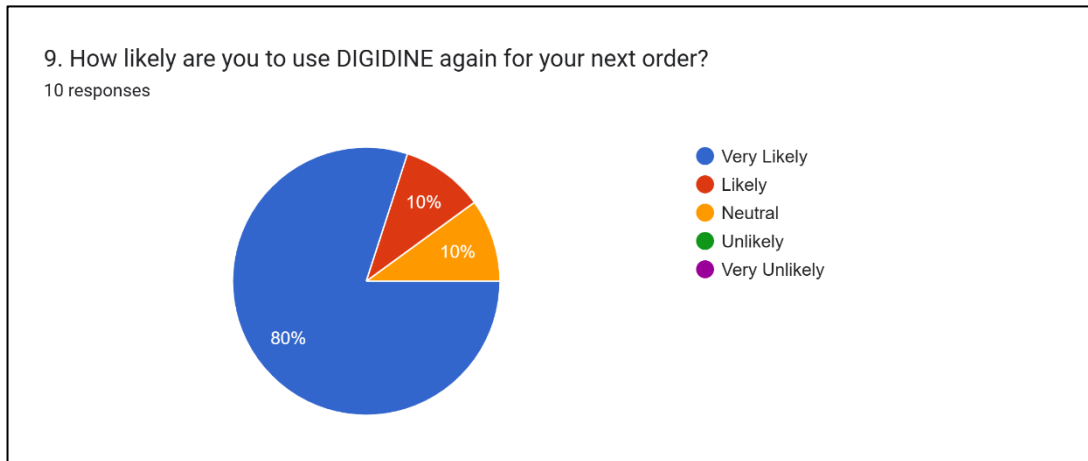
The majority of respondents (70%) found the system very reliable, while 20% rated it reliable and 10% neutral. This indicates that the system effectively confirms and updates order statuses for most users.

Question #8:



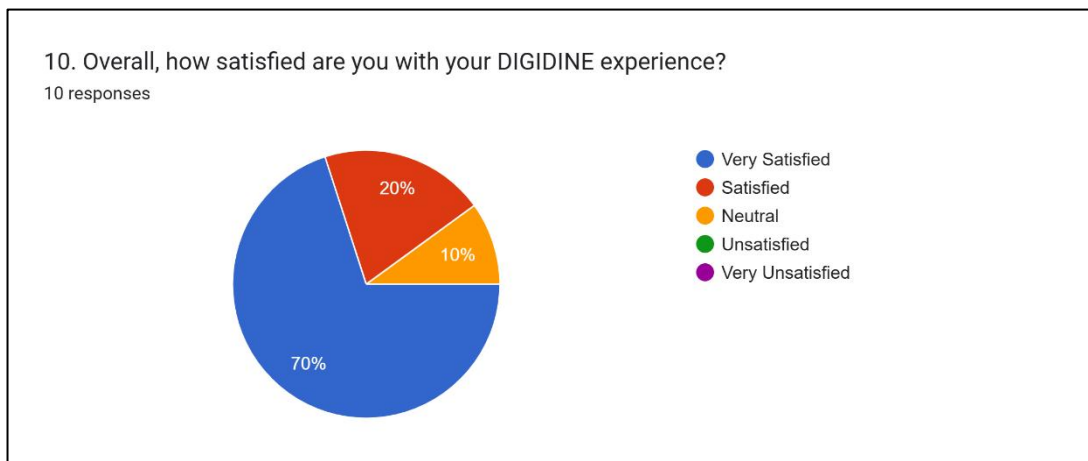
A strong majority (80%) rated the system's service speed as **excellent**, with 10% marking it as **good** and another 10% as **neutral**. This shows that customers are generally **very satisfied** with the speed of order processing and delivery.

Question #9:



The chart shows that **80%** of customers are **very likely** to use DIGIDINE again, **10% likely**, and **10% neutral**, indicating strong customer loyalty and satisfaction.

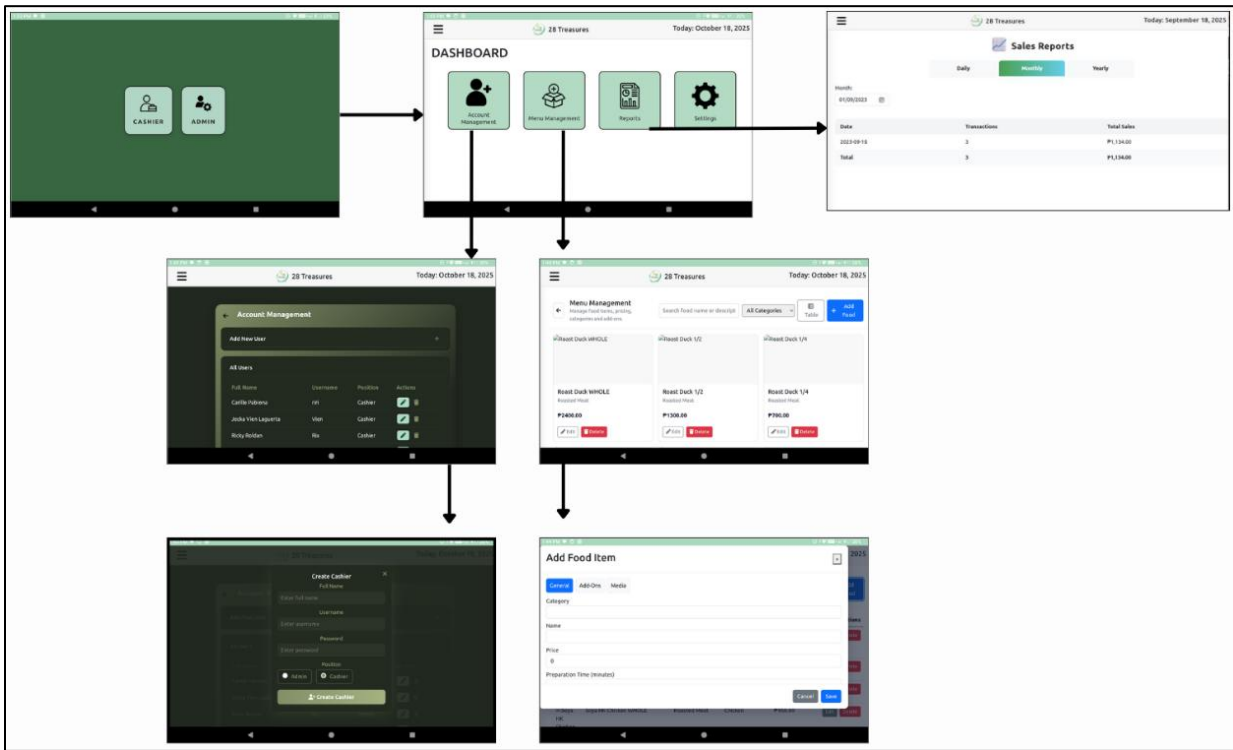
Question #10:



The chart shows that most respondents are highly satisfied with their DIGIDINE experience — **70% are “Very Satisfied,” 20% are “Satisfied,” and 10% are “Neutral.”** No one reported being unsatisfied. Overall, satisfaction is very high.

APPENDIX E. USER'S GUIDE

POS ADMIN APP



Admin Access

1. On the login screen, enter the Admin Password.
2. Tap Login as Admin to open the Admin Dashboard.

The Admin Dashboard includes the following modules:

- Account Management
- Menu Management
- Reports
- Setting

Admin Dashboard Features

1. Account Manageme

Used to handle staff accounts.

Features:

- Add New Staff Account – For new cashiers or employees.
- Add Admin Account – Create additional admin users.

- Edit Account – Update usernames, passwords, or roles.
- Delete Account – Remove inactive or old accounts

2. Menu Management

Used to organize all food and drink items available in the restaurant.

Features:

- Add Food Item – Create new menu entries with name, category, price, and image.
- Edit Food Item – Update prices, names, or details when needed.
- Delete Food Item – Remove items no longer offered.
- Add Customizations/Add-ons – Add options like sizes, toppings, or side dishes to improve ordering flexibility

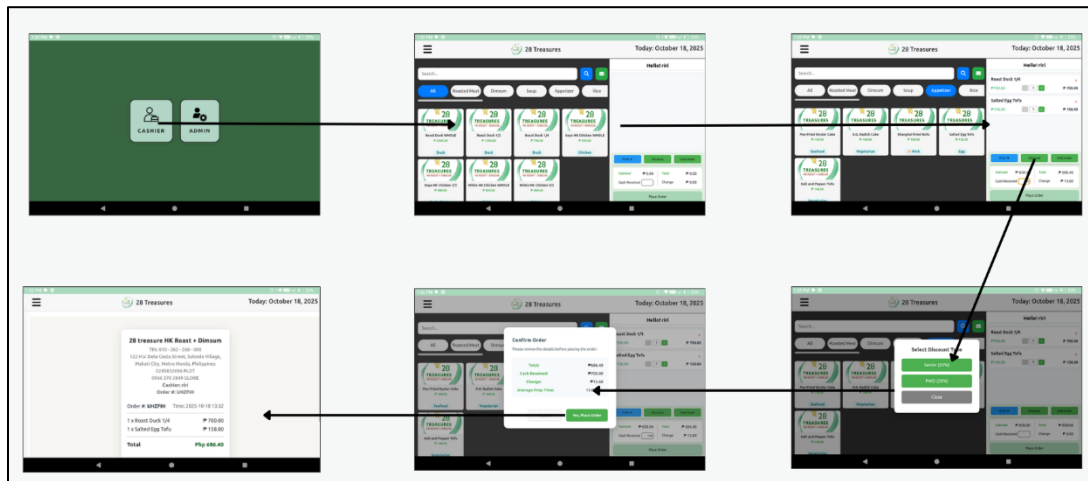
3. Reports

Monitor and analyze your restaurant's sales performance.

Report Types:

- Daily Sales Report – Shows all transactions made on the current day.
- Monthly Sales Report – Displays summarized sales for the entire month.
- Yearly Sales Report – Summarizes total annual sales performance.

POS CASHIER APP



Login

1. Open the POS Ordering App.
2. Enter Cashier Password.
3. Tap Login to enter the Cashier Dashboard

TAKE-OUT ORDER PROCESS

Step 1: Create a Take-Out Order

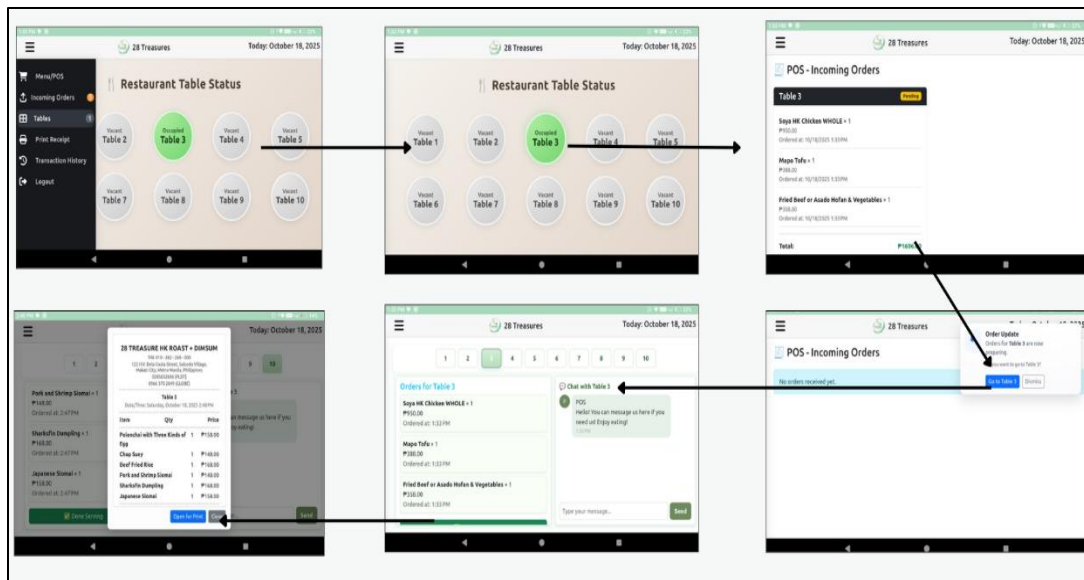
1. Tap “New Order” and choose Take-Out.
2. Browse through the menu and select the food items.
3. Add any customizations or add-ons if applicable.
4. Tap Add to Cart when done.

Step 2: Confirm and Process Payment

1. Review all items in the cart.
2. Ask if there’s a discounted ID’s
3. Tap Checkout to proceed.
4. Once payment is confirmed, tap Confirm Order.
5. The system will print or display a receipt for the customer.

Step 3: Order Flow

1. The order automatically appears in the Kitchen Display System (KDS).
2. The kitchen staff will prepare the order.
3. Once marked Done in KDS, the cashier can hand the order to the customer.



DINE-IN ORDER PROCESS

Step 1: Receive Customer Order

- For dine-in tables, orders are placed by the Staff using the Menu Ordering App (tablet or kiosk).
- Once the customer confirms their selection, the order automatically appears in the Cashier Dashboard.

Step 2: Mark as “Preparing”

- The cashier reviews the dine-in order.
- Tap Mark as Preparing.
- This sends the order to the Kitchen Display System (KDS) for preparation.
- The order status changes from “Pending” to “Preparing.”

Step 3: Customer Chat or Inquiry

- While the order is being prepared, the customer can chat or send inquiries using the Menu Ordering App.
- The cashier can respond or acknowledge these messages through the cashier tables dashboard.

Step 4: Billing Out Process

- When the customer is ready to pay, they tap the “Bill Out” button on their ordering app.
- The cashier receives a notification: “Customer is requesting to bill out.”
- The cashier then reviews the table’s order and ensures everything has been served.

- Tap Done Serving to close the order.
- This action will generate the final receipt view for the customer.
- The staff can now proceed with the billing process and accept payment

Menu Ordering App



Step 1: Start Ordering

1. On the screen, tap “Start Ordering.”
2. This action notifies the cashier that the table number is now occupied.
3. The customer can now browse the available food and drink items.

Step 2: Add Items to Cart

1. Browse the Menu Categories (e.g., Meals, Drinks, Desserts).
2. Tap an item to view details such as description, price, and available add-ons or customizations.
3. Choose quantity and tap “Add to Cart.”
4. Repeat this process until all desired items are added.

Step 3: Confirm and Place Order

1. Review all items in the Cart.
2. If satisfied, tap “Order Now” to confirm the order.
3. Once confirmed, the order is sent to the Cashier Dashboard and Kitchen Display System (KDS).
4. The order status changes from Pending to Preparing.

Step 4: Communicate with the Cashier

- Customers can send messages or inquiries to the cashier using the built-in Chat feature.

Examples:

- “Can I change my drink to Iced Tea?”
- “How long will my order take?”
- The cashier can respond directly from their dashboard.

Step 5: Canceling an Order

- If the customer changes their mind before preparation starts, they can tap “Cancel Order.”
- Once the cashier confirms the cancellation, the order will be removed from the active list.

Step 6: Track Order Status

- While waiting, the customer can view real-time order updates, such as:
- Pending: Waiting for cashier confirmation.
- Preparing: The kitchen is making your order.
- Ready to Serve: The food is on its way to your table.
- Served: Order has been delivered.

This helps customers know their order’s progress without calling the staff.

Step 7: Billing Out

1. When customers are ready to pay, they tap the “Bill Out” button.
2. The cashier receives a notification that the customer is requesting to pay.
3. The cashier reviews the order and taps “Done Serving.”
4. The final receipt appears on the customer’s screen, and the staff proceeds to collect payment.

APPENDIX F. PERSONAL TECHNICAL VITAE

Curriculum Vitae of
CARILLE B. PABIONA
 122 Scout De Guia St. Brgy. Sacred Heart, Quezon City
 carillepabiona49@gmail.com
 0956 600 2931

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022 – Present	STI College Cubao
Senior High School	2020 – 2022	San Francisco High School
High School	2016 – 2020	Quezon City High School
Elementary	2010 – 2016	Kamuning Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
2022-2023	Service Crew	McDonalds

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
2019-2020	Diwa ng Kabataan	Member

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Office (Word, PowerPoint, and Excel)	Proficient	2016-Present
Photo Editing (Adobe Photoshop, Canva, Adobe Lightroom)	Competent	2022 – Present
Programming Skills (HTML, Java, C#, C++, JavaScript, CSS, SQL)	Competent	2023 – Present
Mobile Development (.NET Maui)	Competent	2024 – Present
Version Control (Git, GitHub)	Excellent	2024 – Present

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
June 2023	Java Fundamentals
June 2023	Systems Administration

Curriculum Vitae of
JECKA VIEN R. LAGUERTA
 37 Kabalitang St. Krus na Ligas, Quezon City
 vienlaguertajecka@gmail.com
 0968 272 8828

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022 – Present	STI College Cubao
Senior High School	2020 – 2022	National College of Business and Arts
High School	2016 – 2020	Krus na Ligas High School
Elementary	2010 – 2016	Upper Bicutan Elementary School

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Office (Word, PowerPoint, and Excel)	Proficient	2022-Present
Editing (Canva, Figma)	Competent	2020 – Present
Programming Skills (HTML, Java, C#, C++, JavaScript, CSS, SQL)	Competent	2022 – Present
Mobile Development (.NET Maui)	Competent	2024 – Present
Version Control (Git, GitHub)	Competent	2023 – Present

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
June 2023	Java Fundamentals
June 2023	Systems Administration

Curriculum Vitae of
RICKY Z. ROLDAN
 2G F. Tuazon, Barnagka, Marikina City
 rickyzroldan@gmail.com
 0921 718 5745

EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2022 – Present	STI College Cubao
High School	2001 – 2005	Roosevelt College Marikina
Elementary	1995 – 2001	Barangka Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
2013-2022	Consular Personnel	Department of Foreign Affairs, Alimall, Cubao
2011 - 2013	Shakey's Pizza	Marcos Highway, Pasig City
2010 - 2011	Jollibee	Riverbanks, Marikina City

SKILLS

SKILLS	Level of Competency	Date Acquired
MS Office (Word, PowerPoint, and Excel)	Proficient	2010-Present
Photo Editing (Canva)	Competent	2022 – Present
Programming Skills (HTML, Java, C#, C++, JavaScript, CSS, SQL)	Competent	2023 – Present
Mobile Development (.NET Maui)	Competent	2024 – Present
Version Control (Git, GitHub)	Competent	2024 – Present

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
June 2023	Java Fundamentals
June 2023	Systems Administration

Curriculum Vitae of
JAN GABRIELLE M. INTOD
 Blk 45 Lot 37 Phase 2 Tierra Monte Silangan S.M.R
 gabrielleintod14@gmail.com
 0968 893 1403

EDUCATIONAL BACKGROUND

Tertiary	2022 – Present	STI College Cubao
Senior High School	2020 – 2022	Silangan National High School
High School	2016 – 2020	Silangan National High School
Elementary	2010 - 2016	St. Joseph Kiddie Center
Tertiary	2022 – Present	STI College Cubao

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
2021 - Present	Incoder	RM Creatv Events Services
2023 – Present	Project Liaison Officer	Phirst Park Homes Inc.
2019 – 2021	Cashier	Kaynette

SKILLS

SKILLS	Level of Competency	Date Acquired
Adobe Photoshop	Intermediate	2018 - Present
Programming Skill (Python, HTML, CSS, JAVA)	Proficient	2023 - Present
Google Workspace (Docs, Sheets, Slides)	Proficient	2015 – Present
MS Office (Word, PowerPoint, Excel)	Proficient	2015 - Present
Visual Studio 2022	Intermediate	2023 - Present

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
June 2023	Java Fundamentals
June 2023	Systems Administration